

РЕФЕРАТ

Выпускная квалификационная работа на тему «Автоматизация контроля качества данных в системе оценки страховых рисков» выполнена на 133 страницах, с использованием 32 рисунков, 18 таблиц, 23 источников и 5 приложений.

Ключевые слова: качество данных, автоматизация, страховые риски, кредитный рейтинг, валидация.

Цель работы – совершенствование системы оценки страховых рисков за счет разработки и внедрения модуля автоматизированного контроля качества данных.

Решаемая проблема: необходимость автоматизации контроля качества данных в условиях динамического роста объемов информации и регуляторных требований.

Формализация системы оценки страховых рисков и создание программного решения, призванного обеспечить валидацию входных данных в режиме реального времени, интеграцию с существующей системой скоринга и соответствие стандартам, – есть предмет рассмотрения данной квалификационной работы.

Объектом разработки является модуль Data Quality, который способен: автоматически проверять корректность, полноту и согласованность данных; интегрироваться с компонентами системы оценки рисков; предоставлять инструменты для мониторинга и анализа ошибок.

В процессе исследования используются: язык программирования Python, JavaScript-библиотека React.js версии 19.0.0, СУБД PostgreSQL 15 и Docker Compose.

ABSTRACT

The graduate qualification work on the topic «Automation of data quality control in the insurance risk assessment system» is completed on 133 pages, using 32 figures, 18 tables, 23 sources and 5 appendices.

Keywords: data quality, automation, insurance risks, credit rating, validation.

The purpose of the work is to improve the insurance risk assessment system by developing and implementing an automated data quality control module.

The problem being solved: the need to automate data quality control in the context of dynamic growth in information volumes and regulatory requirements.

Formalization of the insurance risk assessment system and creation of a software solution designed to ensure validation of input data in real time, integration with the existing scoring system and compliance with standards are the subject of this qualifying work.

The object of development is the Data Quality module, which is capable of: automatically checking the correctness, completeness and consistency of data; integrating with the components of the risk assessment system; providing tools for monitoring and error analysis.

The following were used in the research process: the Python programming language, the React.js JavaScript library version 19.0.0, the PostgreSQL 15 DBMS, and Docker Compose.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....	8
ВВЕДЕНИЕ.....	9
1 ЭВОЛЮЦИЯ СИСТЕМ ПРОВЕРКИ ДАННЫХ И СТАНДАРТЫ ИХ КАЧЕСТВА В БАНКОВСКОЙ СФЕРЕ	11
1.1 Понятие данных и качества.....	11
1.2 Исторический обзор систем проверки качества данных	12
1.2.1 Зарождение концепций качества данных: доцифровая эпоха.....	12
1.2.2 Ранние этапы автоматизации: 1950–1980-е годы	13
1.2.3 Становление Data Quality как дисциплины: 1990–2000-е годы	14
1.2.4 Современные подходы к обеспечению качества данных: от больших данных к ИИ-решениям (2010-е — настоящее время).....	16
1.3 Ключевые критерии качества данных.....	18
2 АКТУАЛЬНОСТЬ РАЗРАБОТКИ И ПОСТАНОВКА ЦЕЛИ.....	20
2.1 Актуальность разработки модуля Data Quality	20
2.2 Цели и задачи модуля	21
2.3 Назначение модуля	22
3 ОПИСАНИЕ АВТОМАТИЗИРУЕМЫХ БИЗНЕС-ПРОЦЕССОВ	23
3.1 Текущие бизнес-процессы до внедрения модуля обеспечения качества данных	23
3.1.1 Ручные процессы проверки данных.....	23
3.1.2 Проблемы и ограничения	23
3.2 Целевые бизнес-процессы после внедрения модуля обеспечения качества данных	25
3.2.1 Описание логики целевых бизнес-процессов	25
3.2.2 Техническое взаимодействие компонентов АС.....	28

4 ТРЕБОВАНИЯ К МОДУЛЮ	31
4.1 Функциональные требования	31
4.1.1 Требования к функциональным возможностям.....	31
4.1.2 Требования к проверкам DQ.....	33
4.1.3 Требования к графическому интерфейсу	36
4.1.4 Требования к алгоритмам обработки.....	44
4.2 Архитектурные требования.....	46
4.2.1 Системная архитектура решения и требования к интеграции	46
4.2.2 Модель данных.....	51
4.3 Нефункциональные требования	56
5 РАЗРАБОТКА МОДУЛЯ КАЧЕСТВА ДАННЫХ.....	59
5.1 Функции общей работы.....	59
5.1.1 Расчёт кредитного рейтинга.....	59
5.1.2 Управление проверками	61
5.1.3 История проверок.....	65
5.1.4 История действий администратора.....	71
5.1.5 Аналитика запросов	77
6 АНАЛИЗ ЭФФЕКТИВНОСТИ ВНЕДРЕНИЯ МОДУЛЯ АВТОМАТИЗАЦИИ КОНТРОЛЯ КАЧЕСТВА ДАННЫХ И ОЦЕНКА ЗНАЧИМОСТИ ИЗМЕНЕНИЙ ОСНОВНЫХ ПОКАЗАТЕЛЕЙ	81
6.1 Методология анализа.....	81
6.2 Методы расчета и интерпретация метрик	81
6.2.1 Точность данных.....	82
6.2.2 Скорость обработки запросов.....	85
6.2.3 Качество скоринга.....	89

6.2.4 Уровень автоматизации.....	93
6.2.5 Удовлетворенность клиентов	95
ЗАКЛЮЧЕНИЕ	100
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	101
ПРИЛОЖЕНИЕ А	104
ПРИЛОЖЕНИЕ Б.....	106
ПРИЛОЖЕНИЕ В	112
ПРИЛОЖЕНИЕ Г	113
ПРИЛОЖЕНИЕ Д	114

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

АС – автоматизированная система

БД – база данных. Упорядоченный набор структурированной информации, которая хранится в электронном виде в системе

ВИ – вариант использования (use case)

ОСАГО – обязательное страхование автогражданской ответственности

ПИ – пользовательская история (user story)

DQ – Data Quality (качество данных). Модуль проверки качества данных входящей котировки

Администратор – сотрудник страховой компании, которому доступны функции: управление сервисом DQ, просмотр служебной информации DQ.

Индекс NPS (Net Promoter Score) – индекс определения приверженности потребителей услуге или компании, используется для оценки готовности к повторным покупкам.

Кредитный рейтинг (скоринг) – количественная оценка кредитоспособности физического или юридического лица, основанная на анализе финансовых, поведенческих и демографических данных. В контексте системы – числовой балл: чем он выше, тем надёжнее клиент и ниже риск страхового события.

Пользователь – сотрудник страховой компании/клиент, которому доступна функция просмотра служебной информации сервиса DQ.

Страховая котировка – ставка премии, по которой страховщик готов принять на страхование соответствующий риск. В контексте системы – входящая заявка на оценку по страховому продукту.

Business Process Model and Notation (BPMN) – система условных обозначений и их описания в XML для моделирования бизнес-процессов.

ВВЕДЕНИЕ

Постепенное развитие общественного производства, его постоянное совершенствование являются фундаментальными закономерностями экономической жизни человечества. Они основываются на прогрессе науки и техники.

Страховая отрасль находится в процессе активной цифровой трансформации, где ключевую роль играют данные, лежащие в основе оценки рисков и формирования страховых продуктов. Точность прогнозирования страховых случаев, расчет тарифов и управление финансовыми рисками напрямую зависят от качества исходных данных. С ростом объема и сложности информации, поступающей из разнородных источников, возникают значительные трудности, связанные с корректностью, полнотой и соответствием данных установленным требованиям.

Особую актуальность проблема контроля качества данных приобретает в контексте АС оценки кредитоспособности (кредитных рисков) лица, где даже незначительные ошибки или пропуски в данных могут привести к искажению кредитного рейтинга, а следовательно – к финансовым потерям, репутационным рискам и снижению доверия клиентов. Несмотря на внедрение современных аналитических инструментов, многие процессы валидации данных до сих пор остаются рутинными, что замедляет обработку запросов и увеличивает вероятность человеческих ошибок.

Целью данного исследования является совершенствование системы оценки страховых рисков за счет разработки и внедрения модуля автоматизированного контроля качества данных.

Создаваемый модуль должен решать следующие задачи:

- валидация входящих котировок, проверка их соответствия эталонным схемам;
- мониторинг обязательных атрибутов и выявление аномалий в режиме реального времени;
- минимизация рисков, связанных с некорректными данными;

- повышение точности скоринговых моделей и оптимизация процессов их интеграции в соответствии с требованиями, предъявляемыми к качеству данных в страховой отрасли.

С этой целью запланирована следующая последовательность исследовательских работ для реализации цели и соответствующих задач:

- изучение подходов к обеспечению качества данных в страховой отрасли;
- обзор методов автоматизации валидации и их адаптация к специфике страховых продуктов;
- разработка архитектуры модуля;
- интеграция создаваемого модуля с существующими компонентами системы;
- оценка значимости изменений ключевых показателей после внедрения модуля.

Ожидаемым практическим результатом работы станет внедрение модуля, способного сократить время обработки котировок, уменьшить долю ошибок в данных и повысить прозрачность процессов оценки рисков. Решение ориентировано не только на техническую оптимизацию, но и на создание стандартов качества данных, что соответствует стратегическим целям развития цифровых страховых платформ.

1 ЭВОЛЮЦИЯ СИСТЕМ ПРОВЕРКИ ДАННЫХ И СТАНДАРТЫ ИХ КАЧЕСТВА В БАНКОВСКОЙ СФЕРЕ

1.1 Понятие данных и качества

Термин «данные» происходит от латинского слова *datum* (букв. «нечто данное») [1]. В общем смысле данные представляют собой информацию, особенно факты или числа, собранные для анализа и принятия решений, или информацию в электронной форме, хранимую и обрабатываемую компьютером [2]. Значение данных зависит от контекста: например, исследователи часто работают с наборами данных – структурированными коллекциями, обычно представленными в табличной форме.

Однако данные не тождественны информации. Клод Шеннон определял информацию как единицу сообщения (например, бит) [3], тогда как данные носят технический характер и требуют интерпретации для превращения в информацию [4, 5]. Таким образом, информация – это осмысленные данные, готовые для использования.

Важнейшим аспектом работы с данными является их качество. Согласно ISO 9000:2015, качество (лат. *qualitas* – характеристика, природа) – это «степень соответствия характеристик объекта требованиям» [6]. Применительно к данным качество отражает их пригодность для решения конкретных задач.

На основе представленных выше терминов можно сформулировать определение качества данных как степень соответствия данных установленным требованиям. Однако в литературе предлагаются и более широкие трактовки: данные должны быть пригодны для целевого использования, свободны от недостатков [7] и удовлетворять потребности пользователей [8, 9]. При этом требования к качеству данных могут определяться стандартами, законодательством, политиками организации, заинтересованными сторонами или целями применения данных [10].

Особое значение качество данных приобретает в регулируемых сферах, таких как банковская. Банк России устанавливает строгие требования к

данным в информационных системах кредитных организаций. В разделе 1.3 данной работы подробно рассматриваются критерии, которым должны соответствовать качественные данные в банковском секторе.

1.2 Исторический обзор систем проверки качества данных

Эволюция методов контроля качества данных отражает технологический прогресс и меняющиеся бизнес-потребности. Изучение этой истории помогает понять, как современные инструменты унаследовали принципы прошлого и какие сложности остаются актуальными.

1.2.1 Зарождение концепций качества данных: доцифровая эпоха

Контроль качества данных существовал задолго до появления цифровых технологий. В 1930–1940-х годах банковская система СССР функционировала в условиях плановой экономики. Все операции, включая выдачу кредитов, учет вкладов и контроль денежного обращения, выполнялись вручную. Основными носителями информации были бумажные документы, учётные книги и архивы. Уже тогда предприятия, особенно в банковской и страховой сферах, сталкивались с проблемами ошибок в записях, их дублирования и неполноты [11].

Ключевыми методами контроля качества данных в этот период были:

- визуальная верификация – сверка записей несколькими сотрудниками;
- формализованные шаблоны – использование стандартизированных бланков для минимизации разночтений;
- статистические выборки – выборочная проверка данных на репрезентативность.

Несмотря на эффективность ручных методов, их трудоемкость и высокая вероятность ошибок создали предпосылки для автоматизации – следующего этапа эволюции контроля качества данных.

1.2.2 Ранние этапы автоматизации: 1950–1980-е годы

В 1950-х годах в СССР началось внедрение первых электронно-вычислительных машин (ЭВМ), таких как «Минск-1» (представлена на рисунке 1*) и «Урал-1». Эти устройства использовали перфокарты и магнитные ленты для хранения информации, что позволило автоматизировать рутинные операции: начисление процентов по вкладам и формирование балансов [12].

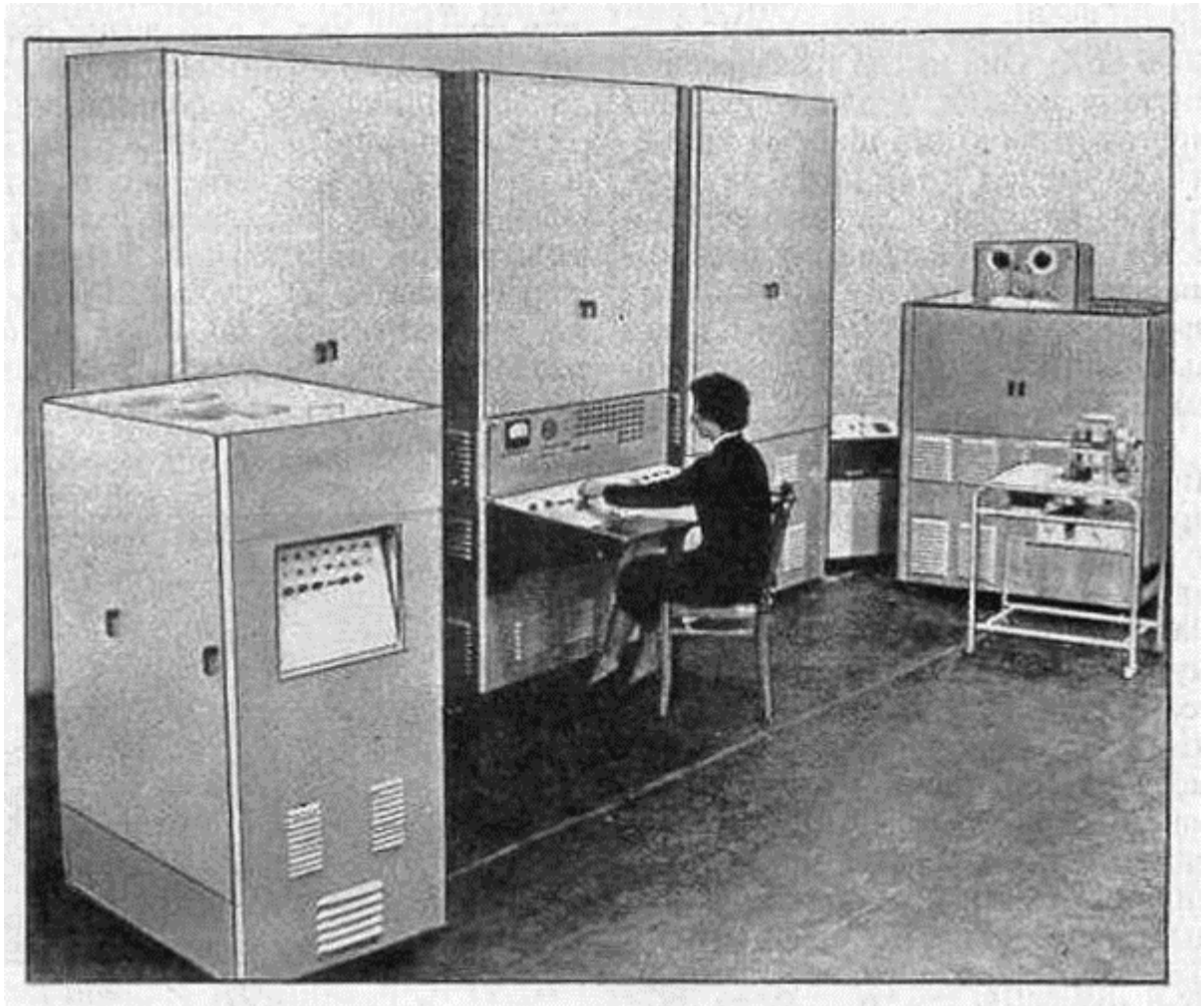


Рисунок 1 — Компьютер первого поколения «Минск-1»

Особенности проверки качества данных в этот период:

- данные вводились раз в сутки, а ошибки исправлялись операторами вручную;
- алгоритмы проверяли корректность номеров счетов и дат операций.

* Рисунок 1 заимствован из журнала «Наука и жизнь», №11, 1958 год.

В 1970-х годах в СССР была разработана Автоматизированная Система Управления Госбанком. Структурная схема системы «Банк» изображена на рисунке 2*. Она объединила филиалы в единую сеть, что позволило обрабатывать данные в режиме, близком к реальному времени [12].

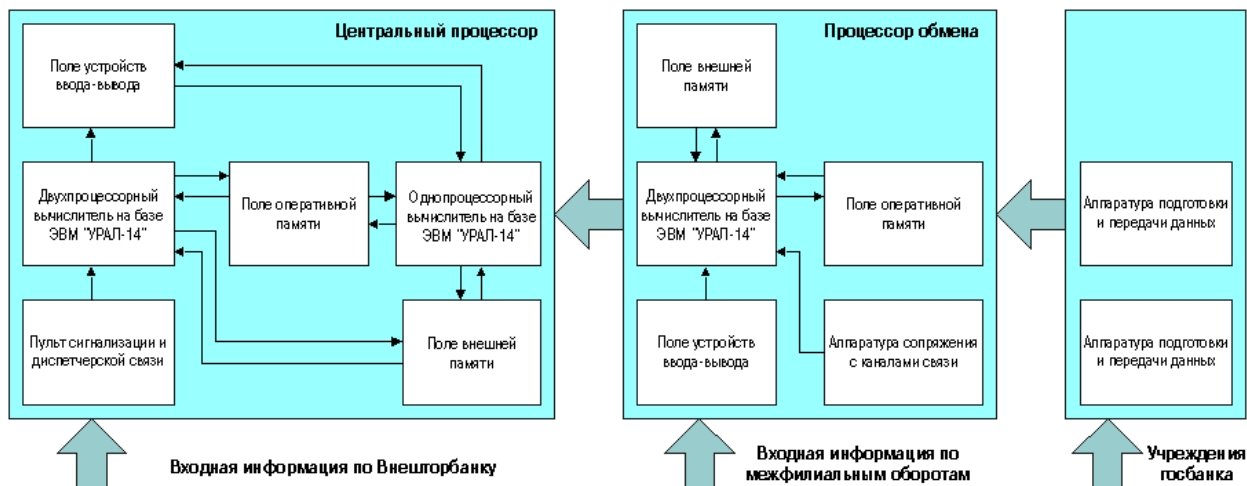


Рисунок 2 — Структурная схема системы «Банк»

В этот период сформировались базовые принципы контроля качества:

- валидация на уровне ввода – проверка форматов данных при заполнении полей;
- выявление дублей;
- аудит изменений – журналирование операций для отслеживания истории изменений.

Ограниченность ранних ЭВМ и отсутствие единых стандартов замедляли развитие систем, но заложенные принципы (валидация, аудит изменений) стали основой для становления Data Quality как самостоятельной дисциплины в 1990-х.

1.2.3 Становление Data Quality как дисциплины: 1990–2000-е годы

После распада СССР в 1991 году банковская система столкнулась с необходимостью адаптации к международным стандартам. Появились коммерческие банки, а Госбанк трансформировался в Центральный Банк

* Рисунок 2 заимствован с сайта История банковских систем. Компьютерный музей. URL: <https://www.computer-museum.ru/histsoft/bank1.htm> (дата обращения: 11.04.2025).

России [13]. С распространением корпоративных хранилищ данных (Data Warehouse), контроль качества данных выделился в отдельную дисциплину. Рост объемов информации и появление регуляторных требований (Базель II, ФАТФ) заставили компании внедрять специализированные инструменты.

Основные достижения этого периода:

- ETL-процессы (Extract, Transform, Load) – встроенные механизмы очистки данных при переносе между системами (представлены на рисунке 3*) [14];
- data profiling – автоматический анализ распределений, частот и аномалий в данных;
- стандарты качества – формализация метрик, таких как точность, полнота, согласованность.

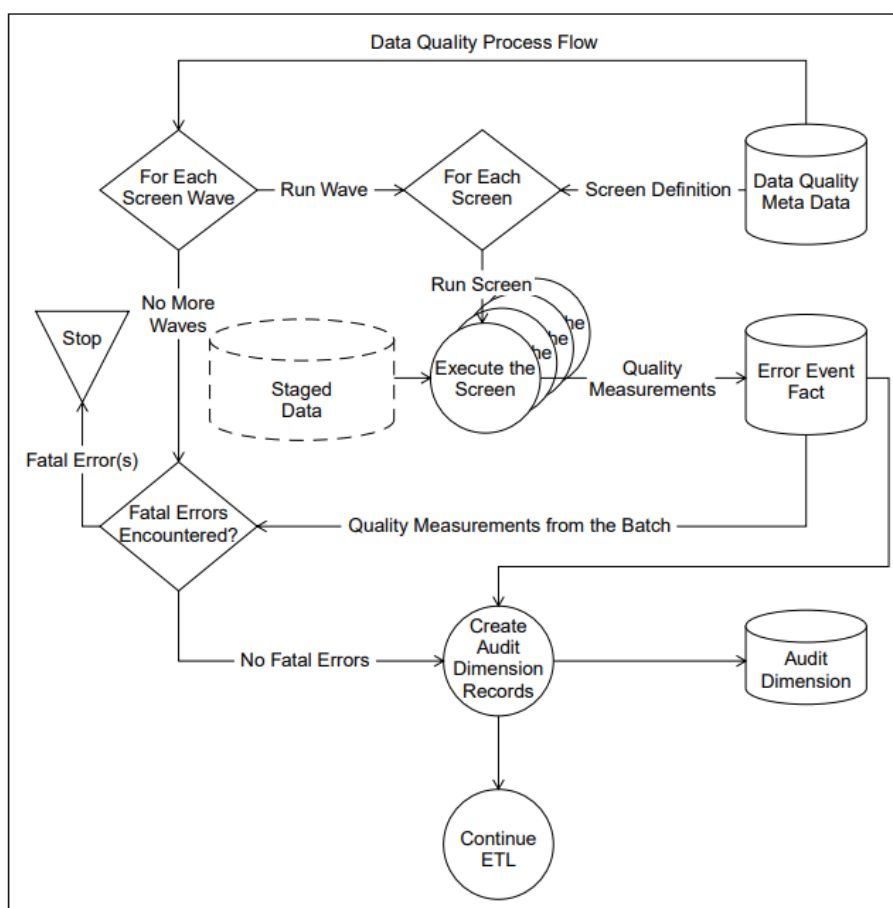


Рисунок 3 — Схема ETL-процесса с этапами проверки качества данных

* Рисунок 3 заимствован из книги Кимбалла Р. «Инструментарий ETL для хранилищ данных». Wiley, 2002. С. 137.

Ключевые технологии:

- инструменты от компаний Informatica, IBM, SAS – первые платформы для мониторинга качества данных [15];
- методологии TDQM (Total Data Quality Management) – адаптация принципов TQM к данным [16].

Несмотря на значительный прогресс в стандартизации и автоматизации процессов контроля качества данных, достигнутый к началу 2000-х годов, перед отраслью встали новые масштабные вызовы. Экспоненциальный рост объемов данных, необходимость их обработки в режиме реального времени и усложнение бизнес-процессов потребовали перехода от традиционных ETL-решений к интеллектуальным системам нового поколения. Однако активное внедрение технологий искусственного интеллекта (далее ИИ) и машинного обучения, ознаменовавшее новый этап в эволюции DQ, стало возможным лишь в 2010-х годах благодаря развитию вычислительных мощностей и алгоритмов.

1.2.4 Современные подходы к обеспечению качества данных: от больших данных к ИИ-решениям (2010-е — настоящее время)

С 2010-х годов российские банки начали активно использовать искусственный интеллект и облачные платформы. Например, ЦБ РФ запустил систему «БЭСП» для мгновенных платежей, где проверка данных (валидация, блокировка ошибок, сопровождение транзакций аудитом) происходила в реальном времени [17].

Современные подходы включают:

- интеллектуальный анализ данных – алгоритмы машинного обучения (Isolation Forest, Autoencoders) [18] позволяют выявлять сложные аномалии и скрытые паттерны, которые невозможно обнаружить традиционными методами;

- data observability – непрерывный мониторинг данных в режиме реального времени с использованием дашбордов (представлен на рисунке 4*) [19];
- графовые базы данных – анализ связей между сущностями для выявления скрытых ошибок (дубли, противоречия, корреляции).

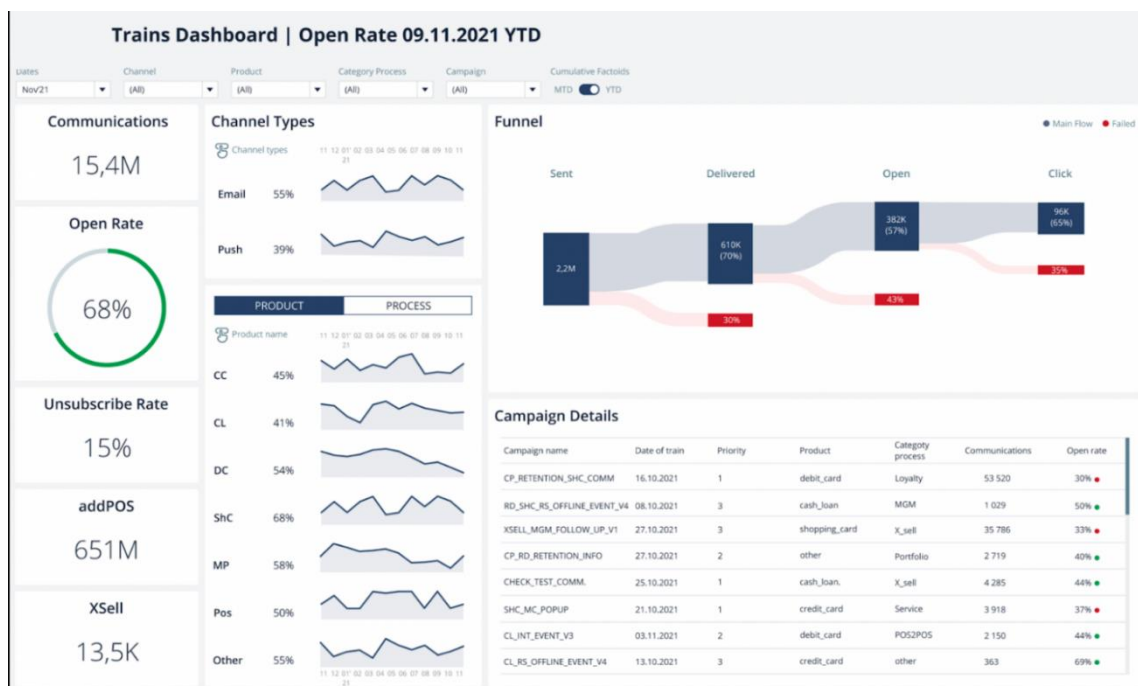


Рисунок 4 — Дашборд одного из банков России

Параллельно с технологическим прогрессом развивались и нормативные рамки:

- ISO 8000 – международный стандарт для качества данных [20];
- DCAM (Data Management Capability Assessment Model) – модель зрелости управления данными [21];
- GDPR (статьи 15-20) – право на исправление и удаление некорректных данных [22].

История развития систем контроля качества данных демонстрирует переход от ручных методов сверки в СССР к подходам, основанным на автоматизации и ИИ-решениях в современной России.

* Рисунок 4 заимствован из проекта компании T1A. URL: <https://t1a.com> (дата обращения: 08.05.2025).

Если в начале пути качество данных обеспечивалось человеческой внимательностью и стандартизированными бланками, то сегодня это результат симбиоза передовых технологий – машинного обучения, графовых баз данных и сквозного мониторинга. Особенно ярко эта трансформация проявилась в банковской сфере, где данные стали не просто учетными записями, а стратегическим активом, от качества которого зависят финансовая стабильность и доверие клиентов.

Качество данных перестало быть технической задачей, превратившись в философию управления информацией, где технологии – лишь инструменты для достижения главной цели – создания доверенной цифровой среды для бизнеса и общества.

1.3 Ключевые критерии качества данных

Согласно Приложению 3 Положения Банка России №483-П «О порядке расчета кредитными организациями величины кредитного риска» [23], устанавливаются нормативные требования к качеству данных, используемых кредитными организациями для разработки и применения моделей количественной оценки кредитного риска при расчете нормативов достаточности капитала. Данные требования направлены на обеспечение надежности моделей и минимизацию системных ошибок в оценке финансовых рисков. Ключевые критерии качества данных включают:

1. Точность и достоверность данных.

Данные должны быть свободны от синтаксических (форматных) и семантических (смысловых) ошибок. Их значения обязаны соответствовать реальным параметрам объектов учета, а также статистически обоснованным диапазонам, характерным для анализируемых процессов.

2. Полнота данных.

Критерий подразумевает три аспекта:

- объем данных – достаточное количество записей для статистической значимости выборки;

- глубина данных – период данных, необходимый для создания и применения моделей оценки риска;
- широта данных – включение всех разрезов, свойств и характеристик объектов, к которым применяются модели оценки риска.

3. Актуальность данных.

Данные должны фиксироваться и обновляться в соответствии с требованиями моделей оценки риска.

4. Согласованность данных.

Информация, хранящаяся во внутренних информационных системах банка, а также во внешних источниках, должна быть взаимно непротиворечивой. Особое внимание уделяется целостности идентификационных ссылок в базах данных, что исключает ошибки при сопоставлении записей.

5. Доступность данных для обработки.

Данные должны быть представлены в формате, совместимом с используемыми аналитическими инструментами. Это включает структурированность, поддержку стандартов передачи (XML, JSON) и отсутствие технических ограничений для интеграции в модели.

6. Контролируемость данных.

В информационных системах должна быть обеспечена возможность отслеживания происхождения данных, включая: источники их получения, журналирование изменений (история модификаций, удалений), протоколирование передачи между системами.

7. Восстанавливаемость данных.

Кредитная организация обязана обеспечить сохранение функциональности и качества данных в случае их утраты или повреждения из-за сбоев, кибератак или человеческих ошибок. Реализуется через: резервное копирование, механизмы отката транзакций (rollback), ведение логов изменений.

2 АКТУАЛЬНОСТЬ РАЗРАБОТКИ И ПОСТАНОВКА ЦЕЛИ

2.1 Актуальность разработки модуля Data Quality

Разработка и внедрение модуля Data Quality в АС прогнозирования страховых рисков обусловлены рядом факторов, отражающих вызовы страховой отрасли и потребности в цифровой трансформации процессов управления данными.

Во-первых, скоринговые модели, используемые для оценки рисков и расчета тарифов, критически зависят от точности, полноты и согласованности входных данных. Однако увеличение объемов информации сопровождается ростом ошибок, пропусков и некорректных форматов. Модуль DQ устраняет эти риски за счет автоматической валидации, что особенно актуально в условиях, когда даже незначительные погрешности данных могут привести к искажению скорингового балла и, как следствие, к финансовым потерям.

Во-вторых, страховые компании обязаны соблюдать стандарты качества данных, установленные регуляторами (например, Solvency II, GDPR). Ручные методы проверки не обеспечивают прозрачности и воспроизводимости процессов, что повышает риски штрафов и репутационных потерь. Модуль DQ формализует процедуры контроля через логирование проверок, гарантируя соответствие законодательству.

В-третьих, ручная проверка данных сотрудниками занимает до 30 % времени обработки страховых заявок. Автоматизация валидации с помощью модуля DQ позволит: сократить время обработки котировок на 70–80 %; минимизировать человеческие ошибки (до 2 % от общего числа проверок); ускорить принятие решений благодаря мгновенному выявлению аномалий.

Таким образом, актуальность модуля Data Quality обусловлена его ролью в решении наиболее существенных проблем страхового бизнеса: снижение финансовых и регуляторных рисков, повышение операционной эффективности, подготовка инфраструктуры для внедрения инновационных технологий. Его использование не только оптимизирует текущие процессы, но

и сформирует основу для устойчивого развития системы оценки рисков в условиях цифровой экономики.

2.2 Цели и задачи модуля

Главной целью разработки и внедрения модуля обеспечения качества данных для АС прогнозирования страховых рисков является повышение точности и надежности прогнозов, а также улучшение работы с данными, используемыми в процессе оценки рисков. Этот модуль направлен на минимизацию ошибок, связанных с некорректным или неполным вводом информации, и на обеспечение соответствия данных стандартам качества, чтобы улучшить точность предсказаний и снизить финансовые риски.

Также целями внедрения модуля DQ в АС являются:

- автоматизация процесса обработки и проверки данных, которая снизит нагрузку на сотрудников и минимизирует вероятность ошибок при вводе и передаче данных;
- повышение качества данных, включая их валидацию и стандартизацию, это повлияет на точность прогнозирования страховых рисков;
- упрощение контроля за качеством данных, с помощью инструментов для мониторинга и анализа состояния данных на различных этапах их обработки;
- оперативное решение проблем с данными, благодаря возможности быстрого исправления ошибок или недочетов в информации, что значительно ускорит процесс принятия решений.

Задачи модуля DQ:

1. Реализовать набор автоматических проверок, включая контроль структуры данных, проверку корректности значений и соблюдение логических взаимосвязей между атрибутами.
2. Обеспечить взаимодействие модуля DQ с другими сервисами (Orchestrator, Observability, UI) и хранилищем данных (DB DQ service).

3. Внедрить функционал ведения журнала проверок для последующего анализа ошибок, оптимизации правил проверки и подтверждения соответствия требованиям.
4. Организовать периодический контроль за метриками модуля для своевременного выявления проблем.

2.3 Назначение модуля

Модуль Data Quality предназначен для обеспечения высокого уровня качества данных, которые используются для просчета скорингового балла по страховым продуктам. Основная задача модуля заключается в проверке корректности, полноты и соответствия данных установленным требованиям. Он выполняет функции автоматической валидации входящих котировок, сопоставления данных с эталонной схемой, проверки допустимых значений обязательных атрибутов. Данный модуль помогает минимизировать риск ошибок, повысить точность прогнозов, а также создать условия для быстрого выявления и устранения некорректных данных.

3 ОПИСАНИЕ АВТОМАТИЗИРУЕМЫХ БИЗНЕС-ПРОЦЕССОВ

3.1 Текущие бизнес-процессы до внедрения модуля обеспечения качества данных

До внедрения модуля проверки качества данных были возможны два сценария взаимодействия с клиентом. Первый подразумевал, что клиенты приходили в офис страховой компании, заполняли бумажные анкеты, второй – клиенты оставляли онлайн-заявки через форму на сайте.

3.1.1 Ручные процессы проверки данных

Для первого сценария, описанного выше, были характерны: сверка документов (водительское удостоверение/паспорт) с информацией в анкете, перенос сотрудником данных в информационную систему, а также ручной поиск ошибок. Из этого следовал ряд проблем: опечатки при переносе данных из бумажного варианта в электронный, задержки из-за необходимости уточнять данные.

Второй сценарий также не был совершенным. Клиент заполнял электронную форму на сайте, где данные не проходили автоматическую валидацию. Данные с формы сохранялись в БД, сотрудник вручную проверял их корректность и полноту и связывался с клиентом, в случае необходимости исправлений. Некоторые данные нужно было стандартизировать вручную (например, поле с адресом, где клиент писал СПб, а не Санкт-Петербург).

3.1.2 Проблемы и ограничения

Как уже было сказано выше, из-за отсутствия модуля проверки качества данных существовали проблемы, которые необходимо было решить в срочном порядке, чтобы избежать финансовых потерь, недовольства клиентов, неверных прогнозов моделей.

Основными проблемами являлись:

1. Ошибки ввода данных. Данные из офлайн-анкет переносились вручную сотрудниками, что приводило к опечаткам. В онлайн-форме отсутствовала валидация – не было предусмотрено ограничений на

максимальный и минимальный ввод, обязательного заполнения пустых полей, определенного стандарта для вводимых данных.

2. Несогласованность данных. Такие параметры как адрес и пол, могли содержать разные форматы значений, что влекло за собой нужду в их дополнительной стандартизации.
3. Временные затраты. Из-за того, что данные поступали в «сыром» виде, сотрудникам приходилось вручную их обрабатывать, чистить и дополнять. Вследствие чего они тратили до 30 % рабочего времени на исправление ошибок и просчёт кредитного рейтинга затягивался.
4. Риски для бизнеса. Ошибки в данных приводили к некорректным расчётам и последующим судебным искам. Происходила потеря деловой репутации из-за недовольства клиентов полученным коэффициентом.

Имея все эти проблемы, ручные процессы все равно сохранялись в компании долгое время. Это было связано с рядом факторов:

1. Организационные причины. Архитектура системы фокусировалась на масштабируемости и ML-моделях, а не на качестве данных. Также руководство не измеряло метрики, связанные с количеством ошибок и временем обработки данных, поэтому проблема не казалась очевидной.
2. Технические ограничения. Система создавалась на устаревшей платформе, где трудно было внедрить сложные DQ-алгоритмы. Отсутствовали единые стандарты, проверки на дубликаты и валидация форматов.
3. Культурные аспекты. Сотрудники воспринимали качество данных как «техническую деталь», а не критический фактор для бизнеса. Они не доверяли автоматизации, опасаясь сокращения штата или увеличения нагрузки из-за необходимости осваивать новые инструменты.

3.2 Целевые бизнес-процессы после внедрения модуля обеспечения качества данных

Изначально система оценки страховых рисков полагалась на ручные проверки из-за ограниченной зрелости ИТ-инфраструктуры и приоритета скорости над качеством. Однако с увеличением объема данных и требований к точности прогнозов, ручные процессы стали узким местом, из-за чего потребовалось внедрение модуля Data Quality для автоматизации валидации, стандартизации и мониторинга данных.

3.2.1 Описание логики целевых бизнес-процессов

Бизнес-процессы модуля DQ представлены в виде диаграммы BPMN на рисунках 5–8.

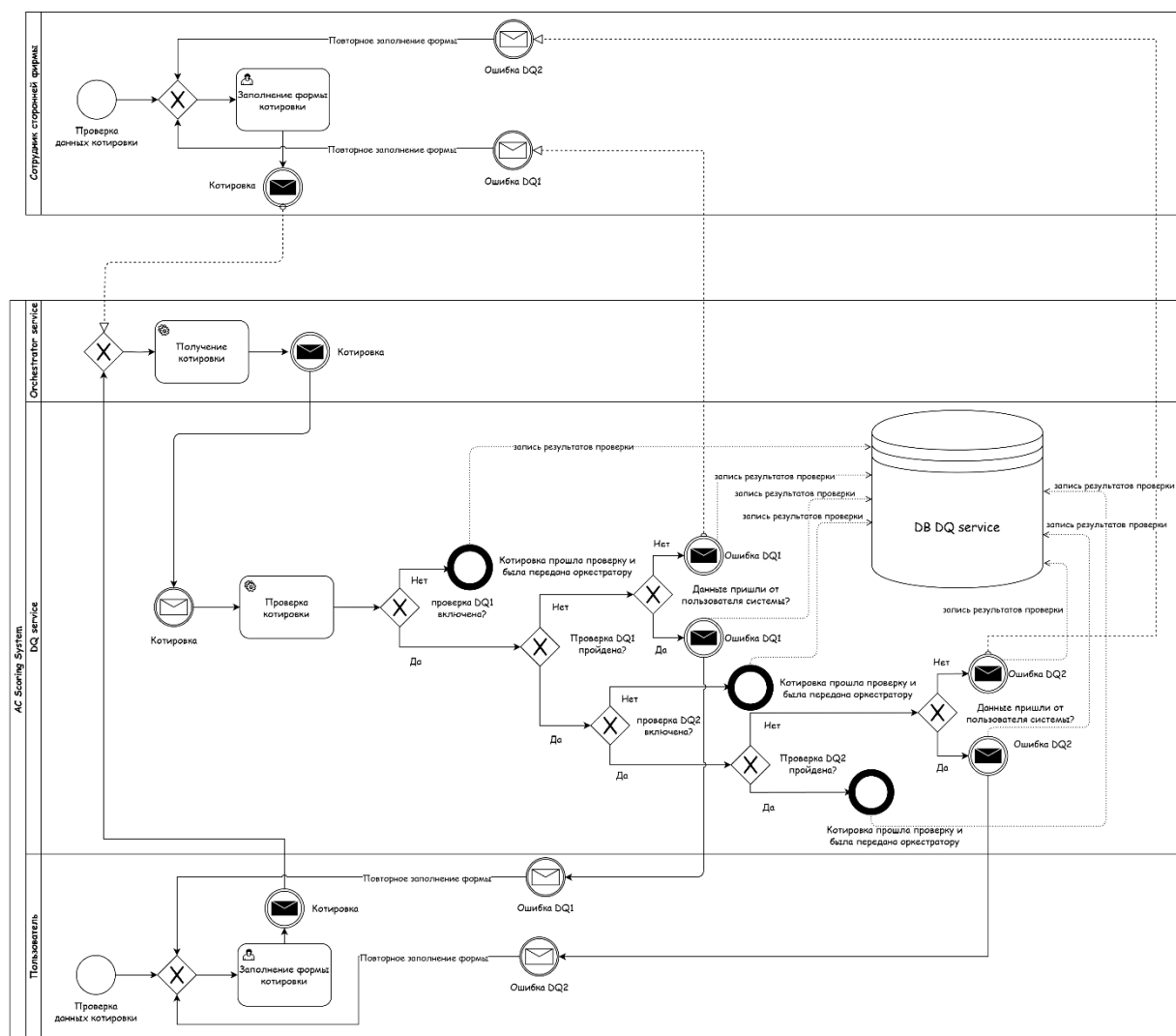


Рисунок 5 — Бизнес-процесс «Внесение данных на проверку»

Внесение данных на проверку (рисунок 5). Этот процесс начинается с подготовки и передачи данных в систему для проверки их качества. Данные собираются из различных источников, поступают на оркестратор, далее проходят проверки DQ 1 и DQ 2, а полученные результаты записываются в DB DQ. Если входящая котировка не прошла проверки, то она отправляется на доработку или исправление с соответствующими ошибками. Если проверки пройдены, котировка передается в оркестратор для дальнейших манипуляций.

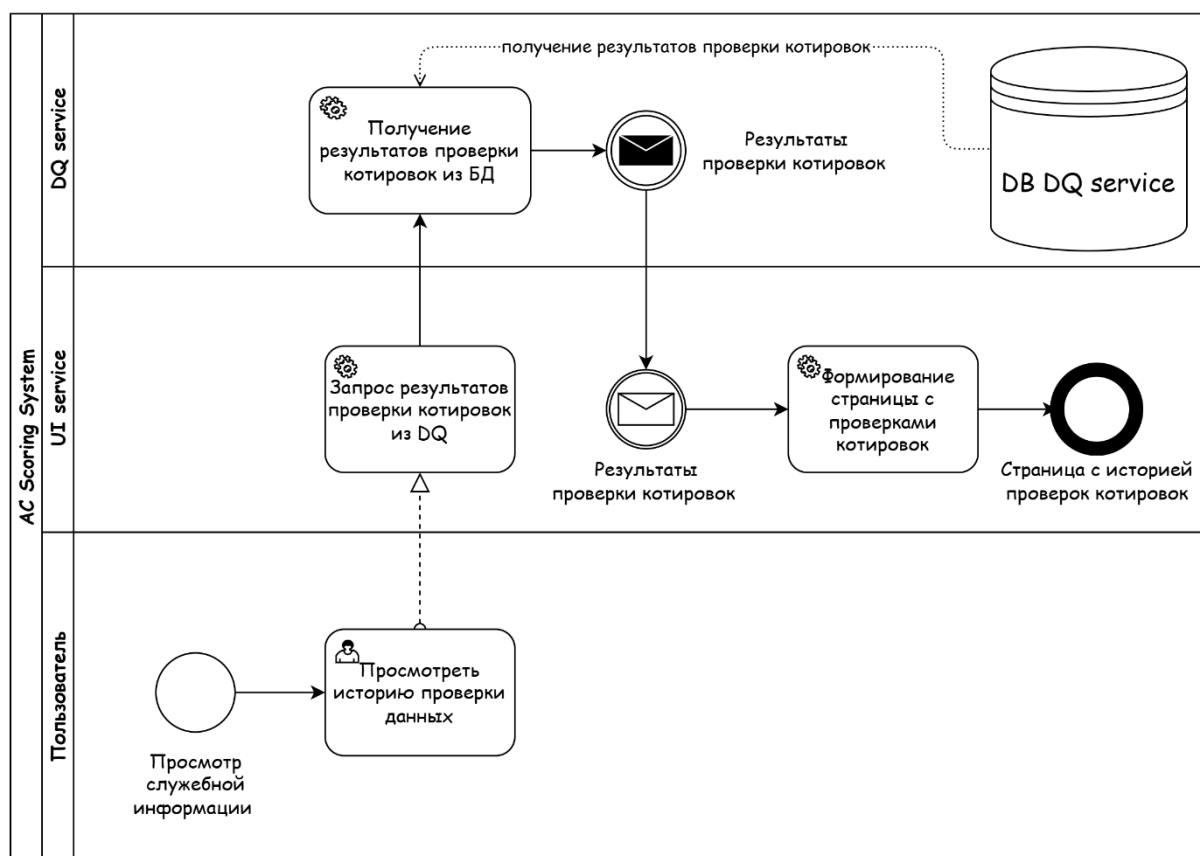


Рисунок 6 — Бизнес-процесс «Действия пользователя»

Действия пользователя в системе DQ (рисунок 6). Пользователи системы DQ могут отправлять данные на проверку, получать результаты с отчетами об ошибках и принимать решения о дальнейшем их исправлении. Этот процесс позволяет пользователям взаимодействовать с системой для обеспечения чистоты и корректности данных, а также следить за статусом проверок через интерфейс системы.

Действия администратора в системе DQ (рисунок 7). Администратор системы отвечает за настройку параметров проверки и мониторинг процессов. Он контролирует выполнение проверок, анализирует метрики и устраняет инциденты для поддержания стабильности системы. Эти действия позволяют системе DQ эффективно функционировать, обеспечивая высокий уровень качества данных и безопасность операций.

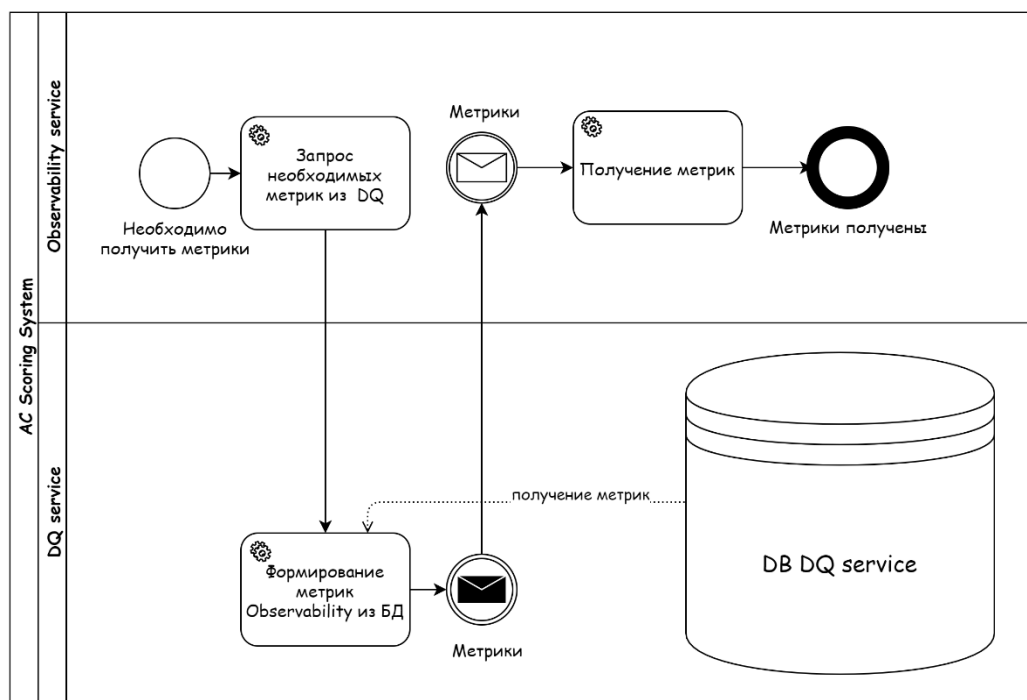


Рисунок 8 — Бизнес-процесс «Работа с метриками»

Работа с метриками (рисунок 8) предполагает их формирование на основе данных из БД и передачу в сервис Observability (один раз в сутки). Ознакомиться с формированием метрик можно в разделе 4.2.2 Модель данных.

3.2.2 Техническое взаимодействие компонентов АС

Процессы, изображенные на BPMN-диаграммах в разделе 3.1, технически реализуются через последовательность действий, которая отражена на UML-диаграмме (рисунок 9).

Первым представлен процесс формирования страниц в DQ. Сотрудник инициирует запрос через пользовательский интерфейс (далее UI), который взаимодействует с модулем качества данных для получения необходимых

данных из БД. После этого DQ возвращает структурированные данные в UI, где происходит автоматическое формирование страницы с визуализацией информации. Результатом является отображенная веб-страница.

Удаление записи о проверке котировки является следующим процессом на диаграмме ниже. Он начинается с инициации сотрудником мягкого удаления выбранных записей через сервис UI, который, в свою очередь, обращается к DQ. Модуль выполняет изменение состояния записи в БД и возвращает подтверждение успешного выполнения операции. Логическое удаление обеспечивает сохранность исторических данных и возможность аудита, что соответствует принципам ACID (атомарность, согласованность, изоляция, устойчивость).

Третий процесс – обновление статуса проверки DQ 1/DQ 2. Сотрудник изменяет положение переключателя на странице (вкл./выкл.) через UI, который передает этот запрос в DQ. Модуль качества данных обновляет статус в БД и возвращает подтверждение. Интерфейс обновляется для отображения актуальных данных о состоянии проверок.

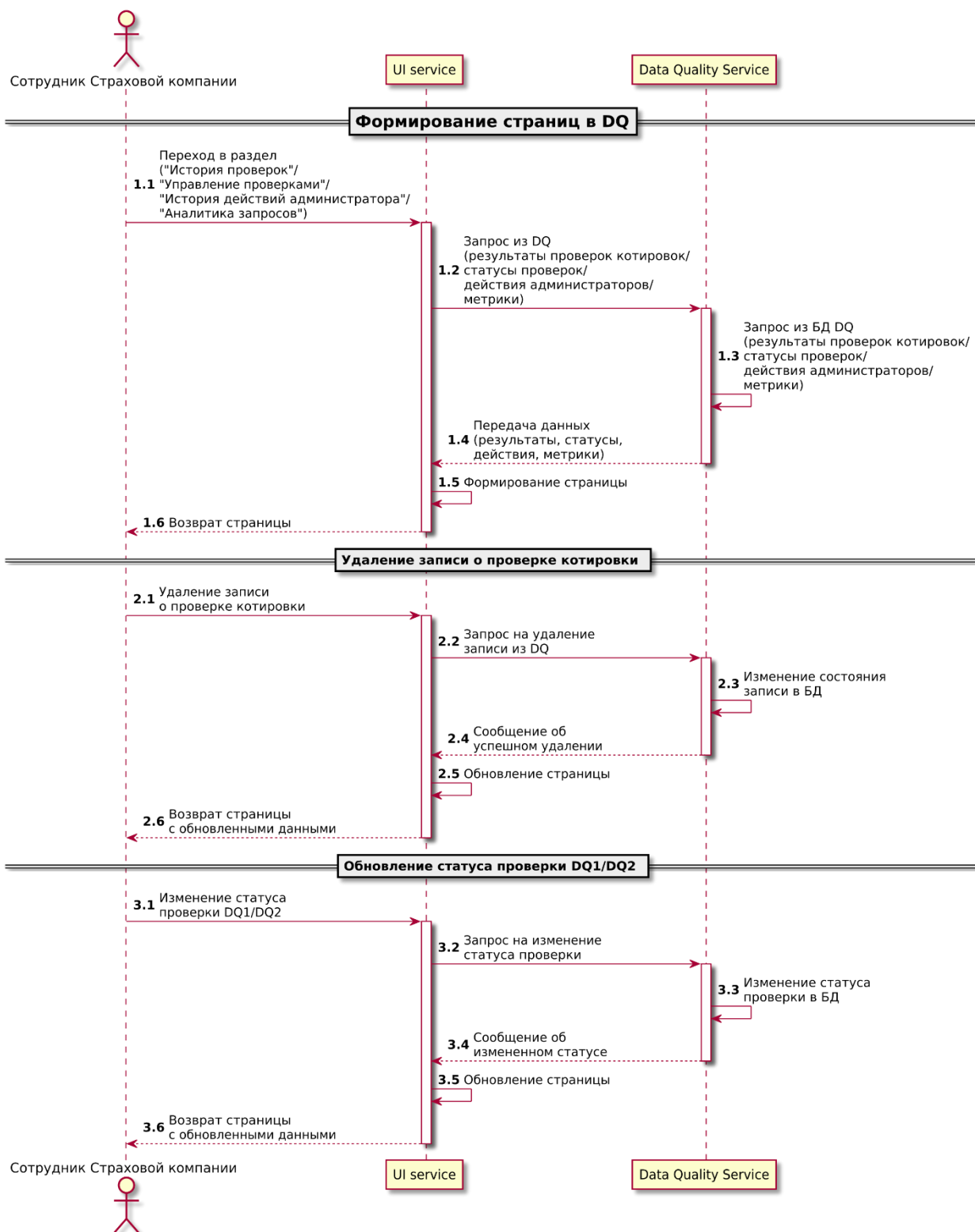


Рисунок 9 — Диаграмма последовательности

4 ТРЕБОВАНИЯ К МОДУЛЮ

4.1 Функциональные требования

4.1.1 Требования к функциональным возможностям

В модуле DQ должна быть следующая ролевая модель: пользователь, администратор.

Пользователем является сотрудник страховой компании или клиент, которому доступны функции просмотра служебной информации сервиса DQ.

Администратором является сотрудник страховой, которому доступны функции: управление сервисом DQ, просмотр служебной информации DQ.

Таблица 1 – Пользовательские истории

Идентификатор ПИ	User story
US-1	Я, как пользователь, хочу просматривать список результатов проверок DQ по своим котировкам, чтобы следить за хронологией запросов на проверку DQ.
US-2	Я, как пользователь, хочу иметь возможность просмотреть подробную информацию по каждому результату проверки DQ котировки, чтобы понимать, какие данные были отправлены, и узнать подробный статус их проверки.
US-3	Я, как пользователь, хочу иметь возможность фильтровать список результатов проверки DQ своих котировок по дате, статусу и типу продукта, чтобы быстрее находить необходимые записи.
US-4	Я, как администратор, хочу иметь возможность включать и выключать проверку DQ 1 по каждому страховому продукту, чтобы влиять на производительность системы.

Продолжение таблицы 1

US-5	Я, как администратор, хочу иметь возможность отдельно включать и выключать каждую проверку DQ 2 по каждому страховому продукту, чтобы влиять на производительность системы.
US-6	Я, как администратор, хочу иметь возможность включать и выключать одновременно все проверки DQ 2 по каждому страховому продукту, чтобы влиять на производительность системы.
US-7	Я, как администратор, хочу просматривать список результатов проверок DQ по всем котировкам, чтобы следить за хронологией запросов на проверку DQ.
US-8	Я, как администратор, хочу иметь возможность просмотреть подробную информацию по каждому результату проверки DQ котировки, чтобы понимать, какие данные были отправлены, и узнать подробный статус их проверки.
US-9	Я, как администратор, хочу иметь возможность фильтровать список результатов проверки DQ всех котировок по дате, статусу и типу продукта, чтобы быстрее находить необходимые записи.
US-10	Я, как администратор, хочу иметь возможность временно исключать результаты проверки DQ котировок из рабочего пространства, чтобы поддерживать актуальность базы данных.
US-11	Я, как администратор, хочу иметь возможность просматривать хронологию действий всех администраторов, чтобы установить возможную причину возникновения ошибки при проверке DQ котировки.

Продолжение таблицы 1

US-12	Я, как администратор, хочу иметь возможность фильтровать список действий администраторов по дате и типу действия, чтобы быстрее находить необходимые записи.
US-13	Я, как администратор, хочу просматривать значения бизнес-метрик сервиса DQ за указанный временной промежуток по каждому страховому продукту с целью проведения анализа.

4.1.2 Требования к проверкам DQ

Проверка DQ 1 заключается в валидации входящей котировки на соответствие эталонной JSON-схеме (представлена в приложении А).

Проверки DQ 2 подразделяются на три основные категории: проверка продукта, проверка субъекта и проверка документа.

Процесс проверки котировки осуществляется строго последовательно: от соответствия JSON-схеме (DQ 1) до проверки типа документа (DQ 2, проверка 2.3). Если проверка DQ 1 не пройдена, процесс прекращается и проверки DQ 2 не выполняются. В случае возникновения ошибки на одном из этапов DQ 2 она фиксируется, после чего проверка продолжается в следующем разделе. Благодаря этому пользователь получит полный список обнаруженных ошибок в рамках проверок DQ 2 за один цикл анализа.

Перечень проверок DQ 2:

1. Проверка продукта.

```
"product": {  
  "productType": "ОСАГО",  
  "productCode": "prod001"  
}
```

1.1 Проверка наличия кода продукта в таблице products.

Цель: убедиться, что указанный код продукта (productCode) существует в DB DQ.

Входной параметр – значение productCode.

Логика выполнения: если значение `productCode` отсутствует в таблице `products`, возвращается ошибка «Расчет скорингового балла по данному страховому продукту не предусмотрен в системе».

Ожидаемый результат: код продукта найден, проверка пройдена.

1.2 Проверка наличия типа продукта в таблице `products`.

Цель: убедиться, что указанный тип продукта (`productType`) существует в DB DQ.

Входной параметр – значение `productType`.

Логика выполнения: если значение `productType` отсутствует в таблице `products`, возвращается ошибка «Расчет скорингового балла по данному страховому продукту не предусмотрен в системе».

Ожидаемый результат: тип продукта найден, проверка пройдена.

1.3. Проверка соответствия типа и кода продукта в таблице `products`.

Цель: проверить, соответствует ли указанный тип продукта (`productType`) значению кода продукта (`productCode`).

Входные параметры – `productCode`, `productType`.

Логика выполнения: если `productType` не соответствует `productCode`, возвращается ошибка «Расчет скорингового балла по данному страховому продукту не предусмотрен в системе».

Ожидаемый результат: тип продукта соответствует коду, проверка успешно завершена.

2. Проверка субъекта.

```
"subjects": [  
  {  
    "firstName": "Алексей",  
    "secondName": "Петров",  
    "middleName": "Владимирович",  
    "birthDate": "1990-09-02",  
    "gender": "мужской",  
    "addresses": [  
      {
```



```

        "country": "string",
        "region": "string",
        "city": "string",
        "street": "string",
        "houseNumber": "string",
        "apartmentNumber": "string"
    }
],
...

```

2.1 Проверка возраста субъекта (минимум).

Цель: убедиться, что субъект достиг совершеннолетия (18 лет).

Входной параметр – значение birthDate.

Логика выполнения: вычислить возраст на основании значения birthDate.

Если возраст < 18, возвращается ошибка «Клиент не достиг совершеннолетия».

Ожидаемый результат: возраст ≥ 18 лет, проверка пройдена.

2.2 Проверка возраста субъекта (максимум).

Цель: убедиться, что возраст субъекта не превышает допустимое значение (90 лет).

Входной параметр – значение birthDate.

Логика выполнения: вычислить возраст на основании значения birthDate.

Если возраст > 90, возвращается ошибка «Возраст клиента выше допустимого значения».

Ожидаемый результат: возраст ≤ 90 лет, проверка пройдена.

2.3 Проверка корректности пола субъекта.

Цель: убедиться, что значение атрибута gender корректно.

Входной параметр – значение gender.

Логика выполнения: если значение gender не «мужской» или «женский», возвращается ошибка «Выберите пол: мужской/женский».

Ожидаемый результат: пол соответствует допустимым значениям, проверка пройдена.

3. Проверка документа.

```
"documents": [  
  {  
    "documentType": "паспорт",  
    "documentNumber": "string",  
    "issueDate": "string"  
  }  
]
```

3.1 Проверка типа документа.

Цель: убедиться, что тип документа (documentType) соответствует допустимым значениям.

Входной параметр – значение documentType.

Логика выполнения: если значение documentType не равно «паспорт» или «водительское удостоверение», возвращается ошибка «Данный тип документа не подходит для расчета скорингового балла».

Ожидаемый результат: тип документа соответствует допустимым значениям, проверка пройдена.

4.1.3 Требования к графическому интерфейсу

Уровень вложенности для администратора не должен содержать более двух переходов.

Администратору доступны следующие разделы:

1. Управление проверками – раздел, в котором происходит переключение между панелями управления проверками DQ для каждого страхового продукта.

2. История проверки данных – раздел, в котором отображается таблица с записями о проверках данных котировок.

3. История действий администратора – раздел, в котором отображается таблица с записями о действиях администратора.

4. Аналитика запросов – раздел, в котором отображаются бизнес-метрики сервиса DQ.

Уровень вложенности для пользователя не должен содержать более одного перехода.

Пользователю доступны следующие разделы:

1. История проверки данных – раздел, в котором отображается таблица с записями о проверках данных котировок.

Интерфейс и его описание представлены в разделе 5.

4.1.3.1 Спецификация вариантов использования

Таблица 2 — ВИ «Изменение состояния проверки DQ»

Идентификатор ВИ	UC-1
Наименование	Изменение состояния проверки DQ.
Автор	Гришкина Полина
Дата создания	02.02.2025
Основное действующее лицо	Администратор
Дополнительное действующее лицо	Система
Описание	Управление процессом проверки качества данных для обеспечения гибкости и контроля над производительностью системы.
Приоритет	Высокий
Условие-триггер	Администратор в разделе «Управление проверками» выбрал страховой продукт и нажал на ползунок «вкл./выкл.» проверки DQ.
Предварительные условия	1. Пользователь должен быть авторизован и иметь права администратора. 2. Проверки DQ ранее настроены и находятся в базе.
Выходные условия	1. Проверка DQ успешно включена/выключена. 2. Изменения сохранены в базе и отображены на странице.
Основные потоки	1. В разделе «Управление проверками» Администратор выбирает страховой продукт. 2. Система предоставляет проверки DQ и их состояние (включена/выключена). 3. Администратор нажимает на ползунок рядом с проверкой и меняет ее состояние (включает/выключает). 4. Система применяет изменения и подтверждает успешное выполнение операции.
Альтернативные потоки	3.1 Администратор нажимает на ползунок рядом с DQ 2 и включает/выключает все проверки DQ 2 сразу. Переход на шаг 4.

Продолжение таблицы 2

Исключения	1. Система не может сохранить изменения из-за ошибки (сбой соединения с сервером и т.п.). 1.1 Система возвращает ошибку. 1.2 Завершение ВИ.
------------	--

Таблица 3 — ВИ «Удаление записи о проверке данных котировки»

Идентификатор ВИ	UC-2
Наименование	Удаление записи о проверке данных котировки.
Автор	Гришкина Полина
Дата создания	02.02.2025
Основное действующее лицо	Администратор
Дополнительное действующее лицо	Система
Описание	Обеспечение актуальности и чистоты данных в системе.
Приоритет	Высокий
Условие-триггер	Администратор инициирует удаление информации о проверке, нажав на checkbox рядом с соответствующей записью.
Предварительные условия	1. Пользователь должен быть авторизован и иметь права администратора. 2. Существует запись о проверке, доступная для удаления.
Выходные условия	1. Запись успешно удалена. 2. Изменения сохранены в базе и отображены на странице.
Основные потоки	1. В разделе «История проверок» Администратор устанавливает checkbox рядом с записью для удаления. 2. Нажимает на кнопку «Удалить». 3. Система подтверждает удаление записи, данные в таблице обновляются.
Альтернативные потоки	1.1 Администратор устанавливает несколько checkbox-ов для удаления нескольких записей за раз. Переход на шаг 2. 1.2 Администратор устанавливает checkbox в ячейке «Удалить все записи» для удаления всех данных в таблице. Переход на шаг 2.

Продолжение таблицы 3

Исключения	1. Запись уже была удалена другим пользователем. 1.1 Система возвращает ошибку и просит перезагрузить страницу. 1.2 Завершение ВИ. 2. Ошибка связи с базой данных, удаление не может быть выполнено. 2.1 Система возвращает ошибку. 2.2 Завершение ВИ. 3. Администратор случайно выбирает неверную запись и отменяет операцию. 3.1 Система отменяет удаление. 3.2 Завершение ВИ.
------------	---

Таблица 4 — ВИ «Фильтрация записей о проверке данных котировки»

Идентификатор ВИ	УС-3
Наименование	Фильтрация записей о проверке данных котировки.
Автор	Гришкина Полина
Дата создания	02.02.2025
Основное действующее лицо	Администратор/Пользователь
Дополнительное действующее лицо	Система
Описание	Упрощение поиска и анализа результатов проверок путем предоставления инструментов для быстрого отображения только релевантных записей.
Приоритет	Средний
Условие-триггер	Администратор/Пользователь перешёл в раздел «История проверок» и задал критерии фильтрации.
Предварительные условия	1. Администратор/Пользователь должен быть авторизован. 2. Существуют записи, которые можно фильтровать.
Выходные условия	1. Система отображает данные, отфильтрованные по указанным параметрам. 2. Если фильтры очищены, отображаются все доступные записи.
Основные потоки	1. В разделе «История проверок» Администратор/Пользователь задает критерии фильтрации. 2. Система обрабатывает запрос и отображает записи, соответствующие указанным критериям.

Продолжение таблицы 4

Альтернативные потоки	1.1 Администратор/Пользователь задаёт только часть критериев. Переход на шаг 2. 1.2 Администратор/Пользователь сбрасывает все фильтры. Переход на шаг 2. 2. Администратор/Пользователь указывает некорректный формат диапазона дат (дата окончания раньше даты начала). 2.1 Система меняет даты местами. Переход на шаг 2.
Исключения	1. В базе данных отсутствуют записи, соответствующие параметрам фильтрации. 1.1 Система отображает сообщение «Нет данных». 1.2 Завершение ВИ.

Таблица 5 — ВИ «Фильтрация записей о действиях администратора»

Идентификатор ВИ	УС-4
Наименование	Фильтрация записей о действиях администратора.
Автор	Гришкина Полина
Дата создания	02.02.2025
Основное действующее лицо	Администратор
Дополнительное действующее лицо	Система
Описание	Упрощение поиска и мониторинга действий администратора путем предоставления инструментов для быстрого отображения только релевантных записей.
Приоритет	Средний
Условие-триггер	Администратор перешёл в раздел «История действий администратора» и задал критерии фильтрации.
Предварительные условия	1. Пользователь должен быть авторизован и иметь права администратора. 2. Существуют записи, которые можно фильтровать.
Выходные условия	1. Система отображает данные, отфильтрованные по указанным параметрам. 2. Если фильтры очищены, отображаются все доступные записи.
Основные потоки	1. В разделе «История действий администратора» Администратор задает критерии фильтрации. 2. Система обрабатывает запрос и отображает записи, соответствующие указанным критериям.

Продолжение таблицы 5

Альтернативные потоки	1.1 Администратор задаёт только часть критериев. Переход на шаг 2. 1.2 Администратор сбрасывает все фильтры. Переход на шаг 2. 2. Администратор указывает некорректный формат диапазона дат (дата окончания раньше даты начала). 2.1 Система меняет даты местами. Переход на шаг 2.
Исключения	1. В базе данных отсутствуют записи, соответствующие параметрам фильтрации. 1.1 Система отображает сообщение «Нет данных». 1.2 Завершение ВИ.

Таблица 6 — ВИ «Детализация статуса проверки данных»

Идентификатор ВИ	UC-5
Наименование	Детализация статуса проверки данных.
Автор	Гришкина Полина
Дата создания	02.02.2025
Основное действующее лицо	Администратор/Пользователь
Дополнительное действующее лицо	Система
Описание	Предоставление возможности получить развернутую информацию о процессе и результате проверки данных котировки.
Приоритет	Высокий
Условие-триггер	Администратор/Пользователь перешёл в раздел «История проверок» и нажал на иконку рядом со статусом интересующей его проверки.
Предварительные условия	1. Администратор/Пользователь должен быть авторизован. 2. Существуют записи, у которых можно просмотреть делаги статуса.
Выходные условия	1. Администратор/Пользователь ознакомился с детальной информацией о статусе проверки данных. 2. Администратор/Пользователь может вернуться к общему списку проверок, закрыв модальное окно.

Продолжение таблицы 6

Основные потоки	1. В разделе «История проверок» Администратор/Пользователь нажимает на иконку рядом со статусом интересующей его проверки. 2. Система отображает детальную информацию о статусе проверки, включая: ○ номер проверки; ○ вид страхового продукта; ○ дата проверки; ○ поступивший JSON; ○ подробности проверки.
Альтернативные потоки	–
Исключения	1. Запись была удалена до того, как Администратор/Пользователь попытался открыть детальный статус. 1.1 Система отображает сообщение «Запись недоступна». 1.2 Завершение ВИ. 2. Ошибка загрузки данных из базы (сбой соединения и т.п.). 2.1 Система выводит сообщение об ошибке. 2.2 Завершение ВИ.

Таблица 7 — ВИ «Детализация действия администратора»

Идентификатор ВИ	UC-6
Наименование	Детализация действия администратора.
Автор	Гришкина Полина
Дата создания	02.02.2025
Основное действующее лицо	Администратор
Дополнительное действующее лицо	Система
Описание	Предоставление возможности получить развернутую информацию о процессе и результате действия администратора.
Приоритет	Средний
Условие-триггер	Администратор перешёл в раздел «История действий администратора» и нажал на иконку рядом с действием, для детального ознакомления.
Предварительные условия	1. Пользователь должен быть авторизован и иметь права администратора. 2. Существуют записи действий администратора, у которых можно просмотреть дела.
Выходные условия	1. Администратор ознакомился с детальной информацией о действии администратора. 2. Администратор может вернуться к общему списку действий администратора, закрыв модальное окно.

Продолжение таблицы 7

Основные потоки	1. В разделе «История действий администратора» Администратор нажимает на иконку рядом с действием, которое его интересует. 2. Система отображает детальную информацию о действии администратора, включая: ○ номер действия; ○ вид страхового продукта; ○ дата действия; ○ подробности действия.
Альтернативные потоки	–
Исключения	1. Запись была удалена до того, как Администратор попытался открыть детализацию. 1.1 Система отображает сообщение «Запись недоступна». 1.2 Завершение ВИ. 2. Ошибка загрузки данных из базы (сбой соединения и т.п.). 2.1 Система выводит сообщение об ошибке. 2.2 Завершение ВИ.

Таблица 8 — ВИ «Выбор периода для отображения аналитики запросов»

Идентификатор ВИ	UC-7
Наименование	Выбор периода для отображения аналитики запросов.
Автор	Гришкина Полина
Дата создания	02.02.2025
Основное действующее лицо	Администратор
Дополнительное действующее лицо	Система
Описание	Упрощение анализа запросов, улучшение управления процессами проверки и выявление тенденции за выбранный промежуток времени.
Приоритет	Средний
Условие-триггер	Администратор в разделе «Аналитика запросов» выбрал страховой продукт и нажал на поля выбора периода (дата начала и дата окончания) для настройки отображаемой аналитики.
Предварительные условия	1. Пользователь должен быть авторизован и иметь права администратора. 2. Данные проверок доступны в системе и связаны с временными метками.
Выходные условия	1. Администратор видит аналитику проверок за выбранный период. 2. Администратор может изменить выбранный период или вернуться к отображению всех данных.

Продолжение таблицы 8

Основные потоки	1. В разделе «Аналитика запросов» Администратор выбирает страховой продукт. 2. Администратор выбирает дату начала и окончания периода. 3. Система обновляет графики и метрики, отображая данные только за выбранный период.
Альтернативные потоки	2.1 Администратор вручную вводит даты в поля вместо использования календаря. Переход на шаг 3. 2.2 Администратор очищает даты, система отображает данные за весь доступный период.
Исключения	1. Администратор выбирает период, для которого нет данных. 1.1 Система отображает сообщение «Данные за указанный период отсутствуют». 1.2 Завершение ВИ.

4.1.4 Требования к алгоритмам обработки

При разработке модуля контроля качества данных предъявляются следующие требования к алгоритмам серверной обработки:

1. Язык программирования: все серверные алгоритмы реализуются на языке Python (версия не ниже 3.11) с использованием фреймворка FastAPI для построения REST API.
2. Архитектура приложений:
 - вся серверная часть должна быть модульной, с разделением на слои обработки запросов (endpoints), бизнес-логики (services) и работы с базой данных (repository);
 - использование асинхронного программирования (async) там, где требуется обработка I/O операций (работа с БД, внешние запросы).
3. Валидация данных:
 - при получении котировок на проверку сервер выполняет валидацию JSON-структуры согласно утверждённой JSON-схеме;
 - валидация должна включать проверку наличия обязательных полей, корректности форматов (дата, UUID и др.), а также соблюдение ограничений на длину строк и допустимые значения перечислений.

4. Обработка ошибок:

- все ошибки валидации или обработки должны быть перехвачены и возвращены в виде стандартизированного ответа с кодом ошибки, описанием причины и возможным путём её исправления;
- логирование ошибок производится в системный журнал приложения.

5. Алгоритмы расчёта:

- на основе данных котировки производится расчёт кредитного рейтинга в диапазоне от 0 до 999 с категоризацией результата (очень высокий, высокий, средний, низкий);
- алгоритм обработки должен учитывать результаты проверок DQ (успех или наличие ошибок).

6. Работа с базой данных (PostgreSQL 15):

- все операции с данными реализуются через ORM SQLAlchemy 2.0 с поддержкой асинхронных операций (async с использованием `async_session`);
- данные о результатах валидаций, действиях администратора и ответах системы сохраняются в соответствующие таблицы с учётом требований к логическому удалению (soft delete) записей.

7. Интеграция с frontend:

- все методы API должны возвращать стандартизированные JSON-ответы;
- backend обрабатывает запросы от frontend, реализованного на базе библиотеки React.js 19.0.0.

8. Контейнеризация:

- backend-сервис разворачивается в контейнере Docker;
- настройки окружения (переменные подключения к БД и параметры запуска сервиса) передаются через Docker Compose-файл.

Разработка backend-модуля ориентирована на обеспечение высокой скорости обработки запросов, масштабируемости и лёгкости в сопровождении.

4.2 Архитектурные требования

4.2.1 Системная архитектура решения и требования к интеграции

Требования к архитектуре:

1. Сервис DQ должен являться отдельным микросервисом.
2. Хранилище данных сервиса DQ должно быть построено на основании реляционной СУБД.
3. Взаимодействие сервиса DQ с другими сервисами осуществляется посредством REST API.
4. Формат обмена данными сервиса DQ с другими сервисами – JSON.
5. Сервис DQ должен взаимодействовать со следующими сервисами: Observability, UI.
6. Поступление котировок в сервис DQ осуществляется из разных источников через Orchestrator.

Архитектура до внедрения модуля DQ представлена на рисунке 10. Система базировалась на микросервисной архитектуре с использованием Kubernetes и Kafka для оркестрации и обработки данных.

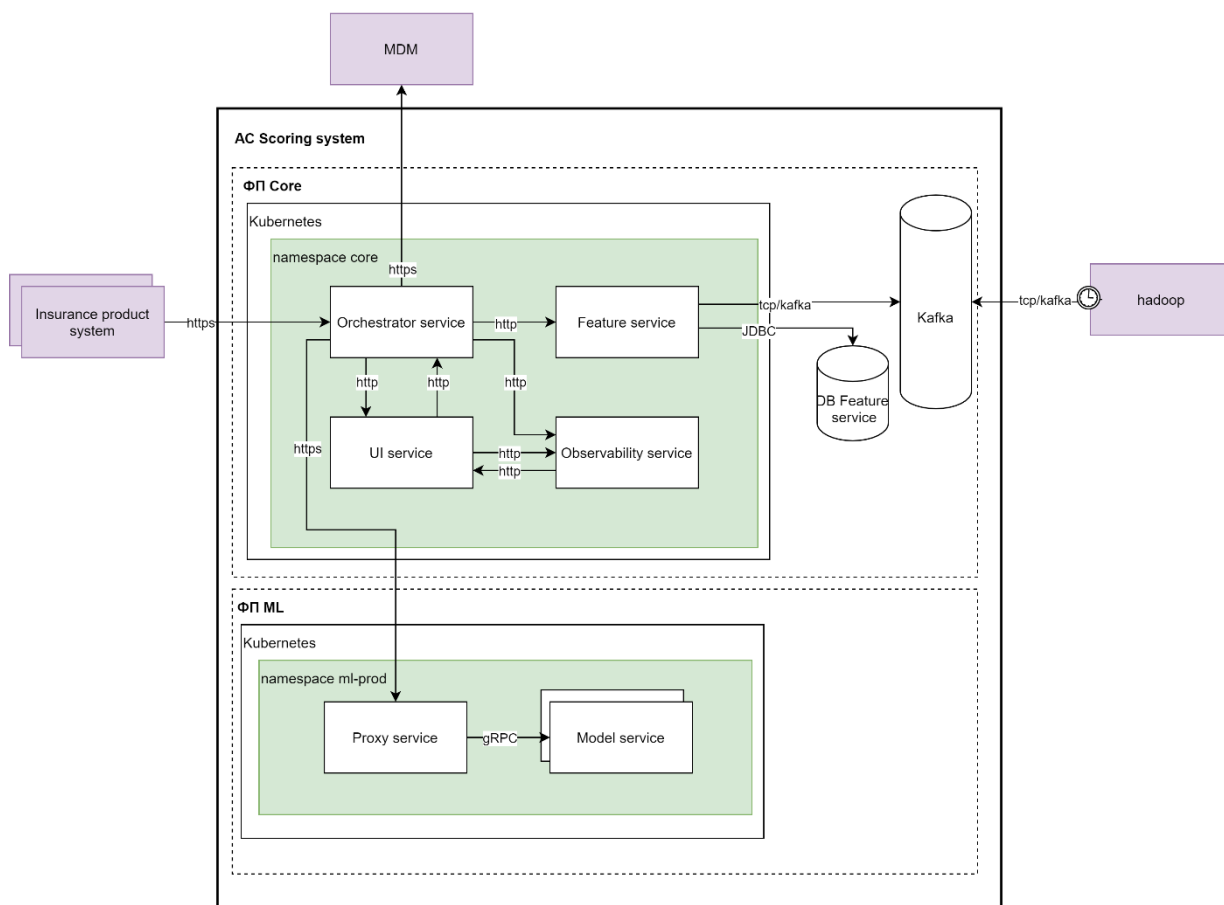


Рисунок 10 — Архитектура АС до внедрения модуля DQ

Система (сервис Оркестратор) получает на вход котировку (например, запрос на оценку ОСАГО продукта) от внешнего отправителя или из интерфейса системы.

1. Запрос проходит первичную валидацию (здесь и далее в обмене используется JSON).
2. Оркестратор:
 - 1) идентифицирует (находит) субъекты через запрос в MDM (система управления мастер-данными, хранит «золотые записи» по всем субъектам);
 - 2) запрашивает у Feature Service фичи (features) по идентифицированным субъектам;
 - 3) на основе фичей формирует вектор для вызова модели;
 - 4) направляет вектор в Proxy Service.
3. Proxy Service:

- 1) определяет, какие модели доступны для данного типа продукта (например, ОСАГО);
 - 2) отправляет запрос на predict (запуск) выбранной модели;
 - 3) возвращает ответ (скоринг) Оркестратору.
4. Оркестратор возвращает ответ на входную котировку внешнему отправителю или в интерфейс системы.

Архитектура модуля DQ представлена на рисунке 11.

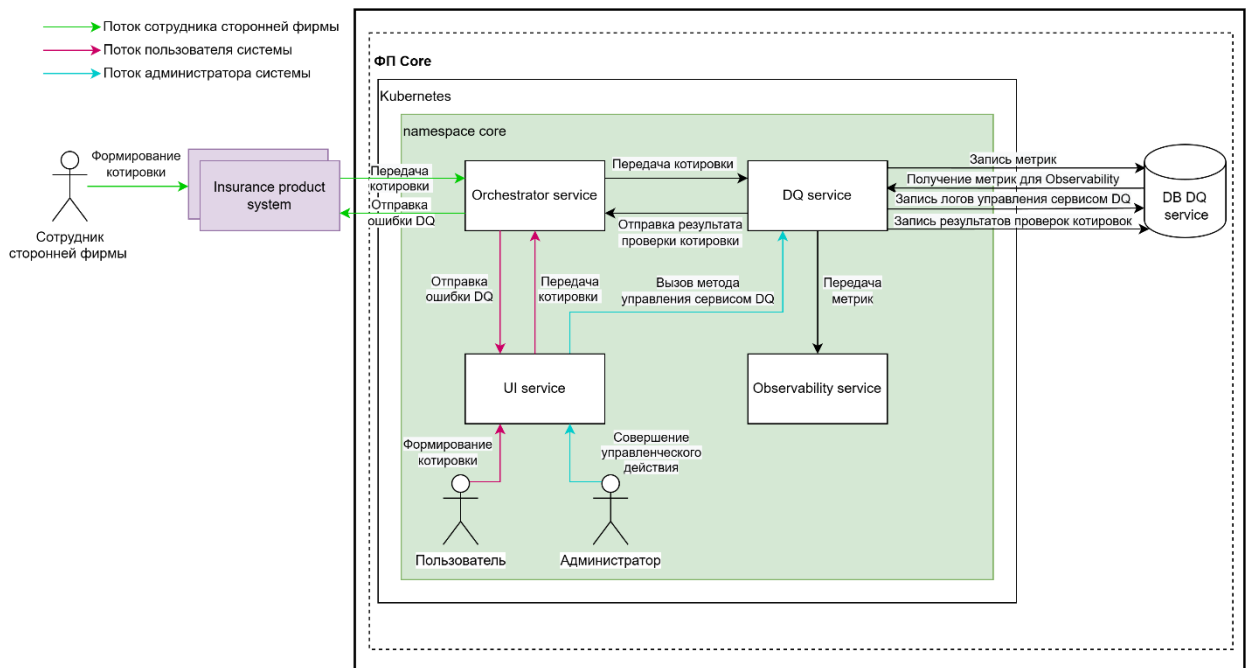


Рисунок 11 — Архитектура модуля DQ

Модуль DQ интегрирован в ядро системы, обеспечивая сквозную проверку данных на всех этапах.

Подробнее про каждое действие:

1. Формирование котировки.

Пользователь создает котировку через UI service. UI service передает котировку в Orchestrator service.

2. Проверка качества данных.

Orchestrator service передает котировку в DQ service для проверки. DQ service проверяет котировку на соответствие правилам качества данных (DQ 1, DQ 2). Если проверки пройдены, котировка передаётся обратно в Orchestrator

service. Если обнаружены ошибки, они передаются либо в UI service для отображения пользователю, либо в Insurance product system.

3. Управление и мониторинг.

Администратор DQ может выполнять управленческие действия через UI service (вкл./выкл. проверок, удаление записей). UI service вызывает методы управления в DQ service для внесения изменений. DQ service передает метрики в Observability service для мониторинга.

4. Сохранение результатов и логирование.

Результаты проверок котировок записываются в базу данных DQ service, также фиксируются логи управленческих действий.

5. Взаимодействие с внешними системами.

Если котировка поступила из Insurance product system и не прошла проверки, информация об ошибках будет передана обратно в эту систему для дальнейшей работы.

Требования к интеграции:

Спецификация OpenAPI для сервиса DQ представлена на рисунке 12, листинг кода в приложении Б.

Data Quality API 1.0.0 OAS 3.0

Управление проверками качества данных и обработка котировок.

Поддерживаются операции включения и отключения проверок, получения метрик, авторизации, а также обработки котировок и управления записями.

Servers

<https://ScoringSystem.com/UI/>

quotes Операции с котировками

POST /quotes Принять котировку

DELETE /quotes/{quoteId} Удалить котировку

metrics Операции с метриками качества данных

GET /metrics Получить метрики

GET /metrics/{metricName} Получить метрику

GET /metrics/by-date Получить все метрики за одну дату

GET /metrics/by-date-range Получить метрики за диапазон дат

checks Операции по управлению проверками

PUT /checks/DQ-1 Включить/отключить проверку DQ-1

PUT /checks/DQ-2 Включить/отключить проверку DQ-2

Schemas

QuoteRequest >

Рисунок 12 — Спецификация OpenAPI Data Quality

Описание методов API:

1. Методы, связанные с котировками (/quotes):

POST – принимает JSON с данными котировки, выполняет проверки качества данных. Request Body: объект QuoteRequest (данные котировки).

DELETE /quotes/{quoteId} – удаляет котировку по её уникальному идентификатору. Параметр пути quoteId (строка): уникальный идентификатор котировки.

2. Методы, связанные с метриками (/metrics):

GET – возвращает список доступных метрик качества данных.

GET /metrics/{metricName} – возвращает значение указанной метрики.

Параметр пути metricName (строка): имя метрики.

GET /metrics/by-date – возвращает список всех метрик за указанную дату.

Query параметр date (YYYY-MM-DD): дата для выборки метрик.

GET /metrics/by-date-range – возвращает метрики для заданного диапазона дат. Query параметры startDate (YYYY-MM-DD): начальная дата диапазона; endDate (YYYY-MM-DD): конечная дата диапазона.

3. Методы, связанные с проверками (/checks):

PUT /checks/DQ-1 – управляет статусом проверки DQ 1. Request Body: enabled (логический): true для включения, false для выключения.

PUT /checks/DQ-2 – управляет статусом проверки DQ 2. Request Body: checkName (строка): название проверки; enabled (логический): true для включения, false для выключения.

4.2.2 Модель данных

При разработке логической модели данных необходимо выполнить анализ предметной области и выявить сущности, а также определить их атрибуты и связи.

Полученная ER-диаграмма представлена на рисунке 13.

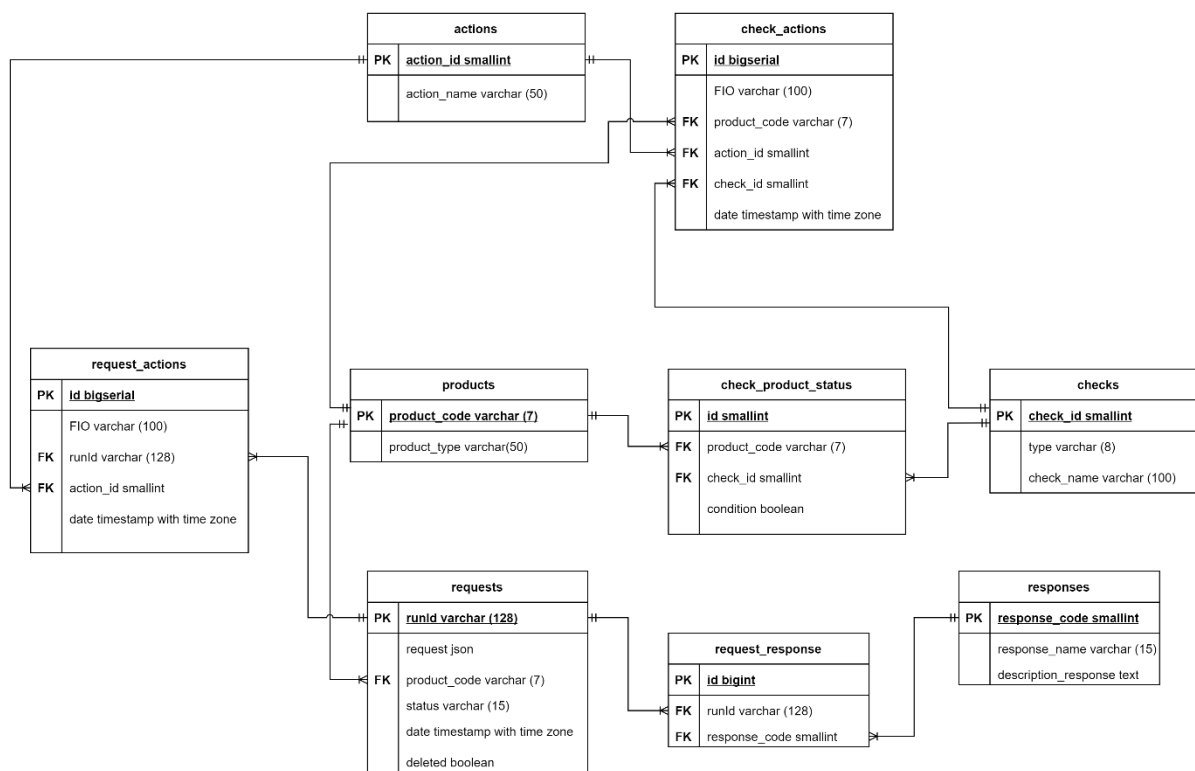


Рисунок 13 — Логическая модель данных в виде ER-диаграммы

Словарь данных

Принятые в словаре данных обозначения представлены в приложении В.

Таблица 9 – Словарь данных

Структура или элемент данных	Тип данных	Длина	Значение
Actions – справочник всех действий в системе.			
action_id	smallint	2	(PK) Первичный ключ — автоинкрементный уникальный идентификатор действия, генерируемый системой, начиная с 1.
action_name	varchar	50	Название действия (удаление записи, включение проверки, выключение проверки). Может содержать только символы латинского алфавита.
Products – информация о страховых продуктах.			
product_code	varchar	7	(PK) Первичный ключ — уникальный код продукта (prod001 для ОСАГО).
product_type	varchar	50	Тип продукта (ОСАГО). Может содержать только символы латинского алфавита.
Requests – информация о запросах на проверку данных котировок.			

Продолжение таблицы 9

runId	varchar	128	(PK) Первичный ключ — уникальный идентификатор запроса, генерируемый системой.
request	json	—	Содержит данные запроса (котировку).
product_code	varchar	7	(FK) Внешний ключ — содержит код связанного страхового продукта.
status	varchar	15	Статус запроса (в очереди, на проверке, проверен, отклонен).
date	timestamp with time zone	—	Дата создания запроса с учётом временной зоны. Формат записи YYYY-MM-DD hh:mm:ss+hh.
deleted	boolean	1	Поле не может быть пустым. Содержит: 0 — не удалено (по умолчанию); 1 — удалено.
Request_actions — журнал действий администратора над запросами (удаление записей).			
id	bigserial	10	(PK) Первичный ключ — уникальный идентификатор действия администратора над запросами, генерируемый системой, начиная с 1.
FIO	varchar	100	ФИО администратора, выполняющего действие.
runId	varchar	128	(FK) Внешний ключ — содержит идентификатор связанного запроса.
action_id	smallint	2	(FK) Внешний ключ — содержит идентификатор связанного действия.
date	timestamp with time zone	—	Дата создания запроса с учётом временной зоны. Формат записи YYYY-MM-DD hh:mm:ss+hh.
Request_response — результаты проверок данных котировок.			
id	bigint	10	(PK) Первичный ключ — уникальный идентификатор записи ответа, генерируемый системой, начиная с 1.
runId	varchar	128	(FK) Внешний ключ — содержит идентификатор связанного запроса.
response_code	smallint	2	(FK) Внешний ключ — содержит код связанного ответа.
Responses — справочник возможных результатов проверки.			
response_code	smallint	2	(PK) Первичный ключ — уникальный код ответа, генерируемый системой, начиная с 1.
response_name	varchar	15	Название ответа (пройдена, не пройдена).
(description_response)	text	—	Описание ответа, если проверка не пройдена (например, «Клиент не достиг совершеннолетия»). Необязательное поле.

Продолжение таблицы 9

Check_actions – журнал действий администратора над проверками (включение/выключение проверок).			
id	bigserial	10	(PK) Первичный ключ – уникальный идентификатор действия администратора над проверками, генерируемый системой, начиная с 1.
FIO	varchar	100	ФИО администратора, выполняющего действие.
product_code	varchar	7	(FK) Внешний ключ – содержит код связанного страхового продукта.
action_id	smallint	2	(FK) Внешний ключ – содержит идентификатор связанного действия.
check_id	smallint	2	(FK) Внешний ключ – содержит идентификатор связанной проверки.
date	timestamp with time zone	–	Дата создания запроса с учётом временной зоны. Формат записи YYYY-MM-DD hh:mm:ss+hh.
Check_product_status – состояние проверок по страховому продукту.			
id	smallint	2	(PK) Первичный ключ – уникальный идентификатор состояния проверки, генерируемый системой, начиная с 1.
product_code	varchar	7	(FK) Внешний ключ – содержит код связанного страхового продукта.
check_id	smallint	2	(FK) Внешний ключ – содержит идентификатор проверки.
condition	boolean	1	Статус проверки, поле не может быть пустым. Содержит: 0 — выключена; 1 — включена (по умолчанию).
Checks – справочник типов проверок.			
check_id	smallint	2	(PK) Первичный ключ – уникальный идентификатор проверки, генерируемый системой, начиная с 1.
type	varchar	8	Тип проверки (DQ 1, DQ 2.1 и т.д.).
check_name	varchar	100	Название проверки (например, «Проверка поступившего JSON по схеме»).

Метрики для аналитики запросов:

1. Количество успешных/неуспешных проверок по каждому страховому продукту за указанный промежуток времени. Данная метрика будет просчитываться на основе таблиц request_response, responses, requests.

```

SELECT
    p.product_code,
    p.product_type,
    SUM(CASE WHEN rs.response_name = 'пройдена' THEN 1 ELSE 0 END) AS
successful_checks,
    SUM(CASE WHEN rs.response_name = 'не пройдена' THEN 1 ELSE 0 END) AS
unsuccessful_checks
FROM
    products p
JOIN
    requests r ON p.product_code = r.product_code
JOIN
    request_response rr ON r.runId = rr.runId
JOIN
    responses rs ON rr.response_code = rs.response_code
WHERE
    r.date >= '2025-01-01 00:00:00+00'::timestampz
    AND r.date < '2025-02-01 00:00:00+00'::timestampz
GROUP BY
    p.product_code, p.product_type;

```

2. Топ-5 частых причин неуспешного прохождения проверки по каждому страховому продукту. Данная метрика будет просчитываться на основе таблиц request_response, responses, requests.

```

WITH unsuccessful_reasons AS (
    SELECT
        p.product_code,
        p.product_type,
        rs.description_response AS reason,
        COUNT(*) AS reason_count
    FROM
        products p
    JOIN
        requests r ON p.product_code = r.product_code
    JOIN
        request_response rr ON r.runId = rr.runId
    JOIN
        responses rs ON rr.response_code = rs.response_code
    WHERE
        rs.response_name = 'не пройдена'
        AND r.date >= '2025-01-01 00:00:00+00'::timestampz
        AND r.date < '2025-02-01 00:00:00+00'::timestampz
    GROUP BY
        p.product_code, p.product_type, rs.description_response
)
SELECT
    product_code,
    product_type,
    reason,

```

```

reason_count
FROM (
  SELECT
    product_code,
    product_type,
    reason,
    reason_count,
    ROW_NUMBER() OVER (PARTITION BY product_code ORDER BY
reason_count DESC) AS rn
  FROM
    unsuccessful_reasons
) ranked_reasons
WHERE
  rn <= 5;

```

3. Общее количество проверок по каждому страховому продукту за указанный промежуток времени. Данная метрика будет просчитываться на основе таблиц request_response, responses, requests.

```

SELECT
  p.product_code,
  p.product_type,
  COUNT(*) AS total_checks
FROM
  products p
JOIN
  requests r ON p.product_code = r.product_code
JOIN
  request_response rr ON r.runId = rr.runId
WHERE
  r.date >= '2025-01-01 00:00:00+00'::timestampz
  AND r.date < '2025-02-01 00:00:00+00'::timestampz
GROUP BY
  p.product_code, p.product_type;

```

4.3 Нефункциональные требования

1. Ограничения проверки данных.

Сервис DQ проверяет только котировки, исключая данные других типов, которые в проверке качества не нуждаются. Поддерживаются проверки котировок исключительно для страховых продуктов.

2. Источники данных.

Котировки могут поступать как из сторонних систем, так и из графического интерфейса (GUI) AC Scoring System.

3. История проверок.

В сервисе DQ должна сохраняться история результатов проверок котировок по страховым продуктам.

4. Интеграция с пользовательским интерфейсом.

Построение GUI для взаимодействия с модулем DQ осуществляется отдельным сервисом UI в рамках AC Scoring System. Управление сервисом DQ возможно только через GUI AC Scoring System.

5. Логирование управленческих действий.

В сервисе DQ должно вестись подробное логирование всех управленческих действий администратора с точностью до секунды.

6. Аналитика и отчётность.

Система аналитики модуля DQ должна предоставлять следующие данные:

- количество успешных/неуспешных проверок по каждому страховому продукту за указанный промежуток времени;
- топ-5 наиболее частых причин неуспешного прохождения проверки по каждому страховому продукту;
- общее количество проверок по каждому страховому продукту за указанный промежуток времени;
- процент успешных/неуспешных проверок по каждому страховому продукту за указанный промежуток времени.

7. Производительность.

Сервис должен обеспечивать обработку не менее 500 котировок в минуту при максимальной нагрузке. Среднее время проверки одной котировки не должно превышать 1 секунды.

8. Доступность.

Сервис DQ должен быть доступен 99,95 % времени в месяц. Время восстановления после сбоя не должно превышать 15 минут.

9. Безопасность.

Доступ к управлению и данным модуля DQ должен быть защищён механизмами аутентификации и авторизации. Все данные, поступающие в

сервис и передаваемые из него, должны быть защищены протоколами шифрования.

10. Масштабируемость.

Сервис DQ должен поддерживать возможность горизонтального масштабирования для обработки увеличивающегося объёма котировок без потери производительности.

5 РАЗРАБОТКА МОДУЛЯ КАЧЕСТВА ДАННЫХ

5.1 Функции общей работы

Данный модуль должен выполнять следующие функции:

для администратора:

- возможность управления проверками;
- возможность просмотра истории проверок;
- возможность просмотра истории действий администратора;
- возможность просмотра аналитики запросов;

для пользователя:

- возможность просмотра истории проверок.

5.1.1 Расчёт кредитного рейтинга

Раздел, в котором происходит расчёт кредитного рейтинга клиента, доступен как администраторам, так и пользователям (рисунок 14).

The screenshot shows a web application interface with a dark theme. On the left is a sidebar menu for user 'Иван Иванов' with options: 'Расчитать кредитный рейтинг', 'Качество данных', 'Управление проверками', 'История проверок', 'История действий администратора', and 'Аналитика запросов'. The main content area is titled 'Персональный кредитный рейтинг'. It contains three sections: 'Страховой продукт' with fields for 'Тип продукта' and 'Код продукта'; 'Субъекты' with fields for 'Фамилия', 'Имя', 'Отчество', 'Адрес регистрации', 'Дата рождения', and 'Пол'; and 'Документы' with fields for 'Тип документа', 'Серия и номер документа', and 'Дата выдачи'. Each section has a 'Добавить' button. A 'Получить результат' button is at the bottom right.

Рисунок 14 — Раздел «Расчитать кредитный рейтинг»

После корректного заполнения всех полей формы и нажатия на кнопку «Получить результат», произойдет отправка котировки в систему, после чего

на экране отобразится модальное окно с кредитным рейтингом и его описанием (рисунок 15).

The screenshot displays a web application interface for calculating a personal credit rating. The main panel is titled 'Персональный кредитный рейтинг'. It contains several input sections: 'Страховой продукт' (Insurance product) with a dropdown set to 'ОСАГО' and a code field 'prod001'; 'Субъекты' (Subjects) with fields for 'Фамилия' (Ivanov), 'Имя' (Ivan), and 'Отчество' (Ivanovich), along with a 'Добавить субъект' button; and 'Документы' (Documents) with a dropdown set to 'паспорт', a document number field '1111 111111', and an issue date field '23.09.2024'. A modal window titled 'Кредитный рейтинг от 15.04.2025' is overlaid, showing a large '900' score on a color-coded scale (red to green). Below the score, it states 'Очень высокий' (Very high) and provides a description: 'Ответственный заемщик с высокими шансами на получение кредита на выгодных условиях.' (Responsible borrower with high chances of getting a loan on favorable terms.). The background interface includes a sidebar with the user 'Иван Иванов' and a list of navigation items: 'Рассчитать кредитный рейтинг', 'Качество данных', 'Управление проверками', 'История проверок', 'История действий администратора', and 'Аналитика запросов'. A search bar and system icons are at the top.

Рисунок 15 — Раздел «Рассчитать кредитный рейтинг», результат

Если данные не пройдут первоначальные проверки модуля Data Quality, то отобразится модальное окно с соответствующими ошибками, пример приведен на рисунке 16.

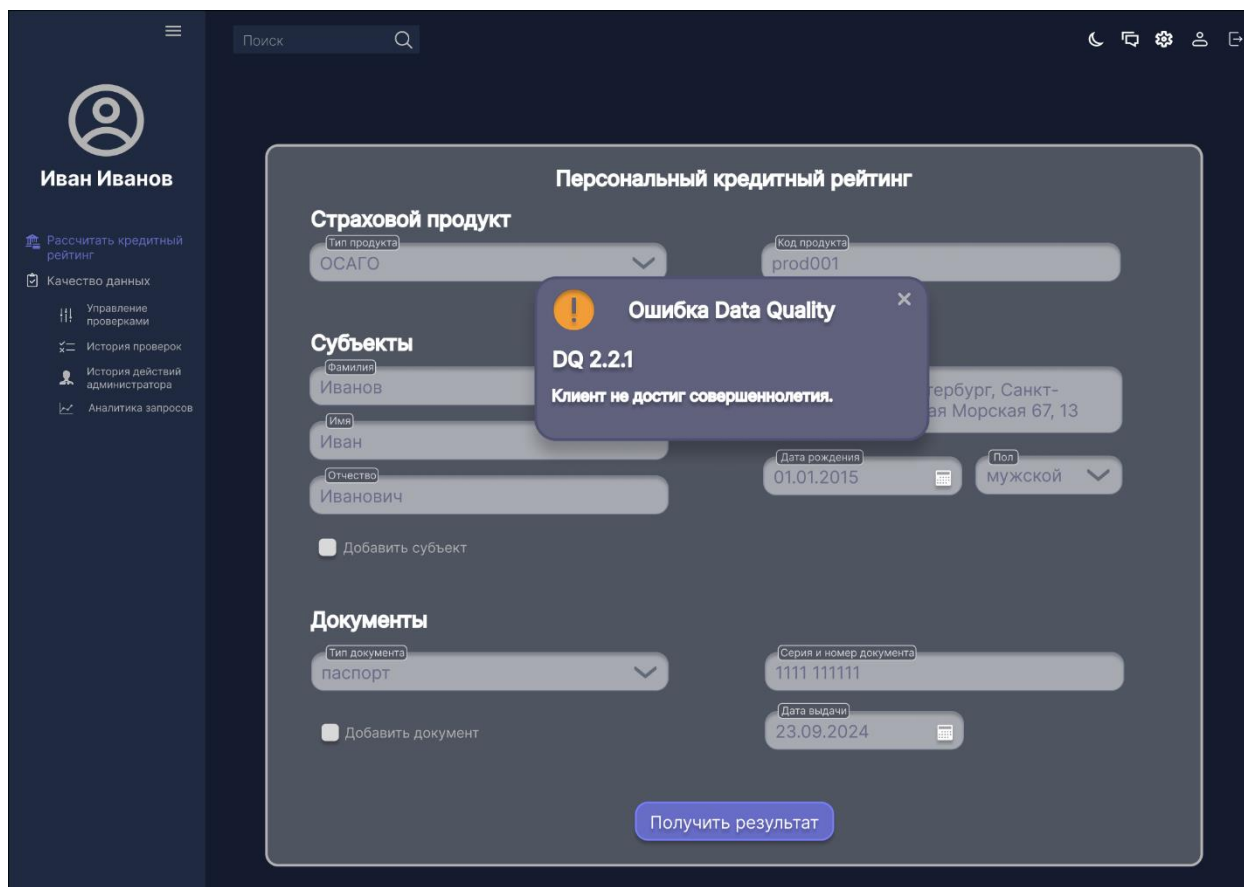


Рисунок 16 — Раздел «Рассчитать кредитный рейтинг», ошибка DQ

5.1.2 Управление проверками

Раздел, в котором происходит переключение между панелями управления проверками DQ по каждому страховому продукту, доступен только для администраторов.

Панели управления:

- 1) ОСАГО (открывается по умолчанию) – панель управления проверками DQ для страхового продукта «ОСАГО»;
- 2) другой страховой продукт (например, страхование жизни) – панель управления проверками DQ для страхового продукта «Страхование жизни».

Приоритет: высокий.

Функциональные требования

Раздел «Управление проверками» содержит следующие элементы:

- 1) переключатель между панелями управления проверками;

- 2) панель управления проверками «ОСАГО»:
 - а) выключатель проверки DQ 1;
 - б) выключатель всех проверок DQ 2;
 - в) выключатель каждой проверки DQ 2;
- 3) панель управления проверками «Страхование жизни» (или любой другой страховой продукт):
 - а) выключатель проверки DQ 1;
 - б) выключатель всех проверок DQ 2;
 - в) выключатель каждой проверки DQ 2.

При нажатии в боковом меню на раздел «Управление проверками» происходит переход на страницу с двумя панелями и переключателями проверок DQ (рисунок 17). Если пользователь попал на эту страницу без авторизации с ролью «Администратор», то доступ будет ограничен и произойдет автоматическое перенаправление на страницу с авторизацией.

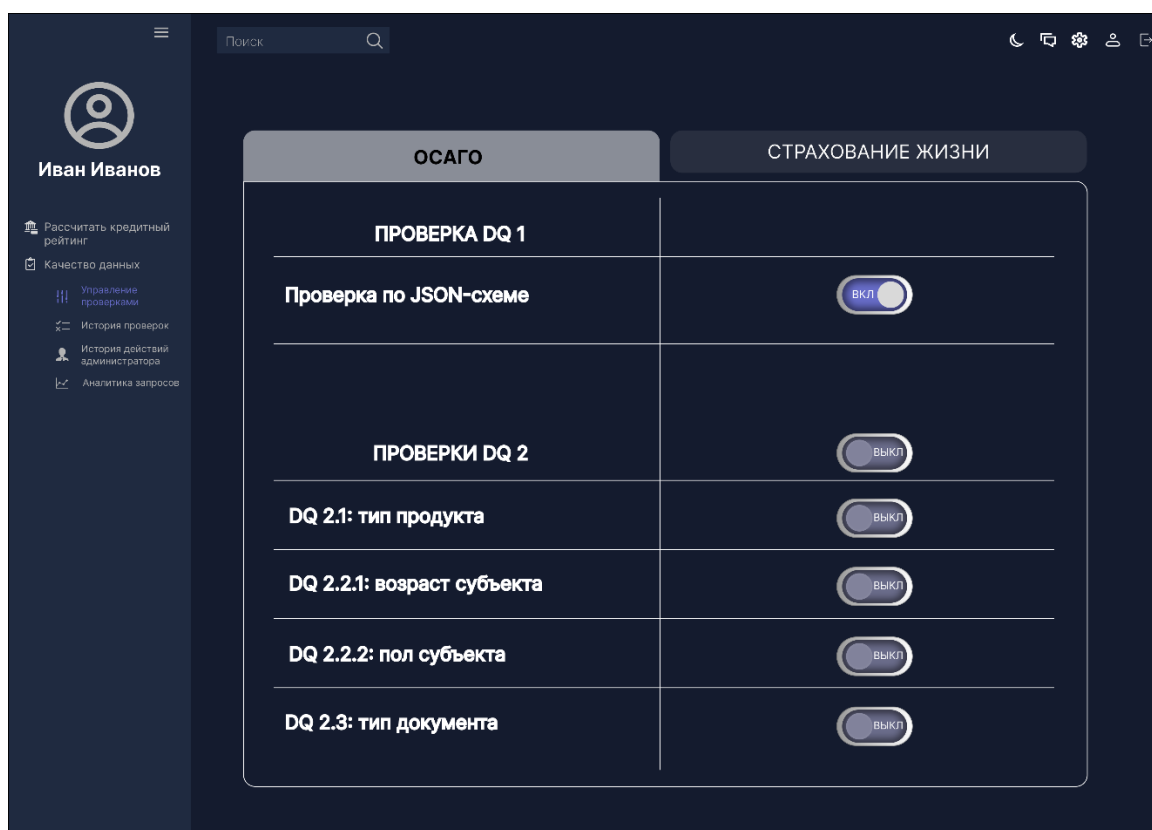


Рисунок 17 — Раздел «Управление проверками»

При инициализации веб-интерфейса осуществляется асинхронная загрузка актуальных состояний проверок качества данных из БД посредством REST API. Пользовательский интерфейс предоставляет механизм интерактивного управления статусами проверок: модификация отдельных параметров реализована через элементы управления типа «toggle switch» (ползунок). Для группы проверок категории DQ 2 предусмотрена функция массовой деактивации всех связанных правил единым действием через специализированный элемент управления.

Все операции администратора, включая активацию/деактивацию проверок, фиксируются в таблице Check_actions. Это обеспечивает соответствие требованиям аудита и анализ исторических изменений.

Также реализовано переключение между страховыми продуктами, для дальнейшего расширения функционала модуля и повторного использования кода.

Реализация функции

```
# Эндпоинт для получения статусов проверок
@app.get("/checks")
async def get_checks(conn: Annotated[asyncpg.Connection, Depends(get_db)]):
    try:
        records = await conn.fetch(
            "SELECT check_id, condition FROM Check_product_status WHERE product_code = 'prod001'"
        )
        return {rec['check_id']: rec['condition'] for rec in records}
    except Exception as e:
        raise HTTPException(500, str(e))

# Эндпоинт для обновления проверки
@app.put("/checks/{check_id}")
async def update_check(
    check_id: str,
    data: CheckUpdate,
    conn: Annotated[asyncpg.Connection, Depends(get_db)]
):
    try:
        async with conn.transaction():
            # 1. Обновляем статус проверки
            await conn.execute(
                """UPDATE Check_product_status
```

```

        SET condition = $1
        WHERE check_id = $2 AND product_code = 'prod001''''',
        data.condition,
        check_id
    )

    # 2. Записываем действие в журнал
    action_id = 2 if data.condition else 3 # 2=включение, 3=выключение
    await conn.execute(
        """INSERT INTO Check_actions
        (FIO, product_code, action_id, check_id, date)
        VALUES ($1, 'prod001', $2, $3, $4)''''',
        data.fio,
        action_id,
        check_id,
        datetime.now(timezone.utc)
    )

    return {"status": "success"}
except Exception as e:
    raise HTTPException(500, str(e))

# Массовое выключение DQ 2
@app.put("/checks/dq2/disable-all")
async def disable_all_dq2(
    data: CheckUpdate,
    conn: Annotated[asyncpg.Connection, Depends(get_db)]
):
    try:
        async with conn.transaction():
            # 1. Выключаем все DQ 2 проверки
            await conn.execute(
                """UPDATE Check_product_status
                SET condition = FALSE
                WHERE check_id LIKE 'DQ 2%'
                AND product_code = 'prod001'"""
            )

            # 2. Получаем список выключенных проверок
            disabled_checks = await conn.fetch(
                "SELECT check_id FROM Check_product_status WHERE check_id LIKE 'DQ 2%'
            )

            # 3. Записываем действия для каждой проверки
            for check in disabled_checks:
                await conn.execute(
                    """INSERT INTO Check_actions
                    (FIO, product_code, action_id, check_id, date)
                    VALUES ($1, 'prod001', 3, $2, $3)''''',
                    data.fio,
                    check['check_id'],
                    datetime.now(timezone.utc)
                )
    
```

```
)  
  
return {"disabled_count": len(disabled_checks)}  
except Exception as e:  
    raise HTTPException(500, str(e))
```

5.1.3 История проверок

Раздел, в котором отображается таблица с записями о проверках данных котировок, доступен для администраторов и пользователей.

Приоритет: высокий.

Функциональные требования

Раздел «История проверок» содержит следующие элементы:

- 1) список результатов проверки данных котировки:
 - a) строка списка содержит: checkbox для выбора строки (для администраторов), номер проверки, вид страхового продукта, статус проверки, дату проверки;
- 2) кнопка «Выбрать все строки» (для администраторов);
- 3) кнопка «Удалить записи» (для администраторов);
- 4) модальное окно «Детализация результата»:
 - a) кнопка «Закрыть»;
 - b) номер проверки;
 - c) вид страхового продукта;
 - d) дата проверки;
 - e) данные поступившей котировки в формате JSON;
 - f) подробное описание результата проверки;
- 5) панель фильтрации:
 - a) дата;
 - b) вид страхового продукта;
 - c) статус проверки.

Реализованный модуль «История проверок» предоставляет интерфейс для анализа результатов верификации данных котировок. При нажатии соответствующего пункта навигационного меню осуществляется переход к интерфейсу, визуализирующему журнал выполненных проверок в табличном формате (рисунки 18, 19). Механизм контроля доступа гарантирует соблюдение политики безопасности: при отсутствии валидной аутентификации пользователь автоматически перенаправляется на страницу авторизации, что исключает несанкционированный доступ к конфиденциальным данным.

	№ проверки	вид продукта	статус проверки	дата
☒	673	-	не пройдена	10.12.2024 12:46:54
☐	672	Страхование жизни	не пройдена	10.12.2024 11:56:01
☐	671	ОСАГО	не пройдена	09.12.2024 8:12:28
☒	670	Страхование жизни	пройдена	08.12.2024 05:46:20
☐	669	ОСАГО	пройдена	07.12.2024 23:32:10
☐	668	Страхование жизни	не пройдена	07.12.2024 15:57:13
☐	667	Страхование жизни	пройдена	07.12.2024 12:46:54
☐	666	ОСАГО	пройдена	06.12.2024 09:04:01
☒	665	ОСАГО	не пройдена	06.12.2024 07:07:07
☐	664	Страхование жизни	пройдена	06.12.2024 06:06:00

Рисунок 18 — Раздел «История проверок» для администратора

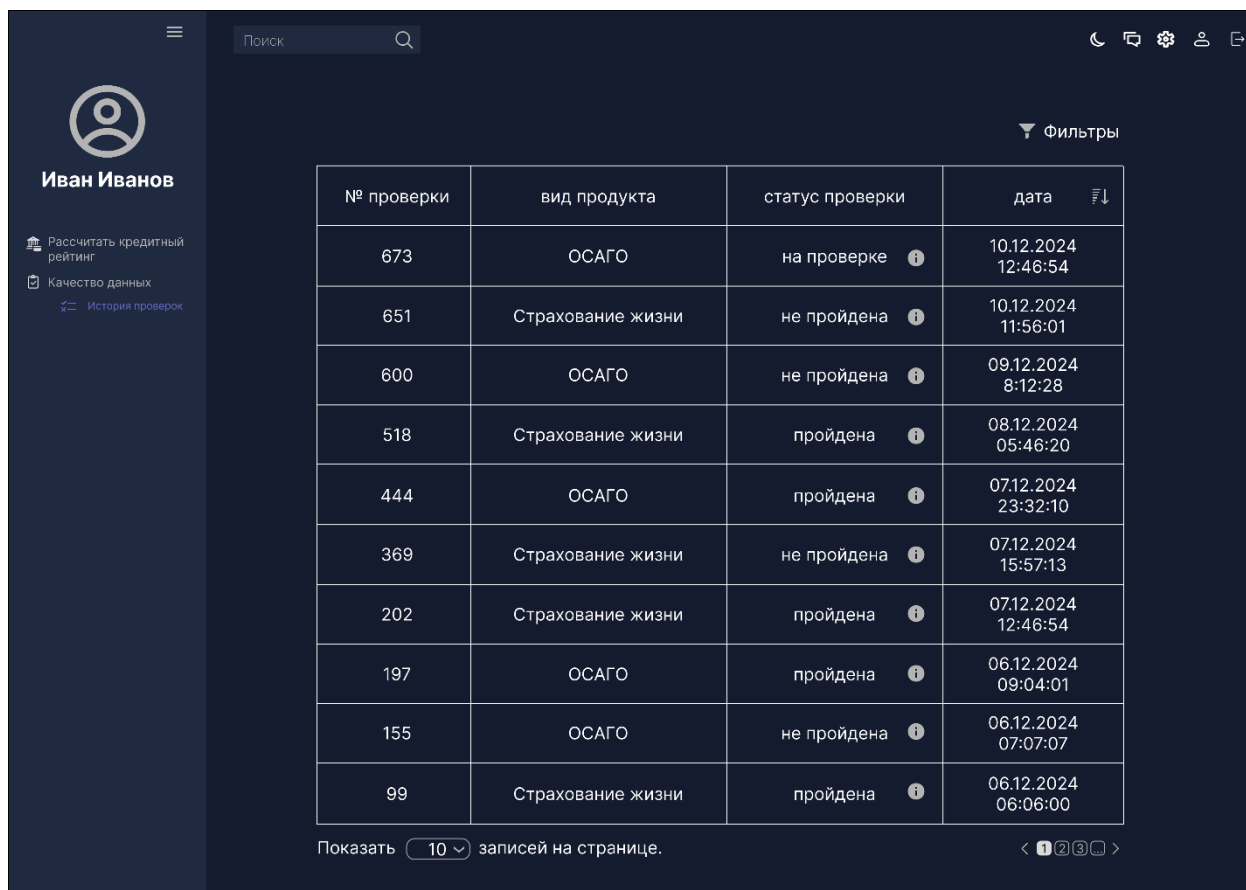


Рисунок 19 — Раздел «История проверок» для пользователя

Для роли администратор реализован расширенный функционал в виде логического удаления выбранных записей с использованием чекбоксов. Подтверждение данного действия осуществляется через модальное диалоговое окно с валидацией намерений пользователя (рисунок 20).

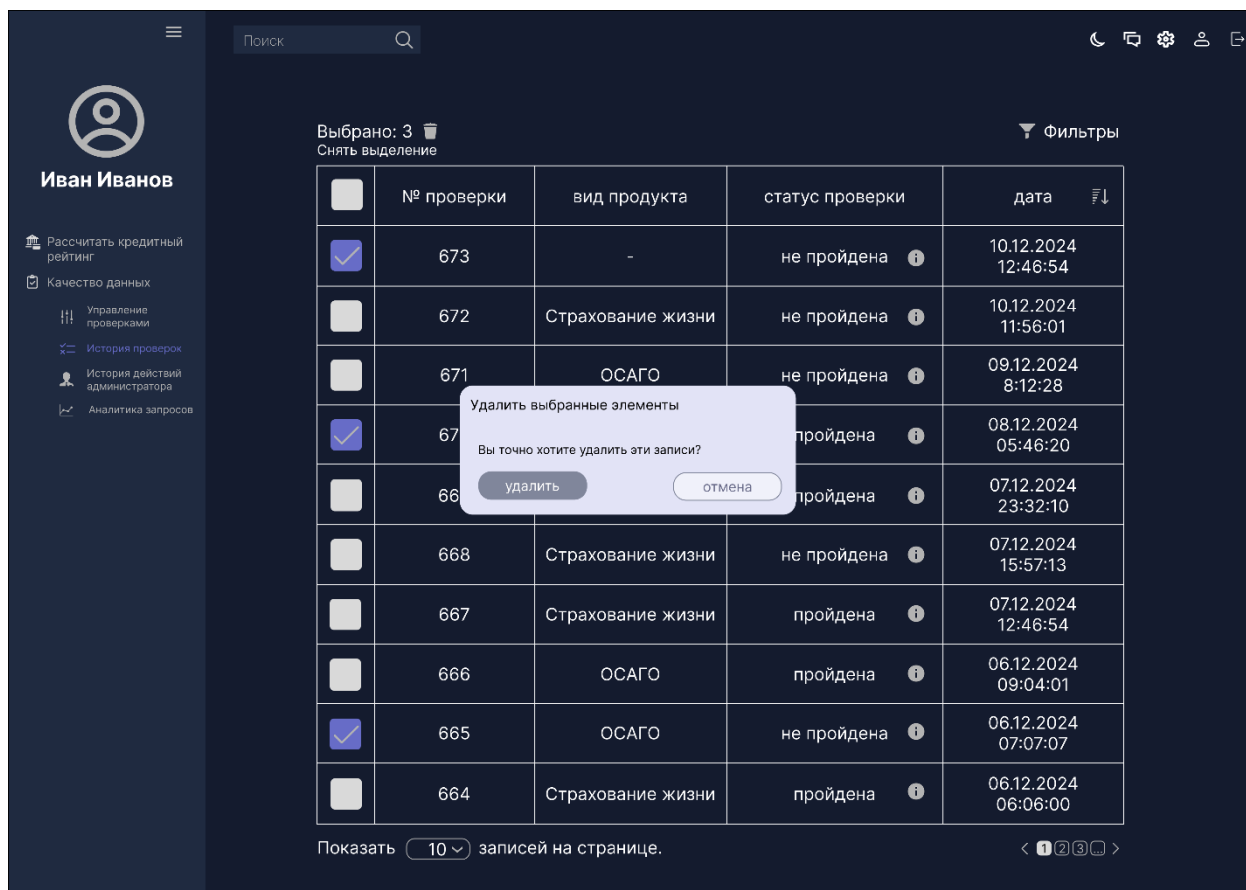


Рисунок 20 — Раздел «История проверок», удаление

На странице присутствует панель фильтрации с поддержкой комбинированных критериев: временной диапазон, категория страхового продукта, статус верификации (рисунок 21).

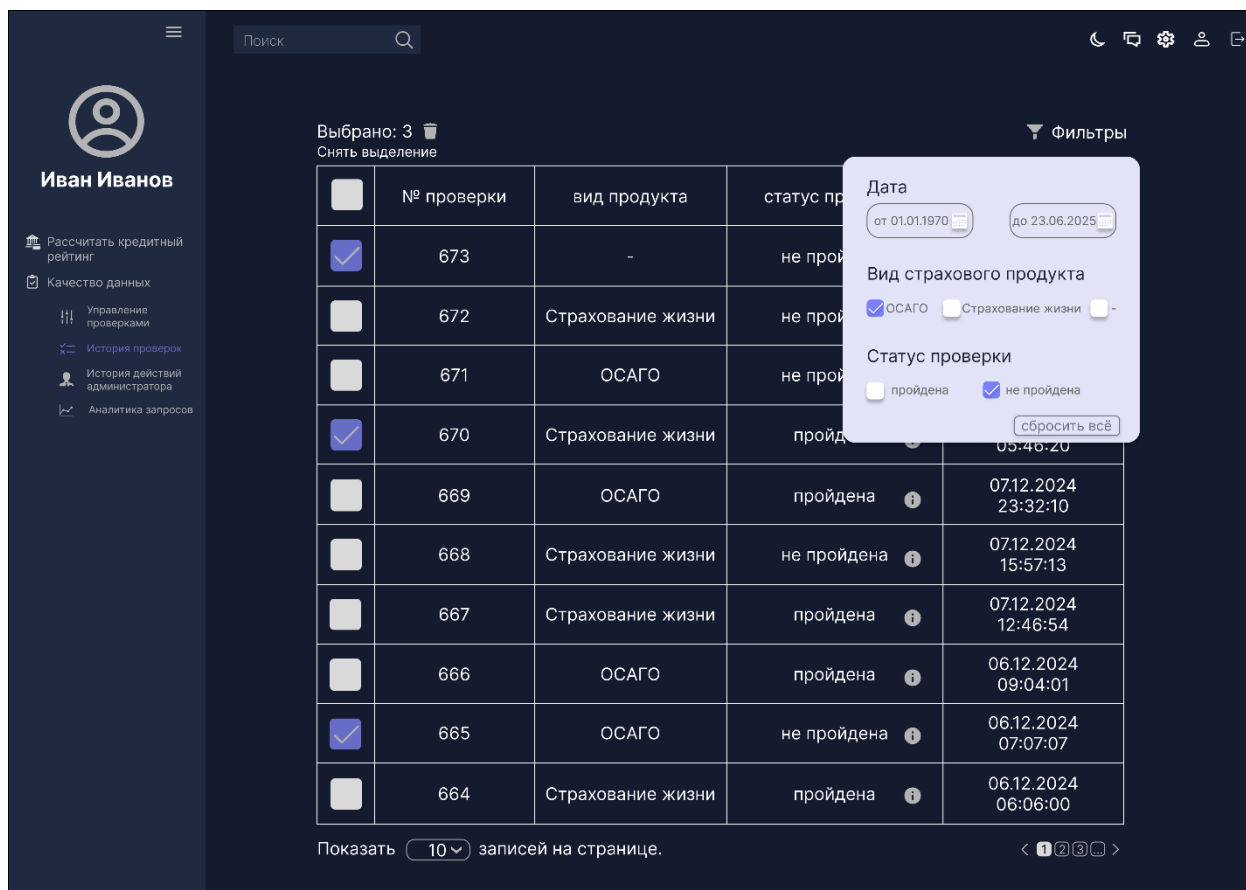


Рисунок 21 — Раздел «История проверок», фильтрация

Нажав на иконку рядом со статусом проверки, пользователь может перейти в модальное окно с детализированным результатом этой проверки, которое включает: исходные данные котировки в формате JSON, структурированное описание выявленных несоответствий (рисунок 22).

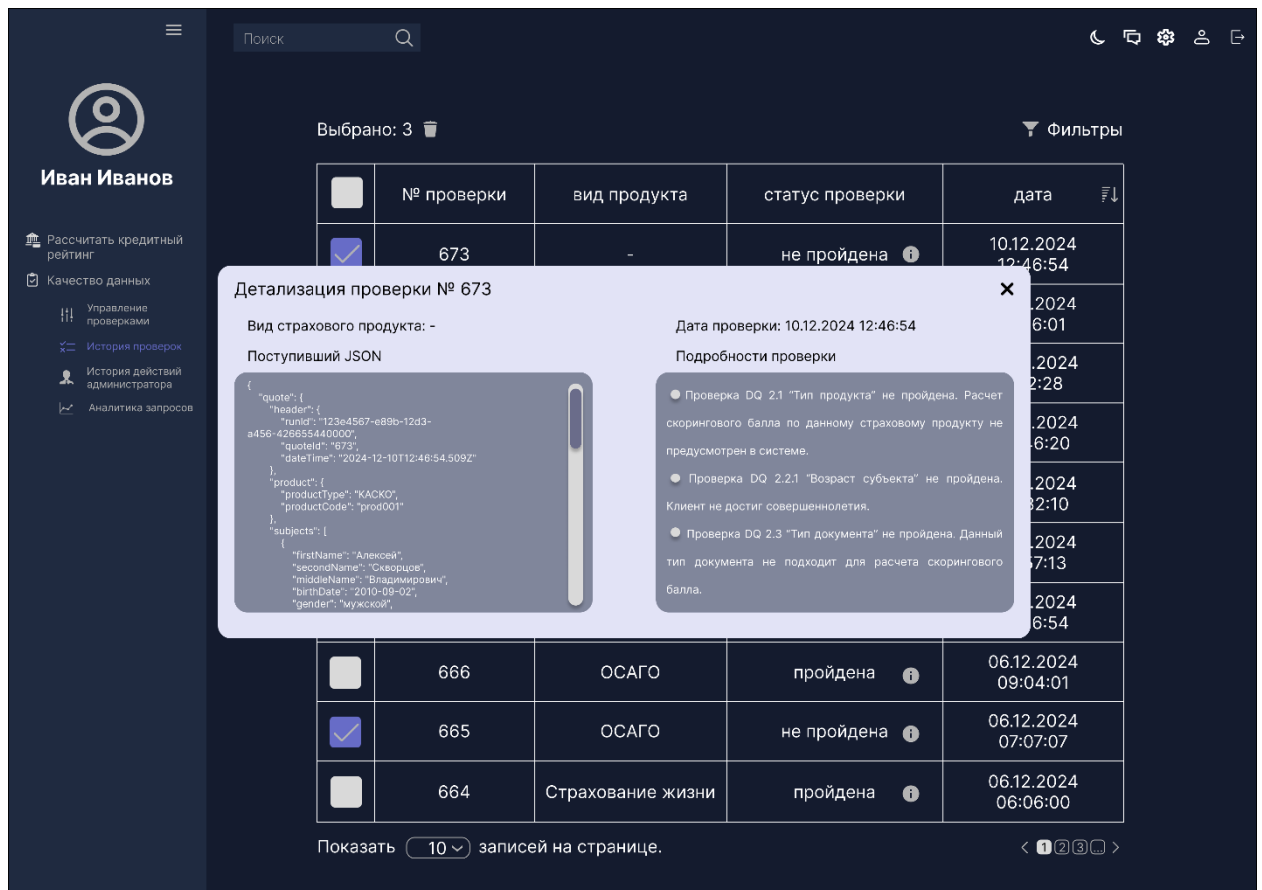


Рисунок 22 — Раздел «История проверок», детализация

Реализация функции

```
@app.get("/history")
async def get_history(
    page: int = 1,
    per_page: int = 10,
    product_type: str = None,
    status: str = None,
    conn: asyncpg.Connection = Depends(get_db)
):
```

"""

Получение пагинированной истории проверок с фильтрацией

Параметры:

- page: номер страницы
- per_page: количество записей на странице
- product_type: фильтр по типу продукта (ОСАГО, Страхование жизни, -)
- status: фильтр по статусу (пройдена/не пройдена)

"""

offset = (page - 1) * per_page

base_query = """

SELECT

r.runId AS "№ проверки",
 p.product_type AS "вид продукта",
 res.response_name AS "статус проверки",

```

        r.date AS "дата"
    FROM Requests r
    JOIN Products p ON r.product_code = p.product_code
    LEFT JOIN Request_response rr ON r.runId = rr.runId
    LEFT JOIN Responses res ON rr.response_code = res.response_code
    WHERE r.deleted = False
    """

    filters = []
    params = []

    if product_type:
        filters.append("p.product_type = $1")
        params.append(product_type)

    if status:
        filters.append("res.response_name = $2")
        params.append(status)

    if filters:
        base_query += " AND " + " AND ".join(filters)

    full_query = f"""
        {base_query}
        ORDER BY r.date DESC
        LIMIT {per_page} OFFSET {offset}
    """

    try:
        records = await conn.fetch(full_query, *params)
        return [dict(rec) for rec in records]
    except Exception as e:
        raise HTTPException(500, detail=str(e))

```

5.1.4 История действий администратора

Раздел, в котором отображается таблица с записями о действиях администратора, доступен только для администраторов.

Приоритет: высокий.

Функциональные требования

Раздел «История действий администратора» содержит следующие элементы:

- 1) список действий администратора:

- а) строка списка содержит: номер действия, ФИО администратора, вид страхового продукта, действие, дату действия;
- 2) модальное окно «Детализация действия»:
 - а) кнопка «Закрыть»;
 - б) номер действия;
 - с) дата действия;
 - д) подробное описание результата действия;
- 3) панель фильтрации:
 - а) дата;
 - б) вид страхового продукта;
 - с) ФИО администратора.

Модуль «История действий администратора» реализует функционал аудита операций, выполняемых уполномоченными сотрудниками в системе управления качеством данных. При активации соответствующего пункта навигационного меню осуществляется переход к интерфейсу, визуализирующему журнал административных действий в табличном формате (рисунок 23). Механизм контроля доступа обеспечивает соблюдение принципов информационной безопасности: в случае отсутствия валидной аутентификации пользователь автоматически перенаправляется на страницу авторизации, что исключает несанкционированный доступ к журналу операций.

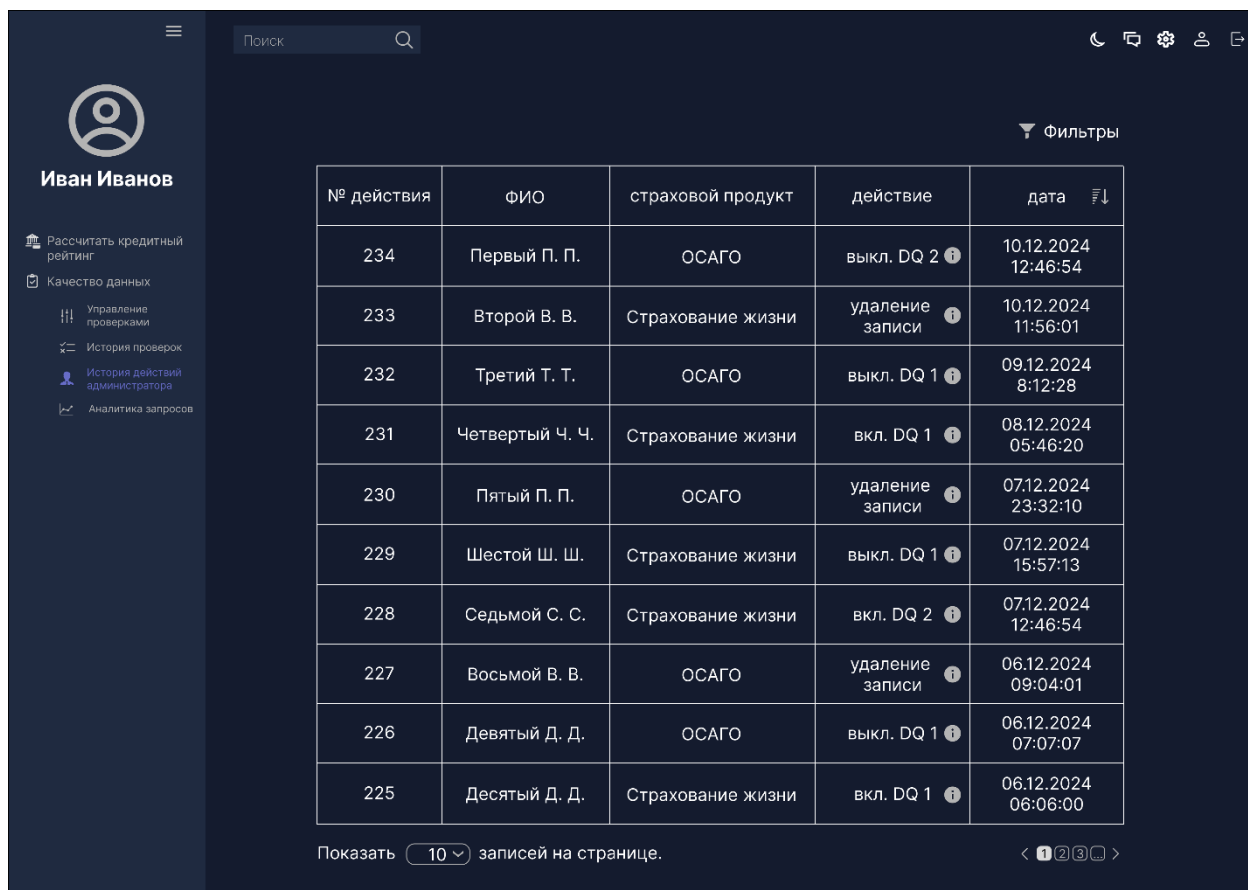


Рисунок 23 — Раздел «История действий администратора»

На странице находится панель фильтрации (рисунок 24) с поддержкой комбинированных критериев: временной диапазон, категория страхового продукта, ФИО администратора.

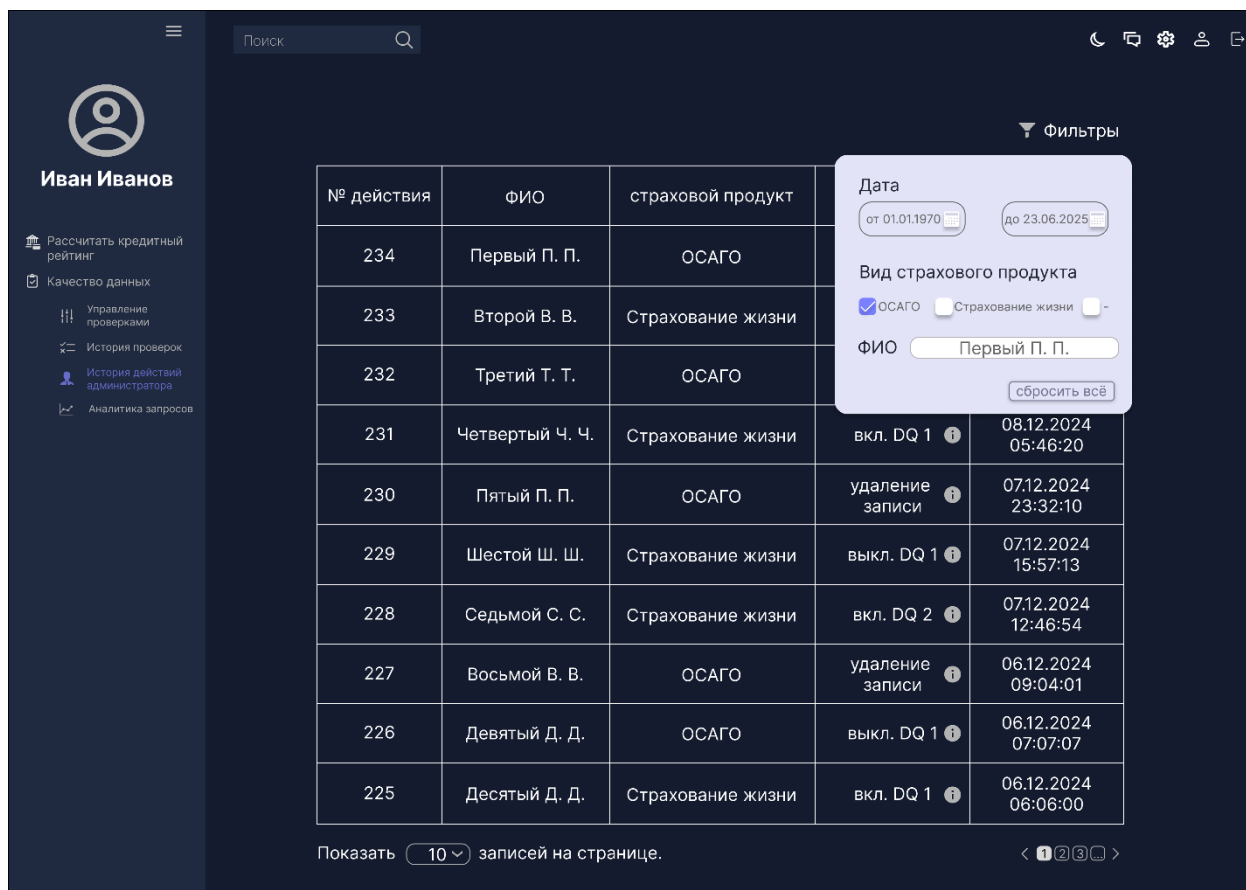


Рисунок 24 — Раздел «История действий администратора», фильтрация

Нажав на иконку рядом с названием действия, пользователь может перейти в модальное окно с детализированным результатом (рисунок 25), которое включает подробное описание совершенного в системе действия.

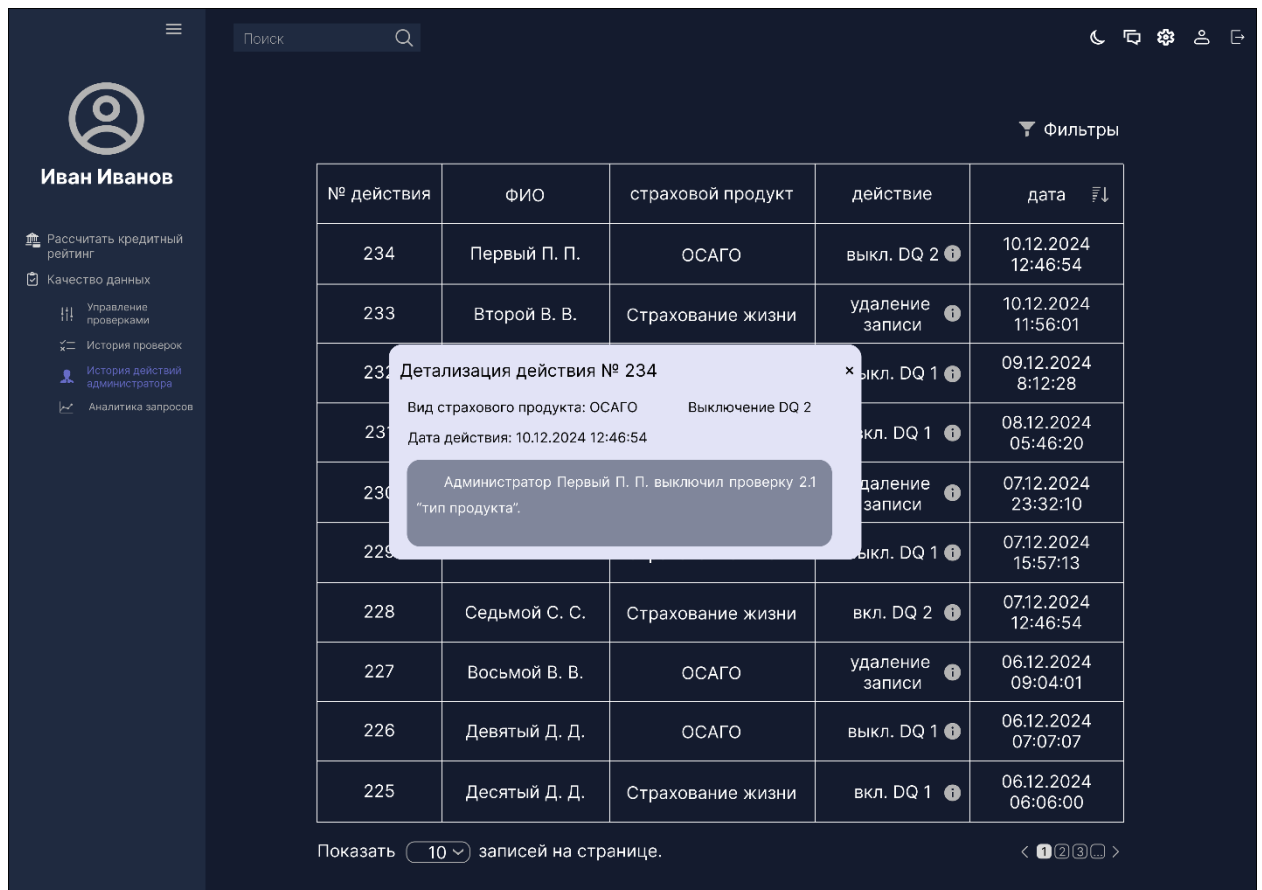


Рисунок 25 — Раздел «История действий администратора», детализация

Реализация функции

```
@app.get("/admin-actions")
async def get_admin_actions(
    page: int = Query(1, ge=1),
    per_page: int = Query(10, ge=1, le=100),
    product_type: str = Query(None),
    action_type: str = Query(None),
    start_date: datetime = Query(None),
    end_date: datetime = Query(None),
    conn: asyncpg.Connection = Depends(get_db)
):
    """
```

Получение журнала действий администратора с фильтрацией и пагинацией.

Параметры:

- page: номер страницы
- per_page: записей на странице
- product_type: фильтр по продукту (ОСАГО, Страхование жизни, -)
- action_type: фильтр по типу действия (вкл./выкл. DQ 1/DQ 2, удаление)
- start_date/end_date: временной диапазон

```
offset = (page - 1) * per_page
query = """
```

```

SELECT
    ca.id AS "№ действия",
    ca.FIO AS "ФИО",
    p.product_type AS "страховой продукт",
    a.action_name || ' ' || c.type AS "действие",
    ca.date AS "дата"
FROM Check_actions ca
JOIN Products p ON ca.product_code = p.product_code
JOIN Actions a ON ca.action_id = a.action_id
LEFT JOIN Checks c ON ca.check_id = c.check_id
WHERE ca.deleted = False
"""

params = []
conditions = []
if product_type:
    conditions.append("p.product_type = $1")
    params.append(product_type)
if action_type:
    conditions.append("(a.action_name || ' ' || COALESCE(c.type, '')) ILIKE $2")
    params.append(f"%{action_type}%")
if start_date and end_date:
    conditions.append("ca.date BETWEEN $3 AND $4")
    params.extend([start_date, end_date])
if conditions:
    query += " AND " + " AND ".join(conditions)
query += f" ORDER BY ca.date DESC LIMIT {per_page} OFFSET {offset}"
try:
    records = await conn.fetch(query, *params)
    return [dict(rec) for rec in records]
except Exception as e:
    raise HTTPException(500, detail=str(e))

@app.delete("/admin-actions/{action_id}")
async def soft_delete_action(
    action_id: int,
    conn: asyncpg.Connection = Depends(get_db)
):
    """Пометить действие как удалённое (soft delete)"""
    try:
        await conn.execute(
            "UPDATE Check_actions SET deleted = TRUE WHERE id = $1",
            action_id
        )
        return {"status": "success"}
    except Exception as e:
        raise HTTPException(500, detail=str(e))

```

5.1.5 Аналитика запросов

Раздел, в котором отображаются бизнес-метрики сервиса DQ.

Приоритет: высокий.

Функциональные требования

Раздел «Аналитика запросов» содержит следующие элементы:

- 1) переключатель между панелями с бизнес-метриками сервиса DQ;
- 2) панель бизнес-метрик по «ОСАГО»:
 - a) количество успешных проверок – числовая метрика;
 - b) количество неуспешных проверок – числовая метрика;
 - c) общее количество проверок – числовая метрика;
 - d) топ-5 причин неуспешного результата проверки – список;
 - e) процент успешных проверок – круговая диаграмма;
 - f) процент неуспешных проверок – круговая диаграмма;
 - g) панель фильтрации;
- 3) панель бизнес-метрик по «Страхованию жизни» (любой другой страховой продукт):
 - a) количество успешных проверок – числовая метрика;
 - b) количество неуспешных проверок – числовая метрика;
 - c) общее количество проверок – числовая метрика;
 - d) топ-5 причин неуспешного результата проверки – список;
 - e) процент успешных проверок – круговая диаграмма;
 - f) процент неуспешных проверок – круговая диаграмма;
 - g) панель фильтрации.

Модуль «Аналитика запросов» реализует функционал визуализации главных бизнес-метрик, связанных с верификацией данных в системе управления качеством. При активации соответствующего пункта навигационного меню пользователь перенаправляется на специализированную панель мониторинга (рисунок 26). В случае отсутствия

валидной сессии осуществляется автоматическое перенаправление на интерфейс авторизации.

Метрики формируются на основе агрегированных записей из таблиц БД (Requests, Responses, Products) посредством SQL-запросов, приведённых в разделе 4.2.2.

Раздел включает в себя: сравнительный анализ успешных/неуспешных проверок по страховым продуктам, ранжированный перечень частых причин отклонений, интерактивные элементы фильтрации по временным диапазонам.

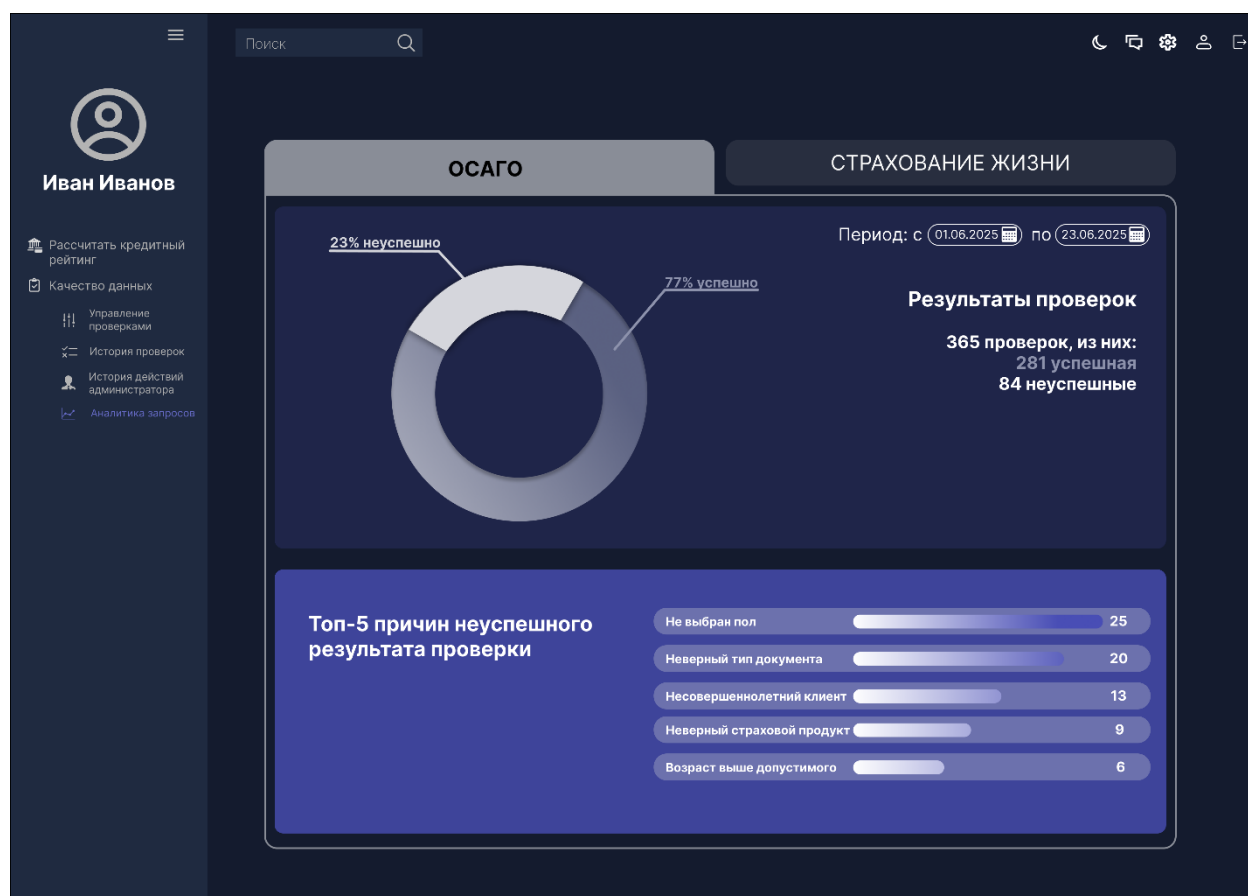


Рисунок 26 — Раздел «Аналитика запросов»

Реализация функции

```
// Загрузка данных
useEffect() => {
  const fetchData = async () => {
    try {
      const [summary, total, failures] = await Promise.all([
        axios.get('/analytics/summary', {
          params: {
            start_date: startDate.toISOString(),

```

```

        end_date: endDate.toISOString()
    }
  ),
  axios.get('/analytics/total', {
    params: {
      start_date: startDate.toISOString(),
      end_date: endDate.toISOString()
    }
  )),
  axios.get('/analytics/top-failures', {
    params: {
      start_date: startDate.toISOString(),
      end_date: endDate.toISOString(),
      limit: 5
    }
  })
]);

const productData = summary.data.find(
  item => item.product_type === selectedProduct
);

setMetrics({
  success: productData?.successful_checks || 0,
  failed: productData?.unsuccessful_checks || 0,
  total: total.data.find(
    item => item.product_type === selectedProduct
  )?.total || 0,
  topReasons: failures.data.filter(
    item => item.product_type === selectedProduct
  )
});
} catch (error) {
  console.error('Ошибка загрузки данных:', error);
}
};
fetchData();
}, [selectedProduct, startDate, endDate]);

// Конфигурация круговой диаграммы
const chartData = {
  labels: ['успешно', 'неуспешно'],
  datasets: [{
    data: [metrics.success, metrics.failed],
    backgroundColor: [chartColors.success, chartColors.failed],
    hoverBackgroundColor: [chartColors.success, chartColors.failed]
  }
  ]
};
return (
  <div className="analytics-container">

    {/* Фильтр по дате */}
    <div className="date-filter">
      <DatePicker
        selected={startDate}
        onChange={date => setStartDate(date)}
        dateFormat="dd.MM.yyyy"/>
      <span> — </span>
      <DatePicker

```

```

    selected={endDate}
    onChange={date => setEndDate(date)}
    dateFormat="dd.MM.yyyy" />
</div>
{/* Числовые метрики */}
<div className="metrics-grid">
  <div className="metric-card">
    <h3>успешные проверки</h3>
    <div className="metric-value">{metrics.success}</div>
  </div>
  <div className="metric-card">
    <h3>неуспешные проверки</h3>
    <div className="metric-value">{metrics.failed}</div>
  </div>
  <div className="metric-card">
    <h3>всего проверок</h3>
    <div className="metric-value">{metrics.total}</div>
  </div>
</div>

{/* Визуализация */}
<div className="visualization-section">
  <div className="doughnut-chart">
    <Doughnut data={chartData} />
    <div className="chart-labels">
      <span style={{ color: chartColors.success }}>
        {((metrics.success / metrics.total) * 100 || 0).toFixed(0)}% </span>
      <span style={{ color: chartColors.failed }}>
        {((metrics.failed / metrics.total) * 100 || 0).toFixed(0)}% </span>
    </div>
  </div>
  <div className="top-reasons">
    <h3>Топ-5 причин ошибок</h3>
    <ol>
      {metrics.topReasons.map((reason, index) => (
        <li key={index}>
          {reason.reason} - {reason.reason_count}
        </li>))}
    </ol>
  </div>
</div>

```

6 АНАЛИЗ ЭФФЕКТИВНОСТИ ВНЕДРЕНИЯ МОДУЛЯ АВТОМАТИЗАЦИИ КОНТРОЛЯ КАЧЕСТВА ДАННЫХ И ОЦЕНКА ЗНАЧИМОСТИ ИЗМЕНЕНИЙ ОСНОВНЫХ ПОКАЗАТЕЛЕЙ

6.1 Методология анализа

Для оценки эффективности внедрения модуля Data Quality в автоматизированную систему оценки страховых рисков используются сравнительный анализ метрик до появления модуля и после, а также статистические методы для проверки значимости изменений.

Основные этапы анализа:

1. Сбор данных: логи ошибок, метрики времени обработки, результаты опросов.
2. Расчет ключевых метрик для двух периодов (до/после).
3. Применение статистических тестов для подтверждения значимости изменений.
4. Интерпретация результатов в контексте бизнес-показателей.

6.2 Методы расчета и интерпретация метрик

Таблица 10 – Метрики оценки качества данных и методы их анализа

Метрика	Описание	Метод анализа	Источник данных
Точность данных	Сравнение доли ошибок до/после	Хи-квадрат Пирсона	Логи ошибок валидации из системы, отчёты DQ-модуля о некорректных полях, журнал отклонённых запросов
Скорость обработки	Сравнение среднего времени выполнения запроса	Т-критерий Стьюдента с поправкой Уэлча для независимых выборок	Логи времени обработки запросов, системные метрики

Продолжение таблицы 10

Качество скоринга	Сравнение точности прогнозов	ROC-анализ	Результаты предсказаний модели, ручная проверка экспертами (эталонные данные)
Уровень автоматизации	Сравнение времени, затрачиваемого сотрудниками на ручные проверки	U-критерий Манна-Уитни	Тайм-трекинг сотрудников (Jira), логи рабочих процессов
Удовлетворенность клиентов	Сравнение индекса потребительской лояльности (NPS)	Z-тест для оценки разности между двумя долями признака	Результаты опросов клиентов (анкеты), журнал обращений в техподдержку

6.2.1 Точность данных

В автоматизированной системе оценки страховых рисков точность входных данных является критически значимым параметром, который непосредственно влияет на достоверность прогноза моделей. Некорректные значения в ведущих полях (дата рождения, номер документа) приводят к серьёзным ошибкам. Благодаря модулю DQ автоматизированная валидация по заранее заданным правилам минимизирует вероятность ошибок ввода и преобразования данных.

Расчёт метрики

Определим гипотезы: нулевая гипотеза (H_0) – внедрение модуля не повлияло на точность данных, альтернативная гипотеза (H_1) – внедрение модуля привело к повышению точности данных.

Построим таблицу сопряженности (таблица 11).

Таблица 11 – Таблица сопряженности (2000 записей)

	Ошибки	Нет ошибок	Всего
	Наблюдаемые (Obs) – Абсолютные (относительные)		
До внедрения	147 (88,6 %)	853 (46,5 %)	1000 (50 %)
После внедрения	19 (11,4 %)	981 (53,5 %)	1000 (50 %)
Всего	166 (8,3 %)	1834 (91,7 %)	2000 (100 %)
	Ожидаемые (Exp) – Абсолютные		
До внедрения	1000*8,3 %=83	1000*91,7 %=917	1000
После внедрения	1000*8,3 %=83	1000*91,7 %=917	1000
Всего	166	1834	2000

Произведем расчет хи-квадрат по формуле:

$$\chi^2 = \sum \frac{(Obs - Exp)^2}{Exp},$$

где χ^2 – значение статистики хи-квадрат;

Σ – сумма по всем ячейкам таблицы 11;

Obs (observed) – наблюдаемая частота;

Exp (expected) – ожидаемая частота.

$$\chi^2 = \frac{(147 - 83)^2}{83} + \frac{(853 - 917)^2}{917} + \frac{(19 - 83)^2}{83} + \frac{(981 - 917)^2}{917} = 107,6.$$

Число степеней свободы = $(2 - 1) * (2 - 1) = 1$, уровень значимости 0,05. Согласно таблице критических значение хи-квадрата:

Критические значения критерия Пирсона (χ^2 -критерия) для различного уровня значимости q (%) и числа степеней свободы v

Число степеней свободы, v	Уровень значимости, q, %							
	20	10	5	2	1	0,5	0,2	0,1
1	1,642	2,706	3,841	5,412	6,635	7,879	9,550	10,83
2	3,219	4,605	5,991	7,824	9,210	10,60	12,43	13,82

Рисунок 27 — Критические значения критерия Пирсона

Критическое значение $\chi^2_{\text{крит}} = 3,841$.

Так как суммарное значение $\chi^2 = 107,6$ значительно превышает критическое значение $\chi^2_{\text{крит}} = 3,841$, то нулевая гипотеза отвергается в пользу H_1 . Статистически значимое снижение доли ошибок подтверждает повышение точности данных после внедрения модуля ($\alpha < 0,05$).

Проведем расчеты для меньшей выборки:

Таблица 12 – Таблица сопряженности (200 записей)

	Ошибки	Нет ошибок	Всего
	Наблюдаемые (Obs) – Абсолютные (относительные)		
До внедрения	17 (81 %)	83 (46,4 %)	100 (50 %)
После внедрения	4 (19 %)	96 (53,6 %)	100 (50 %)
Всего	21 (10,5 %)	179 (89,5 %)	200 (100 %)
	Ожидаемые (Exp) – Абсолютные		
До внедрения	$100 * 10,5 \% = 10,5$	$100 * 89,5 \% = 89,5$	100
После внедрения	$100 * 10,5 \% = 10,5$	$100 * 89,5 \% = 89,5$	100
Всего	21	179	200

$$\chi^2 = \frac{(17 - 10,5)^2}{10,5} + \frac{(83 - 89,5)^2}{89,5} + \frac{(4 - 10,5)^2}{10,5} + \frac{(96 - 89,5)^2}{89,5} = 8,99.$$

Число степеней свободы $= (2 - 1) * (2 - 1) = 1$, уровень значимости 0,05.

Критическое значение $\chi^2_{\text{крит}} = 3,841$.

Так как суммарное значение $\chi^2 = 8,99$ превышает критическое значение $\chi^2_{\text{крит}} = 3,841$, то нулевая гипотеза отвергается в пользу H_1 : точность данных повысилась после внедрения модуля DQ.

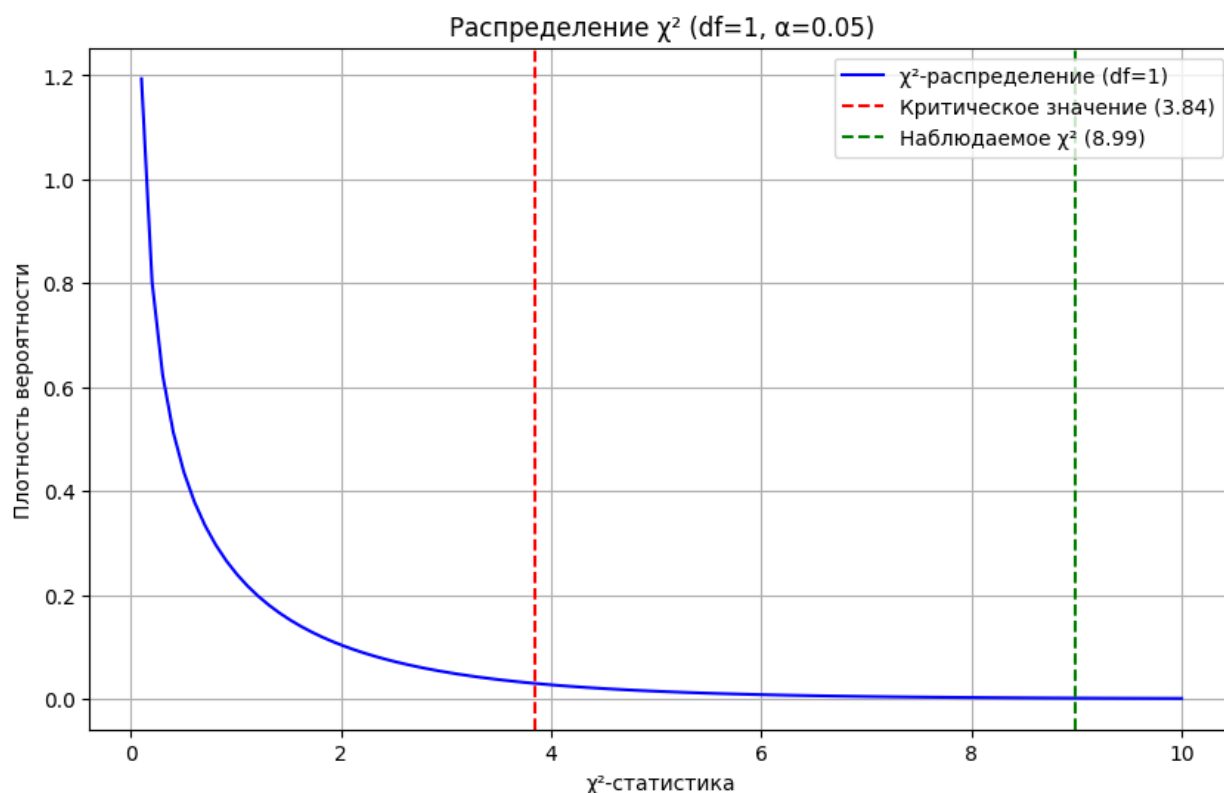


Рисунок 28 — График распределения хи-квадрат

Для построения графика распределения хи-квадрат используем Python (код представлен в приложении Г).

На рисунке 28 область принятия гипотезы находится в интервале от 0 до 3,841; точка 3,841 является критическим значением; область отклонения нулевой гипотезы лежит в интервале (3,841; $+\infty$).

6.2.2 Скорость обработки запросов

Временная эффективность системы напрямую коррелирует с пропускной способностью, удовлетворенностью клиентов, операционными затратами. Оптимизация процесса обработки котировок реализуется за счёт: параллельного выполнения независимых проверок DQ 2 (продукт, документ, субъект); кэширования результатов проверок справочных значений.

Расчёт метрики

Определим гипотезы: нулевая гипотеза (H_0) – средняя скорость обработки данных после внедрения модуля не изменилась/снизилась;

альтернативная гипотеза (H_1) – внедрение модуля привело к повышению средней скорости обработки данных.

В рамках исследования эффективности внедрения модуля качества данных были получены данные:

Таблица 13 – Исходные данные (2000 записей) для t-статистики

	Объём выборок	Выборочные средние	Несмещенные оценки дисперсий
До внедрения	1000 записей	123,49 сек./запрос	232,27
После внедрения	1000 записей	29,99 сек./запрос	25,85

Выполним проверку равенства дисперсий (F-тест на равенство дисперсий):

$$F = \frac{\hat{\sigma}_x^2}{\hat{\sigma}_y^2},$$

где $\hat{\sigma}_x^2$ и $\hat{\sigma}_y^2$ – несмещенные оценки дисперсий.

$$F = \frac{232,27}{25,85} \approx 8,99.$$

Число степеней свободы $df_1, df_2 = 999$, уровень значимости 0,05. Согласно MS Excel и введенной в нём формуле =FРАСПОБР(0,05;999;999): критическое значение $F_{\text{крит}} \approx 1,1$.

Суммарное значение $F = 8,99$ значительно превышает критическое значение $F_{\text{крит}} \approx 1,1$, значит нулевая гипотеза о равенстве дисперсий отвергается в пользу H_1 при уровне значимости $\alpha = 0,05$. Так как дисперсии не равны, используем t-тест с поправкой Уэлча.

t-статистика для проверки нулевой гипотезы равна:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}},$$

где $\bar{X}_1, \bar{X}_2$ – выборочные средние;

s_1^2, s_2^2 – несмещенные оценки дисперсий;

n_1, n_2 – объемы выборок.

$$t = \frac{123,49 - 29,99}{\sqrt{\frac{232,27}{1000} + \frac{25,85}{1000}}} = 184,04.$$

Степени свободы по Уэлчу:

$$v = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{1}{n_1 - 1} \left(\frac{s_1^2}{n_1}\right)^2 + \frac{1}{n_2 - 1} \left(\frac{s_2^2}{n_2}\right)^2};$$
$$v = \frac{\left(\frac{232,27}{1000} + \frac{25,85}{1000}\right)^2}{\frac{1}{999} \left(\frac{232,27}{1000}\right)^2 + \frac{1}{999} \left(\frac{25,85}{1000}\right)^2} \approx 1218,64.$$

Согласно формуле `scipy.stats.t.sf(184.04, df=1218.64)` в Python: p – значение ≈ 0 . Таким образом p – значение $< 0,05$, значит нулевая гипотеза отвергается в пользу H_1 . Статистически доказано, что после внедрения модуля скорость обработки данных повысилась на: $\Delta = \left(1 - \frac{29,99}{123,49}\right) * 100 \% = 75,71 \%$.

Для комплексной оценки дополним анализ расчетом доверительного интервала разницы средних:

$$95 \% CI = (\bar{X}_1 - \bar{X}_2) \pm t_{\frac{\alpha}{2}, v} * \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}.$$

Согласно формуле `stats.t.ppf(1 - 0.05/2, 1218.64)` в Python: $t \approx 1,96$ (значение близко к нормальному распределению).

$$95 \% CI = (123,49 - 29,99) \pm 1,96 * \sqrt{\frac{232,27}{1000} + \frac{25,85}{1000}}.$$

Границы доверительного интервала:

$$95 \% CI \approx [92,5; 94,5] \text{ сек.}$$

Разница среднего времени обработки составляет 93,5 сек., и с вероятностью 95 % истинная разница лежит в интервале [92,5; 94,5] сек.

Это подтверждает, что внедрение модуля сократило время обработки на 92,5 – 94,5 сек., что согласуется с заявленными 75,71 %.

Проведем расчеты для меньшей выборки:

Таблица 14 – Исходные данные (200 записей) для t-статистики

	Объём выборок	Выборочные средние	Несмещенные оценки дисперсий
До внедрения	100 записей	126,93 сек./запрос	169,55
После внедрения	100 записей	31,63 сек./запрос	19,66

Выполним проверку равенства дисперсий (F-тест на равенство дисперсий):

$$F = \frac{\hat{\sigma}_x^2}{\hat{\sigma}_y^2},$$

где: $\hat{\sigma}_x^2$ и $\hat{\sigma}_y^2$ – несмещенные оценки дисперсий.

$$F = \frac{169,55}{19,66} \approx 8,62.$$

Число степеней свободы $df_1, df_2 = 99$, уровень значимости 0,05. Согласно MS Excel и введенной в нём формуле =FРАСПОБР(0,05;99;99): критическое значение $F_{\text{крит}} \approx 1,39$.

Суммарное значение $F = 8,62$ значительно превышает критическое значение $F_{\text{крит}} \approx 1,39$, значит нулевая гипотеза о равенстве дисперсий отвергается в пользу H_1 при уровне значимости $\alpha = 0,05$. Так как дисперсии не равны, используем t-тест с поправкой Уэлча.

t-статистика для проверки нулевой гипотезы равна:

$$t = \frac{126,93 - 31,63}{\sqrt{\frac{169,55}{100} + \frac{19,66}{100}}} = 69,28.$$

Степени свободы по Уэлчу:

$$\nu = \frac{\left(\frac{169,55}{100} + \frac{19,66}{100}\right)^2}{\frac{1}{99}\left(\frac{169,55}{100}\right)^2 + \frac{1}{99}\left(\frac{19,66}{100}\right)^2} \approx 121,65.$$

Согласно формуле `scipy.stats.t.sf(69.28, df=121.65)` в Python: p – значение = $6,59 * 10^{-100} \approx 0$. Таким образом p – значение $< 0,05$, значит нулевая гипотеза отвергается в пользу H_1 . Статистически доказано, что после внедрения модуля скорость обработки данных повысилась на: $\Delta = \left(1 - \frac{31,63}{126,93}\right) * 100 \% = 75,08 \%$.

6.2.3 Качество скоринга

ROC-кривая (англ. Receiver Operating Characteristic, рабочая характеристика приёмника) – график, позволяющий оценить качество бинарной классификации. Кривая отображает соотношение при изменении порога классификации между:

- True Positive Rate (TPR) – долей верно классифицированных положительных примеров (клиентов с ДТП):

$$TPR = \frac{TP}{TP + FN},$$

где TP (True Positive) – корректно предсказанные страховые случаи; FN (False Negative) – ложноотрицательные прогнозы.

- False Positive Rate (FPR) – долей ошибочно классифицированных отрицательных примеров (клиентов без ДТП):

$$FPR = \frac{FP}{FP + TN},$$

где FP (False Positive) – ложноположительные прогнозы; TN (True Negative) – корректно предсказанные безаварийные клиенты.

Количественная интерпретация ROC даёт показатель AUC (англ. Area Under Curve, площадь под кривой) – площадь, ограниченная ROC-кривой и осью доли ложноположительных классификаций.

Точность прогнозных моделей определяет финансовую устойчивость страхового продукта:

- ложноотрицательные оценки (недооценка риска) влекут за собой рост убыточности портфеля;
- ложноположительные оценки (переоценка риска) способствуют потере клиентов из-за завышенных тарифов.

Для верификации гипотезы о повышении точности прогнозирования страховых рисков после внедрения модуля контроля качества данных был применен ROC-анализ. В качестве целевой переменной (target variable) использовался бинарный индикатор наступления страхового случая (ДТП) в течение срока действия полиса ОСАГО:

- 1 (positive class) – наличие страхового случая;
- -1 (negative class) – отсутствие страхового случая.

Для минимизации сезонных искажений сравнивались два временных интервала:

- контрольная группа: клиенты, получившие полисы в период январь–май 2024 (до внедрения DQ);
- экспериментальная группа: клиенты, получившие полисы в период январь–май 2025 (после внедрения DQ).

Объем выборок составил: до внедрения DQ – 1027 наблюдений; после внедрения DQ – 1051 наблюдение.

Фрагмент структурированных данных экспериментальной группы представлен в таблице 15.

Таблица 15 – Репрезентативная выборка для ROC-анализа

Порядковый номер	Скоринговый балл	Фактический исход (ДТП)
1	151	1
2	380	1
3	726	-1

Продолжение таблицы 15

...		
1049	946	-1
1050	449	-1
1051	238	1

Алгоритм проведения ROC-анализа:

1. Все записи в выборках упорядочиваются по убыванию скорингового балла.
2. Для каждого порогового значения вычисляются $TPR(t)$ и $FPR(t)$.
3. На плоскости (FPR, TPR) визуализируется зависимость, которая характеризует компромисс между чувствительностью и специфичностью модели.
4. Рассчитывается интеграл под ROC-кривой для количественной оценки качества модели.

График ROC-кривой, представленный на рисунке 29, визуализирует эффективность внедрения модуля проверки качества данных: до внедрения модуля DQ кривая ($AUC = 0,79$) находится ближе к диагонали угадывания ($AUC = 0,5$), а после ($AUC = 0,95$) – стремится к левому верхнему углу ($AUC = 1,0$ – идеальная модель).

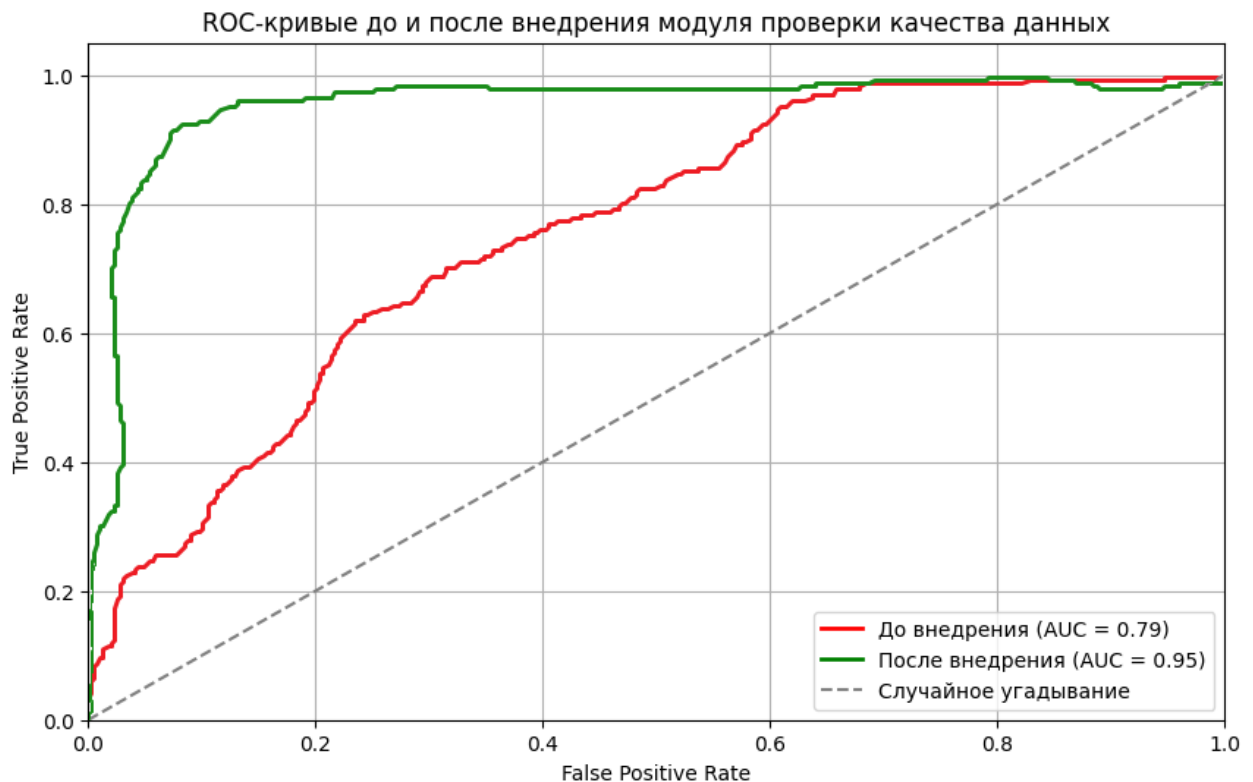


Рисунок 29 — График ROC-кривой

Статистически проверим значимость различий между двумя вариантами AUC, используя тест Делонга.

Запустим непараметрический тест Делонга для сравнения ROC-кривых. Сравниваем модели до и после внедрения модуля проверки качества данных:

```
> roc.test(roc_before, roc_after, method="delong")
```

Проанализируем полученные результаты (рисунок 30):

1. Большое, по абсолютной величине, значение Z-оценки свидетельствует о том, что разница AUC между двумя ROC-кривыми является статистически значимой и не случайной.
2. Р-значение значительно меньше, чем 0,05, что также подтверждает статистическую значимость разности площадей под кривыми.
3. Доверительный интервал $[-0,09; -0,06]$ не содержит нуля, поэтому, аналогично результатам выше, разница между AUC значима.

DeLong's test for two correlated ROC curves

```
data: roc_before and roc_after
Z = -10.125, p-value < 2.2e-16
alternative hypothesis: true difference in AUC is not equal to 0
95 percent confidence interval:
 -0.09427763 -0.06369837
sample estimates:
AUC of roc1 AUC of roc2
 0.792186   0.954271
```

Рисунок 30 — Результаты непараметрического теста Делонга

Результаты теста Делонга подтверждают, что между двумя представленными ROC-кривыми существует статистически значимая разница. После внедрения модуля проверки качества данных модель имеет значительно более высокое значение AUC, что свидетельствует о ее лучшей производительности по сравнению с моделью до внедрения.

6.2.4 Уровень автоматизации

Ключевым аспектом уровня автоматизации является сравнение времени, затраченного на выполнение ручных проверок. Для того чтобы оценить, существует ли статистическая разница во времени до и после внедрения модуля, используем U-критерий Манна-Уитни.

Обозначим: $X_1 = \{x_{11}, x_{12}, \dots, x_{1n}\}$ – выборка времени, затраченного на проверку до автоматизации; $X_2 = \{x_{21}, x_{22}, \dots, x_{2n}\}$ – выборка времени, затраченного на проверку после автоматизации.

Нулевая гипотеза (H_0): время, затраченное на проверки до и после автоматизации, не имеет статистически значимой разницы.

Альтернативная гипотеза (H_1): время, затраченное на проверки до и после автоматизации, имеет статистически значимую разницу.

Проведение теста Манна-Уитни:

1. Извлекаем данные о времени выполнения задач сотрудников до и после внедрения автоматизации из тайм-трекинга Jira.

До автоматизации: $X_1 = \{139, 82, \dots, 106\}$, 50 записей, данные представлены в минутах.

После автоматизации: $X_2 = \{99, 68, \dots, 77\}$, 50 записей, данные представлены в минутах.

2. Объединяем, сортируем по возрастанию и ранжируем данные. В таблице 16 в столбце «Значение» жирным шрифтом указаны данные, относящиеся ко второй выборке.

Таблица 16 – Исходные данные (100 записей) для теста Манна-Уитни

Порядковый номер	Значение (минуты)	Ранг
1	68	1,5
2	68	1,5
3	69	3,5
4	69	3,5
5	70	5,5
...		
96	164	95,5
97	165	97
98	167	98
99	171	99,5
100	171	99,5

3. Вычисляем сумму рангов для каждой выборки и подставляем в формулу U-критерий Манна-Уитни:

$$U = n_{X_1} * n_{X_2} + \frac{n(n+1)}{2} - T,$$

где n_{X_1} и n_{X_2} – объёмы выборок; n – объём выборки, имеющей большую ранговую сумму; T – большая сумма рангов из выборок X_1 и X_2 .

Сумма рангов для первой выборки = 2979,5; для второй = 2070,5.

Подставляем в формулу:

$$U_{\text{факт.}} = 50 * 50 + \frac{50(50+1)}{2} - 2979,5 = 795,5.$$

Количество значений в выборках = 50; уровень значимости 0,01. Согласно таблице «Критические значения критерия U Вилкоксона-Манна-Уитни»: критическое значение $U_{\text{крит.}} = 912$.

Так как эмпирическое значение U-критерия меньше критического ($U_{\text{факт.}} < U_{\text{крит.}}$) при уровне значимости $\beta = 1 \%$, то гипотеза H_0 отвергается в пользу H_1 ; время, затраченное на проверки до и после автоматизации, имеет статистически значимую разницу.

Благодаря внедрению модуля Data Quality появилась возможность перераспределить персонал на другие задачи, а также снизить операционные затраты.

6.2.5 Удовлетворенность клиентов

Для оценки влияния модуля DQ на восприятие клиентами точности, скорости и надежности автоматизированной системы по просчётам страховых рисков используется индекс потребительской лояльности (Net Promoter Score, NPS).

В качестве источников данных применяются: проведенное среди клиентов анкетирование до и после внедрения, анализ журналов обращений в техподдержку за те же периоды.

Анкета представлена в виде Google-формы (рисунки 31, 32) с целью сбора контактов респондентов и содержит вопросы, направленные на оценку ключевых аспектов сервиса.



Опрос клиентов

Пожалуйста, уделите 2–3 минуты для заполнения анкеты. Ваши ответы помогут нам улучшить сервис и понять влияние обновлений системы.

***Обязательный вопрос**

С какой вероятностью вы порекомендуете нашу страховую компанию знакомым, друзьям или родственникам? *

0 1 2 3 4 5 6 7 8 9 10

никогда не порекомендую ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ всем порекомендую

Как вы оцениваете точность расчета вашего страхового риска? *

☐ Совсем неточный

☐ Скорее неточный

☐ Средней точности

☐ Скорее точный

☐ Очень точный

Рисунок 31 — Опрос клиентов (1 часть)

Насколько быстро вы получили результат расчета? *

☐ Заняло слишком много времени

☐ Медленно

☐ Умеренно быстро

☐ Быстро

☐ Почти мгновенно

Возникали ли у вас ошибки при заполнении анкеты или формы? *

☐ Да, часто

☐ Иногда

☐ Нет, всё прошло без ошибок

Как бы вы в целом оценили качество обслуживания? *

очень плохо 1 2 3 4 5 отлично

☐ ☐ ☐ ☐ ☐

Отправить Очистить форму

Рисунок 32 — Опрос клиентов (2 часть)

Первый вопрос анкеты предназначен для расчёта NPS. Ответы распределяются по категориям:

- промоутеры: 9-10 баллов;
- нейтралы: 7-8 баллов;
- критики: 0-6 баллов.

Для подсчёта индекса NPS из числа лояльных клиентов-промоутеров вычитается число клиентов-критиков. Получившийся показатель делится на количество опрошенных, а затем умножается на 100 %.

Формула для расчёта NPS:

$$NPS = \frac{\text{количество промоутеров} - \text{количество критиков}}{\text{количество опрошенных}} * 100 \%$$

В анкетировании до/после участвовало 200 человек, результаты представлены в таблице 17.

Таблица 17 – Результаты расчёта NPS

Период	Промоутеры, кол-во человек	Критики, кол-во человек	NPS, индекс
До внедрения	56 (28 %)	82 (41 %)	-13
После внедрения	130 (65 %)	27 (13,5 %)	+51,5

Для проверки статистической значимости изменений в NPS используется Z-критерий для оценки разности между двумя долями признака:

$$Z = \frac{p_1 - p_2}{\sqrt{p(1-p) * \left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}$$

где p_1 и p_2 – доли критиков/промоутеров до и после внедрения модуля;
 $p = (p_1 + p_2)/(n_1 + n_2)$; n_1 и n_2 – количество опрошенных.

Для промоутеров:

$$Z_{\text{пром.}} = \frac{0,28 - 0,65}{\sqrt{0,465(1 - 0,465) * \left(\frac{1}{200} + \frac{1}{200}\right)}} \approx -7,41.$$

Для критиков:

$$Z_{\text{крит.}} = \frac{0,41 - 0,135}{\sqrt{0,2725(1 - 0,2725) * \left(\frac{1}{200} + \frac{1}{200}\right)}} \approx 6,18.$$

Далее просчитается р-значение – это вероятность получить значение статистики, более экстремальное, чем наблюдаемое, при условии истинности нулевой гипотезы. Для этого берётся абсолютное значение z-статистики, вычисляется площадь хвоста справа от этого значения и умножается на 2, чтобы учесть оба хвоста распределения. В Python введём формулу:

```
> p_value = 2 * (1 - norm.cdf(abs(z)))
```


Для $z = -7,41$, получим p – значение $= 1,26 * 10^{-13}$; для $z = 6,18$, получим p – значение $= 6,41 * 10^{-10}$. Обе доли (промоутеров и критиков) изменились статистически значимо ($p < 0,0001$).

Таким образом, внедрение модуля привело к существенному улучшению показателя NPS: с -13 до 51,5. Это указывает на значительное увеличение лояльности клиентов и сокращение доли недовольных. Положительная динамика подтверждается статистической значимостью изменений, что позволяет уверенно говорить, что рост NPS на 64,5 единицы не является случайным и обусловлен добавлением модуля DQ.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель – усовершенствована система оценки страховых рисков за счет разработки и внедрения модуля автоматизированного контроля качества данных. Проведённый анализ актуальных проблем, связанных с качеством данных в страховой отрасли, подтвердил необходимость автоматизации процессов валидации и обоснованность выбранного направления.

Значимость данной работы состояла в повышении надежности скоринговых моделей для страховых продуктов, а также в создании стандартизированного подхода к управлению данными, что соответствует стратегии цифровой трансформации страховых компаний.

Разработанное программное решение обеспечило комплексную проверку входящих котировок на соответствие установленным требованиям: полноту, корректность, согласованность и соответствие структуре эталонной JSON-схемы.

В результате внедрения модуля достигнуты следующие ключевые результаты:

- снижено количество неуспешных проверок, вызванных техническими ошибками во входных данных, примерно на 87 % согласно пункту 6.2.1;
- сокращено среднее время обработки котировок на 92,5-94,5 сек. согласно пункту 6.2.2;
- улучшена точность скоринговых моделей с 0,79 до 0,95 согласно пункту 6.2.3;
- повышена прозрачность и управляемость процессов оценки рисков.

Перспективы дальнейшего развития проекта включают масштабирование на другие виды страховых продуктов, внедрение моделей машинного обучения для автоматического выявления аномалий, развитие отчётности и адаптацию модуля для работы с большими объёмами данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Оксфордские учебные словари. Data // Издательство Оксфордского университета: [сайт]. – 2025. – URL: <https://www.oxfordlearnersdictionaries.com/definition/english/data> (дата обращения: 19.02.2025). – Текст : электронный.
2. Кембриджский словарь. Data // Cambridge Dictionary : [сайт]. – URL: <https://dictionary.cambridge.org/словарь/англо-русский/data> (дата обращения: 20.02.2025). – Текст : электронный.
3. Shannon, C. E. A Mathematical Theory of Communication // Bell System Technical Journal. – 1948. – Vol. 27. – P. 379–423.
4. Logan, R. K. What Is Information?: Why Is It Relativistic and What Is Its Relationship to Materiality, Meaning and Organization // Information. – 2012. – Vol. 3. – P. 68–91.
5. Hewitt, S. M. Data, Information, and Knowledge // Journal of Histochemistry and Cytochemistry. – 2019. – Vol. 67. – P. 227–228.
6. International Organization for Standardization. ISO 9000:2015, Quality Management Systems — Fundamentals and Vocabulary. – 5th ed. – Geneva, Switzerland, 2015.
7. Redman, T. C. Data Quality: The Field Guide. – Boston, MA : Digital Press, 2001.
8. Wang, R. Y.; Strong, D. M. Beyond Accuracy: What Data Quality Means to Data Consumers // Journal of Management Information Systems. – 1996. – Vol. 12. – P. 5–33.
9. Kahn, B. K.; Strong, D. M.; Wang, R. Y. Information Quality Benchmarks: Product and Service Performance // Communications of the ACM. – 2002. – Vol. 45. – P. 184–192.
10. Fürber, C. Data Quality Management with Semantic Technologies. – 1st ed. – Wiesbaden : Springer Gabler, 2015.
11. Википедия. Банковская система СССР : [сайт]. – URL: https://ru.wikipedia.org/wiki/Банковская_система_СССР (дата обращения: 11.04.2025). – Текст : электронный.

12. Компьютерный музей. История банковских систем : [сайт]. – URL: <https://www.computer-museum.ru/histsoft/bank1.htm> (дата обращения: 11.04.2025). – Текст : электронный.
13. Центральный банк Российской Федерации. История : [сайт]. – URL: https://www.cbr.ru/about_br/history/ (дата обращения: 11.04.2025). – Текст : электронный.
14. Kimball, R. The Data Warehouse ETL Toolkit. – Wiley, 2002. – 137 p.
15. TAdviser. Управление качеством данных : [сайт]. – URL: https://www.tadviser.ru/index.php/Статья:Управление_качеством_данных (дата обращения: 12.04.2025). – Текст : электронный.
16. Wijnhoven, F.; Boelens, R.; Middel, R.; Louissen, K. Total Data Quality Management: A Study of Bridging Rigor and Relevance // Proceedings of the Fifteenth European Conference on Information Systems, ECIS 2007. – St. Gallen : University of St. Gallen, 2007. – P. 925–937. – URL: <http://aisel.aisnet.org/ecis2007/15> (дата обращения: 12.04.2025). – Текст : электронный.
17. Банк России. Пресс-релиз : [сайт]. – URL: https://www.cbr.ru/press/pr/?file=070719_1613161.htm (дата обращения: 13.04.2025). – Текст : электронный.
18. Попова, И. А. Обнаружение аномалий в наборе данных с помощью алгоритмов машинного обучения без учителя Isolation Forest и Local Outlier Factor // StudNet. – 2020. – № 12. – URL: <https://cyberleninka.ru/article/n/obnaruzhenie-anomaliy-v-nabore-dannyh-s-pomoschyu-algoritmov-mashinnogo-obucheniya-bez-uchitelya-isolation-forest-i-local-outlier> (дата обращения: 13.04.2025). – Текст : электронный.
19. T1A. Дашборды в аналитике : [сайт]. – URL: <https://t1a.com/resources> (дата обращения: 20.04.2025). – Текст : электронный.
20. ГОСТ Р 50779.10–2000. Статистические методы. Контроль качества. – URL: <https://docs.cntd.ru/document/1200082616> (дата обращения: 16.04.2025). – Текст : электронный.

21. Кузнецов, С. Ценность ваших данных. – Москва : Альпина, 2022. – 574 с. – ISBN 978-5-206-00054-2. – URL: <https://www.ibooks.ru/bookshelf/393885/reading> (дата обращения: 16.04.2025). – Текст : электронный.

22. Европейский союз. Регламент (ЕС) 2016/679 Европейского парламента и Совета от 27 апреля 2016 г. // Official Journal of the European Union. – 2016. – L 119. – Ст. 15–20.

23. Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации» // КонсультантПлюс : [сайт]. – URL: https://www.consultant.ru/document/cons_doc_LAW_186639/ (дата обращения: 16.04.2025). – Текст : электронный.

ПРИЛОЖЕНИЕ А

ЭТАЛОННАЯ JSON-СХЕМА

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "quote": {
      "type": "object",
      "properties": {
        "header": {
          "type": "object",
          "properties": {
            "runId": {
              "type": "string",
              "format": "uuid"
            },
            "quoteId": {
              "type": "string"
            },
            "dateTime": {
              "type": "string",
              "format": "date-time"
            }
          },
          "required": ["runId", "quoteId", "dateTime"]
        },
        "product": {
          "type": "object",
          "properties": {
            "productType": {
              "type": "string",
              "enum": ["ОСАГО", "Страхование жизни"]
            },
            "productCode": {
              "type": "string",
              "enum": ["prod001", "prod002"]
            }
          },
          "required": ["productType", "productCode"]
        },
        "subjects": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "firstName": {
                "type": "string",
                "minLength": 1
              },
              "secondName": {
                "type": "string",
                "minLength": 1
              },
              "middleName": {
                "type": "string",
                "minLength": 1
              },
              "birthDate": {
                "type": "string",
                "format": "date",
                "pattern": "^\\d{4}-\\d{2}-\\d{2}$"
              }
            }
          }
        }
      }
    }
  }
}
```

```

    },
    "gender": {
      "type": "string",
      "enum": ["мужской", "женский"]
    },
    "addresses": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "country": {
            "type": "string"
          },
          "region": {
            "type": "string"
          },
          "city": {
            "type": "string"
          },
          "street": {
            "type": "string"
          },
          "houseNumber": {
            "type": "string"
          },
          "apartmentNumber": {
            "type": "string"
          }
        }
      },
      "required": ["country", "region", "city", "houseNumber"]
    },
    "documents": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "documentType": {
            "type": "string",
            "enum": ["паспорт", "водительское удостоверение"]
          },
          "documentNumber": {
            "type": "string",
            "minLength": 6
          },
          "issueDate": {
            "type": "string",
            "format": "date"
          }
        }
      },
      "required": ["documentType", "documentNumber", "issueDate"]
    }
  },
  "required": ["firstName", "secondName", "birthDate", "gender", "addresses", "documents"]
}
},
"required": ["header", "product", "subjects"]
}
},
"required": ["quote"]
}

```

ПРИЛОЖЕНИЕ Б

СПЕЦИФИКАЦИЯ OPENAPI DATA QUALITY

openapi: 3.0.0

info:

title: Data Quality API

description: |

Управление проверками качества данных и обработка котировок.

Поддерживаются операции включения и выключения проверок, получения метрик, авторизации, а также обработки котировок и управления записями.

version: 1.0.0

servers:

- url: <https://ScoringSystem.com/UI/>

tags:

- name: "quotes"

description: "Операции с котировками"

- name: "metrics"

description: "Операции с метриками качества данных"

- name: "checks"

description: "Операции по управлению проверками"

components:

responses:

UnauthorizedError:

description: Неавторизованный запрос. Проверьте заголовки авторизации.

BadRequest:

description: Ошибка в запросе. Проверьте параметры и структуру данных.

schemas:

QuoteRequest:

type: object

properties:

quote:

type: object

properties:

header:

type: object

properties:

runId:

type: string

format: uuid

quoteId:

type: string

dateTime:

type: string

format: date-time

product:

type: object

properties:

productType:

type: string

productCode:

type: string

subjects:

type: array

items:

type: object

properties:

firstName:

type: string

secondName:


```

    type: string
  middleName:
    type: string
  birthDate:
    type: string
    format: date
  gender:
    type: string
    enum: [мужской, женский]
  addresses:
    type: array
    items:
      type: object
      properties:
        country:
          type: string
        region:
          type: string
        city:
          type: string
        street:
          type: string
        houseNumber:
          type: string
        apartmentNumber:
          type: string
  documents:
    type: array
    items:
      type: object
      properties:
        documentType:
          type: string
          enum: [паспорт, водительское удостоверение]
        documentNumber:
          type: string
        issueDate:
          type: string
          format: date

paths:
  /quotes:
    post:
      summary: Принять котировку
      description: Принимает котировку и выполняет проверки Data Quality.
      operationId: processQuote
      tags:
        - "quotes"
      requestBody:
        description: JSON с данными котировки.
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/QuoteRequest'
      responses:
        '200':
          description: Котировка обработана успешно.
        '400':
          $ref: '#/components/responses/BadRequest'
        '401':
          $ref: '#/components/responses/UnauthorizedError'

```

```

/quotes/{quoteId}:
  delete:
    summary: Удалить котировку
    description: Удаляет котировку по её идентификатору.
    operationId: deleteQuote
    tags:
      - "quotes"
    parameters:
      - name: quoteId
        in: path
        required: true
        description: Уникальный идентификатор котировки.
        schema:
          type: string
    responses:
      '200':
        description: Котировка успешно удалена.
      '400':
        $ref: '#/components/responses/BadRequest'
      '401':
        $ref: '#/components/responses/UnauthorizedError'

/metrics:
  get:
    summary: Получить метрики
    description: Возвращает список доступных метрик Data Quality.
    operationId: getMetrics
    tags:
      - "metrics"
    responses:
      '200':
        description: Успешный ответ с метриками.
        content:
          application/json:
            schema:
              type: array
              items:
                type: string
      '401':
        $ref: '#/components/responses/UnauthorizedError'

/metrics/{metricName}:
  get:
    summary: Получить метрику
    description: Возвращает значение указанной метрики.
    operationId: getMetric
    tags:
      - "metrics"
    parameters:
      - name: metricName
        in: path
        required: true
        description: Название метрики.
        schema:
          type: string
    responses:
      '200':
        description: Успешный ответ с данными метрики.
        content:
          application/json:
            schema:
              type: object
              properties:

```

```

      name:
      type: string
    value:
      type: number
  '400':
    $ref: '#/components/responses/BadRequest'
  '401':
    $ref: '#/components/responses/UnauthorizedError'

```

/metrics/by-date:

```

get:
  summary: Получить все метрики за одну дату
  description: Возвращает все метрики за указанную дату.
  operationId: getAllMetricsForDate
  tags:
    - "metrics"
  parameters:
    - name: date
      in: query
      required: true
      description: Дата в формате YYYY-MM-DD, для которой нужно получить все метрики.
      schema:
        type: string
        format: date
  responses:
    '200':
      description: Успешный ответ с данными всех метрик за указанную дату.
      content:
        application/json:
          schema:
            type: array
            items:
              type: object
              properties:
                name:
                  type: string
                value:
                  type: number
                date:
                  type: string
                  format: date
    '400':
      description: Неверный формат даты.
    '401':
      $ref: '#/components/responses/UnauthorizedError'

```

/metrics/by-date-range:

```

get:
  summary: Получить метрики за диапазон дат
  description: Возвращает метрики для заданного диапазона дат.
  operationId: getMetricsForDateRange
  tags:
    - "metrics"
  parameters:
    - name: startDate
      in: query
      required: true
      description: Начальная дата диапазона в формате YYYY-MM-DD.
      schema:
        type: string
        format: date
    - name: endDate

```

in: query
 required: true
 description: Конечная дата диапазона в формате YYYY-MM-DD.
 schema:
 type: string
 format: date
 responses:
 '200':
 description: Успешный ответ с метриками за указанный диапазон.
 content:
 application/json:
 schema:
 type: array
 items:
 type: object
 properties:
 name:
 type: string
 value:
 type: number
 date:
 type: string
 format: date
 '400':
 description: Неверный формат дат или диапазон дат.
 '401':
 \$ref: '#/components/responses/UnauthorizedError'

/checks/DQ-1:
 put:
 summary: Включить/выключить проверку DQ 1
 description: Управляет статусом проверки DQ 1.
 operationId: ChangeCheck-1Status
 tags:
 - "checks"
 requestBody:
 description: JSON с параметрами проверки.
 required: true
 content:
 application/json:
 schema:
 type: object
 properties:
 enabled:
 type: boolean
 description: Включить (true) или выключить (false).
 responses:
 '200':
 description: Статус проверки обновлён.
 '400':
 \$ref: '#/components/responses/BadRequest'
 '401':
 \$ref: '#/components/responses/UnauthorizedError'

/checks/DQ-2:
 put:
 summary: Включить/выключить проверку DQ 2
 description: Управляет статусом проверки DQ 2.
 operationId: ChangeCheck-2Status
 tags:
 - "checks"
 requestBody:

description: JSON с параметрами проверки.
required: true
content:
 application/json:
 schema:
 type: object
 properties:
 checkName:
 type: string
 description: Название проверки.
 enabled:
 type: boolean
 description: Включить (true) или выключить (false).
responses:
 '200':
 description: Статус проверки обновлён.
 '400':
 \$ref: '#/components/responses/BadRequest'
 '401':
 \$ref: '#/components/responses/UnauthorizedError'

ПРИЛОЖЕНИЕ В

ТИПЫ ДАННЫХ

Таблица В.1 — Принятые в работе типы данных

1	varchar (n)	Представляет строку из переменной длины. С помощью параметра задается максимальное количество символов в строке (для PostgreSQL — до 1 ГБ).
2	smallint	Хранит числа от -32768 до +32767. Занимает 2 байта.
3	bigint	Хранит числа от -9223372036854775808 до +9223372036854775807. Занимает 8 байт.
4	bigserial	Псевдоним для bigint, с автоматическим инкрементом. Занимает 8 байт и может хранить числа от 1 до 9223372036854775807.
5	boolean	Может хранить одно из двух значений: true или false. Вместо true можно указывать следующие значения: TRUE, 't', 'true', 'y', 'yes', 'on', '1'. Вместо false можно указывать следующие значения: FALSE, 'f', 'false', 'n', 'no', 'off', '0'.
6	Timestamp with time zone	Хранит дату, время и добавляет смещение (при выводе использует текущий часовой пояс сессии). Занимает 8 байт. Для дат самое нижнее значение — 4713 г. до н.э., самое верхнее значение — 294276 г. н.э.
7	text	Представляет текст произвольной длины.
8	json	Хранит данные json в текстовом виде.

ПРИЛОЖЕНИЕ Г

ГРАФИК РАСПРЕДЕЛЕНИЯ ХИ-КВАДРАТ

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chi2

# Параметры распределения
df = 1 # степени свободы
alpha = 0.05
critical_value = chi2.ppf(1 - alpha, df) # 3.841
observed_chi2 = 8.99 # наблюдаемое значение

# Создание данных для графика
x = np.linspace(0, 10, 100)
y = chi2.pdf(x, df)

# Построение графика
plt.figure(figsize=(10, 6))
plt.plot(x, y, 'b-', label=f' $\chi^2$ -распределение (df={df})')

# Вертикальные линии: критическое значение и наблюдаемое  $\chi^2$ 
plt.axvline(critical_value, color='red', linestyle='--', label=f'Критическое значение (3.84)')
plt.axvline(observed_chi2, color='green', linestyle='--', label=f'Наблюдаемое  $\chi^2$  (8.99)')

# Подписи
plt.title('Распределение  $\chi^2$  (df=1,  $\alpha=0.05$ )')
plt.xlabel(' $\chi^2$ -статистика')
plt.ylabel('Плотность вероятности')
plt.legend()
plt.grid(True)
plt.show()
```

ПРИЛОЖЕНИЕ Д

КОД МОДУЛЯ DATA QUALITY

```
backend/app/main.py
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from uuid import UUID
from datetime import datetime
import jsonschema
import json
import random
import os
from fastapi import FastAPI, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import datetime
from app.database import get_db
from app import models
from app.routes import analytics
app.include_router(analytics.router)

app = FastAPI()

@app.get("/admin-actions")
def get_admin_actions(
    page: int = Query(1, ge=1),
    per_page: int = Query(10, le=100),
    product_type: Optional[str] = None,
    action_type: Optional[str] = None,
    fio: Optional[str] = None,
    start_date: Optional[datetime] = None,
    end_date: Optional[datetime] = None,
    db: Session = Depends(get_db)
):
    query =
db.query(models.CheckAction).join(models.Product).join(models.Action).outerjoin(models.Check).filter(models.Ch
eckAction.deleted == False)

    if product_type:
        query = query.filter(models.Product.product_type == product_type)
    if action_type:
        query = query.filter((models.Action.action_name + " " + models.Check.type).ilike(f"%{action_type}%"))
    if fio:
        query = query.filter(models.CheckAction.FIO.ilike(f"%{fio}%"))
    if start_date and end_date:
        query = query.filter(models.CheckAction.date.between(start_date, end_date))

    total = query.count()
    results = query.order_by(models.CheckAction.date.desc()).offset((page - 1) * per_page).limit(per_page).all()

    return [
        {
            "id": act.id,
            "fio": act.FIO,
            "product_type": act.product.product_type,
            "action": f"{act.action.action_name} {act.check.type if act.check else ""}.strip()",
            "date": act.date
        }
        for act in results
    ]
```



```

@app.delete("/admin-actions/{action_id}")
def soft_delete_admin_action(action_id: int, db: Session = Depends(get_db)):
    action = db.query(models.CheckAction).filter_by(id=action_id).first()
    if not action:
        raise HTTPException(404, detail="Action not found")
    action.deleted = True
    db.commit()
    return {"status": "success"}

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

SCHEMA_PATH = os.path.join(os.path.dirname(__file__), "schemas", "quote_schema.json")
with open(SCHEMA_PATH, "r", encoding="utf-8") as f:
    SCHEMA = json.load(f)

@app.post("/api/calculate")
async def calculate_rating(quote: dict):
    # DQ 1 Check
    try:
        jsonschema.validate(instance=quote, schema=SCHEMA)
    except jsonschema.exceptions.ValidationError as e:
        raise HTTPException(status_code=422, detail=f"DQ 1 validation failed: {e.message}")

    # DQ 2 Checks
    header = quote["quote"]["header"]
    product = quote["quote"]["product"]
    subjects = quote["quote"].get("subjects", [])

    # DQ 2.1: Product check
    if product["productType"] != "osago" or product["productCode"] != "prod001":
        raise HTTPException(status_code=422, detail="DQ 2.1 failed: Invalid productType or productCode")

    # DQ 2.2: Subjects exist
    if not subjects:
        raise HTTPException(status_code=422, detail="DQ 2.2 failed: No subjects provided")

    # DQ 2.3: At least one document per subject
    for subject in subjects:
        if not subject.get("documents"):
            raise HTTPException(status_code=422, detail="DQ 2.3 failed: Subject missing documents")

    # All checks passed, generate rating

    if score >= 800:
        level = "Очень высокий", "Ответственный заёмщик с высокими шансами на получение кредита на выгодных условиях."
    elif score >= 600:
        level = "Высокий", "Надёжный заёмщик, вероятность одобрения кредита высока."
    elif score >= 200:
        level = "Средний", "Кредит возможен, но на стандартных условиях или с ограничениями."
    else:
        level = "Низкий", "Высокие риски. Вероятность отказа в кредите значительна."

    return {"score": score, "level": level}

async def get_db():
    conn = await asyncpg.connect("postgres://user:password@localhost/dbname")

```

```

try:
    yield conn
finally:
    await conn.close()

class CheckUpdate(BaseModel):
    fio: str
    condition: bool

@app.get("/checks")
async def get_checks(product: str, conn: Annotated[asyncpg.Connection, Depends(get_db)]):
    try:
        records = await conn.fetch(
            "SELECT check_id, condition FROM Check_product_status WHERE product_code = $1",
            product
        )
        return {rec['check_id']: rec['condition'] for rec in records}
    except Exception as e:
        raise HTTPException(500, str(e))

@app.put("/checks/{check_id}")
async def update_check(check_id: str, data: CheckUpdate, conn: Annotated[asyncpg.Connection, Depends(get_db)]):
    try:
        async with conn.transaction():
            await conn.execute(
                "UPDATE Check_product_status SET condition = $1 WHERE check_id = $2 AND product_code = 'prod001'",
                data.condition, check_id
            )
            action_id = 2 if data.condition else 3
            await conn.execute(
                "INSERT INTO Check_actions (FIO, product_code, action_id, check_id, date) VALUES ($1, 'prod001', $2, $3, $4)",
                data.fio, action_id, check_id, datetime.now(timezone.utc)
            )
            return {"status": "success"}
    except Exception as e:
        raise HTTPException(500, str(e))

@app.put("/checks/dq2/disable-all")
async def disable_all_dq2(data: CheckUpdate, conn: Annotated[asyncpg.Connection, Depends(get_db)]):
    try:
        async with conn.transaction():
            await conn.execute(
                "UPDATE Check_product_status SET condition = FALSE WHERE check_id LIKE 'DQ 2%' AND product_code = 'prod001'"
            )
            disabled_checks = await conn.fetch(
                "SELECT check_id FROM Check_product_status WHERE check_id LIKE 'DQ 2%'"
            )
            for check in disabled_checks:
                await conn.execute(
                    "INSERT INTO Check_actions (FIO, product_code, action_id, check_id, date) VALUES ($1, 'prod001', 3, $2, $3)",
                    data.fio, check['check_id'], datetime.now(timezone.utc)
                )
            return {"disabled_count": len(disabled_checks)}
    except Exception as e:
        raise HTTPException(500, str(e))

```

backend/app/models.py

```

from sqlalchemy import Column, Integer, String, Boolean, JSON, TIMESTAMP, DateTime, ForeignKey
from sqlalchemy.dialects.postgresql import VARCHAR

```

```

from .database import Base
from datetime import datetime
from app.db import Base
from sqlalchemy.orm import relationship

class FailedReason(Base):
    __tablename__ = "Failed_reasons"

    id = Column(Integer, primary_key=True)
    run_id = Column(Integer, ForeignKey("Requests.runId"))
    reason = Column(String)
    reason_code = Column(String)
    product_code = Column(String)

class CheckAction(Base):
    __tablename__ = "Check_actions"
    id = Column(Integer, primary_key=True, index=True)
    FIO = Column(String)
    product_code = Column(String, ForeignKey("Products.product_code"))
    action_id = Column(Integer, ForeignKey("Actions.action_id"))
    check_id = Column(String, ForeignKey("Checks.check_id"), nullable=True)
    date = Column(DateTime)
    deleted = Column(Boolean, default=False)

    product = relationship("Product")
    action = relationship("Action")
    check = relationship("Check", lazy="joined", foreign_keys=[check_id])

class Product(Base):
    __tablename__ = "Products"
    product_code = Column(String, primary_key=True)
    product_type = Column(String)

class Action(Base):
    __tablename__ = "Actions"
    action_id = Column(Integer, primary_key=True)
    action_name = Column(String)

class Check(Base):
    __tablename__ = "Checks"
    check_id = Column(String, primary_key=True)
    type = Column(String)

class CheckRecordBase(BaseModel):
    product_type: Optional[str]
    status: str

class CheckRecordOut(CheckRecordBase):
    id: int
    created_at: datetime
    deleted: bool

class Config:
    orm_mode = True

class CheckRecord(Base):
    __tablename__ = "check_records"

    id = Column(Integer, primary_key=True, index=True)
    product_type = Column(String, nullable=True)
    status = Column(String, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)
    deleted = Column(Boolean, default=False)

```

```

class Request(Base):
    __tablename__ = "requests"

    runId = Column(VARCHAR(128), primary_key=True, index=True)
    request = Column(JSON)
    product_code = Column(VARCHAR(7))
    status = Column(String(15))
    date = Column(TIMESTAMP(timezone=True))
    deleted = Column(Boolean, default=False)

```

```

class CheckUpdate(BaseModel):
    condition: bool
    fio: str

```

```

class CheckStatus(BaseModel):
    product_id: str
    check_id: str
    type: Literal["DQ1", "DQ2"]
    enabled: bool

```

```

class CheckAction(BaseModel):
    timestamp: datetime
    user: str
    product_id: str
    check_id: str
    old_status: bool
    new_status: bool

```

frontend/src/components/admin-history/AdminActionsTable.jsx

```

import React from 'react';
import * as React from 'react';
import { useState, useEffect } from 'react';

import { AdminAction } from '../../types/adminActions';
import { ActionDetailsModal } from './ActionDetailsModal';

export function AdminActionsTable({ actions }: { actions: AdminAction[] }) {
    const [selected, setSelected] = useState<AdminAction | null>(null);

    return (
        <div>
            <table className="w-full border-collapse">
                <thead>
                    <tr>
                        <th>№ действия</th>
                        <th>ФИО</th>
                        <th>страховой продукт</th>
                        <th>действие</th>
                        <th>дата</th>
                    </tr>
                </thead>
                <tbody>
                    {actions.map((act) => (
                        <tr key={act.id} className="cursor-pointer hover:bg-gray-100" onClick={() => setSelected(act)}>
                            <td>{act.id}</td>
                            <td>{act.fio}</td>
                            <td>{act.product_type}</td>
                            <td>{act.action}</td>
                            <td>{act.date}</td>
                        </tr>
                    ))}
                </tbody>
            </table>

```

```

    </tbody>
  </table>

  {selected && (
    <ActionDetailsModal
      action={selected}
      onClose={() => setSelected(null)}
    />
  )}
</div>
);
}

```

frontend/src/components/admin-history/AdminDetailsModal.jsx

```

import * as React from 'react';

import { AdminAction } from '../types/adminActions';

export function ActionDetailsModal({ action, onClose }: { action: AdminAction, onClose: () => void }) {
  return (
    <div className="fixed inset-0 bg-black bg-opacity-50 flex justify-center items-center p-4">
      <div className="bg-white p-4 rounded max-w-lg w-full">
        <h2 className="text-lg font-bold mb-2">Детализация действия №{action.id}</h2>
        <p><strong>Продукт:</strong> {action.product_type}</p>
        <p><strong>Дата:</strong> {action.date}</p>
        <p className="mt-2 bg-gray-100 p-2 rounded text-sm">{action.details}</p>
        <button onClick={onClose} className="mt-4 bg-gray-200 px-4 py-2 rounded">Закрыть</button>
      </div>
    </div>
  );
}

```

frontend/src/components/admin-history/AdminFiltersPanel.jsx

```

import * as React from 'react';

export function AdminFiltersPanel({ filters, setFilters }: any) {
  return (
    <div className="flex gap-4 mb-4 flex-wrap">
      <input type="text" placeholder="ФИО" value={filters.fio} onChange={(e) => setFilters((f: any) => ({ ...f, fio: e.target.value })))} className="p-2 border rounded" />
      <select value={filters.product_type} onChange={(e) => setFilters((f: any) => ({ ...f, product_type: e.target.value })))} className="p-2 border rounded">
        <option value="">Все продукты</option>
        <option value="ОСАГО">ОСАГО</option>
        <option value="Страхование жизни">Страхование жизни</option>
      </select>
      <input type="date" onChange={(e) => setFilters((f: any) => ({ ...f, start_date: e.target.value })))} />
      <input type="date" onChange={(e) => setFilters((f: any) => ({ ...f, end_date: e.target.value })))} />
      <button onClick={() => setFilters({})} className="p-2 bg-gray-200 rounded">Сбросить</button>
    </div>
  );
}

```

frontend/src/components/analytics/ChartSection.jsx

```

export function ChartSection({ metrics }) {
  const successPercent = ((metrics.success / (metrics.total || 1)) * 100).toFixed(0);
  const failedPercent = ((metrics.failed / (metrics.total || 1)) * 100).toFixed(0);

  return (

```

```

<div className="flex gap-8 items-center">
  <div className="w-64 h-64 rounded-full border-8 border-gray-600 relative">
    <div
      className="absolute left-0 top-0 w-full h-full rounded-full"
      style={{
        background: `conic-gradient(#10B981 ${successPercent}%, #EF4444 ${failedPercent}%)`,
      }}
    />
    <div className="absolute inset-0 flex items-center justify-center text-lg font-semibold">
      {successPercent}% / {failedPercent}%
    </div>
  </div>
</div>
);
}

```

frontend/src/components/analytics/MetricCards.jsx

```

export function MetricCards({ metrics }) {
  return (
    <div className="grid grid-cols-3 gap-4">
      <div className="p-4 bg-gray-800 rounded-lg">
        <h3>Успешные проверки</h3>
        <p className="text-xl">{metrics.success}</p>
      </div>
      <div className="p-4 bg-gray-800 rounded-lg">
        <h3>Неуспешные проверки</h3>
        <p className="text-xl">{metrics.failed}</p>
      </div>
      <div className="p-4 bg-gray-800 rounded-lg">
        <h3>Всего проверок</h3>
        <p className="text-xl">{metrics.total}</p>
      </div>
    </div>
  );
}

```

frontend/src/components/analytics/TopReasonsList.jsx

```

export function TopReasonsList({ reasons }) {
  return (
    <div className="p-4 bg-gray-800 rounded-lg mt-4">
      <h3 className="text-lg font-semibold mb-2">Топ-5 причин ошибок</h3>
      <ul className="space-y-1">
        {reasons.map((r, i) => (
          <li key={i} className="flex justify-between">
            <span>{r.reason}</span>
            <span className="text-gray-400">{r.reason_count}</span>
          </li>
        ))}
      </ul>
    </div>
  );
}

```

frontend/src/components/check-history/CheckDetailsModal.jsx

```

import * as React from 'react';
import { CheckRecord } from "../../types/checks";

export function CheckDetailsModal({ record, onClose }) {
  if (!record) return null;

```

```

return (
  <div className="fixed inset-0 bg-black bg-opacity-30 flex justify-center items-center">
    <div className="bg-white rounded-xl p-6 max-w-md w-full shadow-xl">
      <h2 className="text-xl font-bold mb-4">Детали проверки</h2>
      <div className="space-y-2 text-sm">
        <div><strong>ID:</strong> {record.id}</div>
        <div><strong>Продукт:</strong> {record.product_type || "-"}</div>
        <div><strong>Статус:</strong> {record.status}</div>
        <div><strong>Дата:</strong> {record.created_at}</div>
      </div>
      <div className="mt-4 text-muted-foreground text-xs">
        Подробности проверки или сообщение об ошибке...
      </div>
      <button onClick={onClose} className="mt-6 px-4 py-2 bg-gray-200 rounded">
        Закрыть
      </button>
    </div>
  </div>
);
}

```

frontend/src/components/check-history/CheckHistoryTable.jsx

```

import React, { useState } from "react";
import { CheckRecord } from "../../types/checks";
import { ConfirmDeleteModal } from "../ConfirmDeleteModal";

export function CheckHistoryTable(props: {
  records: CheckRecord[];
  isAdmin: boolean;
  onDeleteAll: (ids: number[]) => void;
  onShowDetails: (record: CheckRecord) => void;
}) {
  const { records, isAdmin, onDeleteAll, onShowDetails } = props;
  const [selected, setSelected] = useState([] as number[]);
  const [showConfirm, setShowConfirm] = useState(false);

  const toggleSelect = (id: number) => {
    setSelected(selected.includes(id) ? selected.filter(x => x !== id) : [...selected, id]);
  };

  const clearSelection = () => setSelected([]);

  return (
    <div>
      {isAdmin && (
        <div className="mb-2 space-x-2">
          <button
            className="px-3 py-1 bg-gray-100 rounded"
            onClick={clearSelection}
          >
            Снять выделение
          </button>
          <button
            className="px-3 py-1 bg-red-200 rounded disabled:opacity-50"
            disabled={selected.length === 0}
            onClick={() => setShowConfirm(true)}
          >
            Удалить
          </button>
        </div>
      )}
    </div>
  );
}

```

```

<table className="w-full table-auto border border-gray-300">
  <thead className="bg-gray-100">
    <tr>
      {isAdmin && <th className="p-2"></th>}
      <th className="p-2">ID</th>
      <th className="p-2">Продукт</th>
      <th className="p-2">Статус</th>
      <th className="p-2">Дата</th>
    </tr>
  </thead>
  <tbody>
    {records.map((record) => (
      <tr
        key={record.id}
        onClick={() => onShowDetails(record)}
        className="cursor-pointer hover:bg-gray-50"
      >
        {isAdmin && (
          <td
            className="p-2"
            onClick={(e) => e.stopPropagation()}
          >
            <input
              type="checkbox"
              checked={selected.includes(record.id)}
              onChange={() => toggleSelect(record.id)}
            />
          </td>
        )}
        <td className="p-2">{record.id}</td>
        <td className="p-2">{record.productType || "-"}</td>
        <td className="p-2">{record.status}</td>
        <td className="p-2">{record.createdAt}</td>
      </tr>
    )}
  </tbody>
</table>

{showConfirm && (
  <ConfirmDeleteModal
    count={selected.length}
    onConfirm={() => {
      onDeleteAll(selected);
      setSelected([]);
      setShowConfirm(false);
    }}
    onCancel={() => setShowConfirm(false)}
  />
)}
</div>
);
}

```

frontend/src/components/check-history/ConfirmDeleteModal.jsx
import React from "react";

```

export function ConfirmDeleteModal({ count, onConfirm, onCancel }) {
  return (
    <div className="fixed inset-0 bg-black bg-opacity-30 flex justify-center items-center">
      <div className="bg-white rounded-xl p-6 max-w-sm w-full shadow-xl text-center">
        <p className="text-lg">Удалить {count} запись(и)?</p>
        <div className="mt-6 flex justify-center gap-4">

```



```

        <button onClick={onCancel} className="px-4 py-2 bg-gray-200 rounded">Отмена</button>
        <button onClick={onConfirm} className="px-4 py-2 bg-red-500 text-white rounded">Удалить</button>
      </div>
    </div>
  </div>
);
}

```

frontend/src/components/CheckToggle.jsx

```

import React from 'react';

interface Props {
  label: string;
  value: boolean;
  onToggle: (val: boolean) => void;
}

export const CheckToggle = ({ label, value, onToggle }: {
  label: string;
  value: boolean;
  onToggle: (v: boolean) => void;
}) => {
  return (
    <div className="flex items-center justify-between py-1">
      <span>{label}</span>
      <input
        type="checkbox"
        className="accent-green-500 scale-125"
        checked={value}
        onChange={(e) => onToggle(e.target.checked)}
      />
    </div>
  );
};

```

frontend/src/components/CreditRatingForm.jsx

```

import React, { useState } from 'react';

const CreditRatingForm = () => {
  const [form, setForm] = useState({
    productType: "",
    productCode: "",
    lastName: "",
    firstName: "",
    middleName: "",
    address: "",
    birthDate: "",
    gender: "",
    documentType: "",
    documentNumber: "",
    issueDate: ""
  });

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    console.log(form);
  };

```

```

return (
  <form className="credit-rating-form" onSubmit={handleSubmit}>
    <h2>Персональный кредитный рейтинг</h2>

    <section>
      <label>
        Тип продукта<span className="required">*</span>
        <select name="productType" value={form.productType} onChange={handleChange} required>
          <option value="">Выберите</option>
          <option value="ОСАГО">ОСАГО</option>
          <option value="Страхование жизни">Страхование жизни</option>
        </select>
      </label>

      <label>
        Код продукта<span className="required">*</span>
        <input name="productCode" value={form.productCode} onChange={handleChange} required />
      </label>
    </section>

    <section>
      <label>
        Фамилия<span className="required">*</span>
        <input name="lastName" value={form.lastName} onChange={handleChange} required />
      </label>

      <label>
        Имя<span className="required">*</span>
        <input name="firstName" value={form.firstName} onChange={handleChange} required />
      </label>

      <label>
        Отчество
        <input name="middleName" value={form.middleName} onChange={handleChange} />
      </label>

      <label>
        Адрес регистрации<span className="required">*</span>
        <input name="address" value={form.address} onChange={handleChange} required />
      </label>

      <label>
        Дата рождения<span className="required">*</span>
        <input type="date" name="birthDate" value={form.birthDate} onChange={handleChange} required />
      </label>

      <label>
        Пол<span className="required">*</span>
        <select name="gender" value={form.gender} onChange={handleChange} required>
          <option value="">Выберите</option>
          <option value="мужской">Мужской</option>
          <option value="женский">Женский</option>
        </select>
      </label>
    </section>

    <section>
      <label>
        Тип документа<span className="required">*</span>
        <select name="documentType" value={form.documentType} onChange={handleChange} required>
          <option value="">Выберите</option>
          <option value="паспорт">Паспорт</option>
        </select>
      </label>
    </section>
  </form>
)

```

```

        <option value="водительское удостоверение">Водительское удостоверение</option>
    </select>
</label>

<label>
    Серия и номер документа<span className="required">*</span>
    <input name="documentNumber" value={form.documentNumber} onChange={handleChange} required />
</label>

<label>
    Дата выдачи<span className="required">*</span>
    <input type="date" name="issueDate" value={form.issueDate} onChange={handleChange} required />
</label>
</section>

<button type="submit">Получить результат</button>
</form>
);
};

export default CreditRatingForm;

```

```

frontend/src/components/ProductTabs.jsx
import React from "react";

interface Props {
    selected: string;
    onSelect: (code: string) => void;
}

export const ProductTabs = ({ selected, onSelect }) => {
    return (
        <div className="flex justify-center gap-4 mb-6">
            {[ "prod001", "life001" ].map((code) => (
                <button
                    key={code}
                    onClick={() => onSelect(code)}
                    className={`px-4 py-2 rounded-full ${
                        selected === code ? 'bg-indigo-600 text-white' : 'bg-gray-700 text-gray-300'
                    }`}
                >
                    {code === "prod001" ? "ОСАГО" : "Страхование жизни"}
                </button>
            ))}
        </div>
    );
};

```

```

frontend/src/components/pages/AdminActionsPage.jsx
import * as React from 'react';
import { useState, useEffect } from 'react';
import { fetchAdminActions } from '../utils/api';
import { AdminActionsTable } from '../components/admin-history/AdminActionsTable';
import { AdminFiltersPanel } from '../components/admin-history/AdminFiltersPanel';

export function AdminActionsPage() {
    const [actions, setActions] = useState([]);
    const [filters, setFilters] = useState({});

    const load = async () => {

```

```

    const data = await fetchAdminActions(filters);
    setActions(data);
  };

  useEffect(() => {
    load();
  }, [filters]);

  return (
    <div className="p-6 space-y-4 text-white">
      <h1 className="text-2xl font-bold">История действий администратора</h1>
      <AdminFiltersPanel filters={filters} setFilters={setFilters} />
      <AdminActionsTable actions={actions} />
    </div>
  );
}

```

frontend/src/components/pages/AnalyticsPage.jsx

```

import { useEffect, useState } from "react";
import { MetricCards } from "../components/analytics/MetricCards";
import { TopReasonsList } from "../components/analytics/TopReasonsList";
import { ChartSection } from "../components/analytics/ChartSection";
import { getSummary, getTotal, getTopFailures } from "../utils/api";
import { AnalyticsMetrics } from "../types/analytics";

export function AnalyticsPage() {
  const [selectedProduct, setSelectedProduct] = useState("ОСАГО");
  const [startDate, setStartDate] = useState("2025-06-01");
  const [endDate, setEndDate] = useState("2025-06-23");
  const [metrics, setMetrics] = useState<AnalyticsMetrics>({ success: 0, failed: 0, total: 0, topReasons: [] });

  useEffect(() => {
    const fetchData = async () => {
      const summary = await getSummary(startDate, endDate);
      const total = await getTotal(startDate, endDate);
      const reasons = await getTopFailures(startDate, endDate);

      const productSummary = summary.find((s) => s.product_type === selectedProduct);
      const totalCount = total.find((t) => t.product_type === selectedProduct)?.total || 0;
      const topReasons = reasons.filter((r) => r.product_type === selectedProduct);

      setMetrics({
        success: productSummary?.successful_checks || 0,
        failed: productSummary?.unsuccessful_checks || 0,
        total: totalCount,
        topReasons: topReasons,
      });
    };
    fetchData();
  }, [selectedProduct, startDate, endDate]);

  return (
    <div className="p-4 space-y-4 text-white">
      <div className="flex gap-4 mb-4">
        <button onClick={() => setSelectedProduct("ОСАГО")}>ОСАГО</button>
        <button onClick={() => setSelectedProduct("Страхование жизни")}>Страхование жизни</button>
      </div>
      <div className="ml-auto flex gap-2">
        <input type="date" value={startDate} onChange={(e) => setStartDate(e.target.value)} />
        <span>—</span>
        <input type="date" value={endDate} onChange={(e) => setEndDate(e.target.value)} />
      </div>
    </div>
  );
}

```

```

    <MetricCards metrics={metrics} />
    <ChartSection metrics={metrics} />
    <TopReasonsList reasons={metrics.topReasons} />
  </div>
);
}

```

frontend/src/components/pages/ChecksHistory.jsx

```

import React, { useEffect, useState } from "react";
import { CheckRecord } from "../../types/checks";
import { fetchCheckRecords, deleteCheckRecords } from "../../utils/api";
import { CheckHistoryTable } from "../../components/check-history/CheckHistoryTable";
import { CheckDetailsModal } from "../../components/check-history/CheckDetailsModal";
import { ConfirmDeleteModal } from "../../components/check-history/ConfirmDeleteModal";

export function ChecksHistory() {
  const [records, setRecords] = useState([] as any[]);
  const [filtered, setFiltered] = useState([] as any[]);
  const [selectedProduct, setSelectedProduct] = useState("");
  const [selectedStatus, setSelectedStatus] = useState("");
  const [selectedRecord, setSelectedRecord] = useState(null as any);
  const [toDelete, setToDelete] = useState(null as number[] | null);
  const isAdmin = true;

  const load = async () => {
    const data = await fetchCheckRecords();
    setRecords(data);
  };

  const applyFilters = () => {
    let data = [...records];
    if (selectedProduct) {
      data = data.filter((r) => r.product_type === selectedProduct);
    }
    if (selectedStatus) {
      data = data.filter((r) => r.status === selectedStatus);
    }
    setFiltered(data);
  };

  const handleDelete = async (ids: number[]) => {
    await deleteCheckRecords(ids);
    await load();
  };

  useEffect(() => {
    load();
  }, []);

  useEffect(() => {
    applyFilters();
  }, [records, selectedProduct, selectedStatus]);

  const productOptions = [...new Set(records.map((r) => r.product_type))];
  const statusOptions = [...new Set(records.map((r) => r.status))];

  return (
    <div className="p-4 space-y-4">
      <h1 className="text-2xl font-bold">История проверок</h1>

      { /* Фильтры */ }
    </div>
  );
}

```

```

<div className="flex gap-4 flex-wrap">
  <select
    value={selectedProduct}
    onChange={(e) => setSelectedProduct(e.target.value)}
    className="border p-2 rounded"
  >
    <option value="">Все продукты</option>
    {productOptions.map((p) => (
      <option key={p} value={p}>
        {p}
      </option>
    ))}
  </select>
  <select
    value={selectedStatus}
    onChange={(e) => setSelectedStatus(e.target.value)}
    className="border p-2 rounded"
  >
    <option value="">Все статусы</option>
    {statusOptions.map((s) => (
      <option key={s} value={s}>
        {s}
      </option>
    ))}
  </select>
  <button
    className="bg-gray-200 px-4 py-2 rounded"
    onClick={() => {
      setSelectedProduct("");
      setSelectedStatus("");
    }}
  >
    Сбросить
  </button>
</div>

{/* Таблица */}
<CheckHistoryTable
  records={filtered}
  isAdmin={isAdmin}
  onShowDetails={setSelectedRecord}
  onDeleteAll={(ids) => setToDelete(ids)}
/>

{/* Модалки */}
{selectedRecord && (
  <CheckDetailsModal record={selectedRecord} onClose={() => setSelectedRecord(null)} />
)}
{toDelete && (
  <ConfirmDeleteModal
    count={toDelete.length}
    onConfirm={() => {
      handleDelete(toDelete);
      setToDelete(null);
    }}
    onCancel={() => setToDelete(null)}
  />
)}
</div>
);
}

```

```

frontend/src/components/pages/ChecksManager.jsx
import React, { useEffect, useState } from 'react';
import { getChecks, updateCheckStatus, disableAllDQ2 } from '../utils/api';
import { CheckToggle } from '../components/CheckToggle';
import { ProductTabs } from '../components/ProductTabs';
import { CheckStatus } from '../types/checks';

export const ChecksManager = () => {
  const [productCode, setProductCode] = useState('prod001');
  const [checks, setChecks] = useState({} as Record<string, boolean>);

  useEffect(() => {
    getChecks(productCode).then(setChecks);
  }, [productCode]);

  const updateCheck = async (id: string, value: boolean) => {
    await updateCheckStatus(id, { fio, condition: value });
    setChecks((prev) => ({ ...prev, [id]: value }));
  };

  const disableDQ2 = async () => {
    await disableAllDQ2({ fio });
    const newChecks = { ...checks };
    Object.keys(newChecks).forEach((id) => {
      if (id.startsWith('DQ 2')) newChecks[id] = false;
    });
    setChecks(newChecks);
  };

  return (
    <div className="p-6 text-white">
      <ProductTabs selected={productCode} onSelect={setProductCode} />
      <div className="bg-gray-800 rounded-xl p-4 space-y-6">
        <div>
          <h2 className="text-xl font-semibold mb-2">ПРОВЕРКА DQ 1</h2>
          <CheckToggle
            label="Проверка по JSON-схеме"
            value={checks['DQ 1'] ?? false}
            onToggle={(v) => updateCheck('DQ 1', v)}
          />
        </div>

        <div>
          <h2 className="text-xl font-semibold mb-2">ПРОВЕРКИ DQ 2</h2>
          <button
            className="text-sm text-red-400 underline mb-2"
            onClick={disableDQ2}
          >
            ВЫКЛЮЧИТЬ ВСЕ DQ 2
          </button>
          {Object.entries(checks)
            .filter(([id]) => id.startsWith('DQ 2'))
            .map(([id, val]) => {
              const checked = val as boolean;
              return (
                <div key={id}>
                  <CheckToggle
                    label={id}
                    value={checked}
                    onToggle={(v) => updateCheck(id, v)}
                  />
                </div>
              );
            })
          };
        </div>
      </div>
    </div>
  );
};

```

```

    )))
  </div>
</div>
</div>
);
};

```

frontend/src/components/types/adminAction.jsx

```

export type AdminAction = {
  id: number;
  fio: string;
  product_type: string;
  action: string;
  date: string;
  details: string;
};

```

frontend/src/components/types/analytics.jsx

```

export type FailureReason = {
  reason: string;
  reason_count: number;
  product_type: string;
};

export type AnalyticsMetrics = {
  success: number;
  failed: number;
  total: number;
  topReasons: FailureReason[];
};

```

frontend/src/components/types/checks.jsx

```

export interface CheckStatus {
  check_id: string;
  condition: boolean;
}

export interface CheckUpdate {
  fio: string;
  condition: boolean;
}

export interface CheckRecord {
  id: number;
  productType: string;
  status: 'пройдена' | 'не пройдена';
  date: string;
  quoteData: Record<string, any>;
  details: string;
  createdAt: string;
}

```

frontend/src/components/utils/api.ts

```

export async function getChecks(productCode: string) {
  const res = await fetch(`/checks?product=${productCode}`);
  return await res.json();
}

```



```

export async function updateCheckStatus(checkId: string, payload: { fio: string, condition: boolean }) {
  return await fetch(`/checks/${checkId}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
  });
}

export async function disableAllDQ2(payload: { fio: string }) {
  return await fetch('/checks/dq2/disable-all', {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
  });
}

export const fetchProducts = async () => {
  const res = await fetch('/api/products');
  return res.json();
};

export const fetchCheckStatuses = async (productId: string) => {
  const res = await fetch(`/api/checks/status/${productId}`);
  return res.json();
};

export async function fetchCheckRecords() {
  const res = await fetch("/api/checks");
  return res.json();
}

export async function deleteCheckRecords(ids: number[]) {
  await fetch("/api/checks", {
    method: "DELETE",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(ids),
  });
}

export async function fetchAdminActions(filters = {}) {
  const query = new URLSearchParams(filters as any).toString();
  const res = await fetch(`/admin-actions?${query}`);
  return await res.json();
}

export async function getSummary(start: string, end: string) {
  const res = await fetch(`/analytics/summary?start_date=${start}&end_date=${end}`);
  return await res.json();
}

export async function getTotal(start: string, end: string) {
  const res = await fetch(`/analytics/total?start_date=${start}&end_date=${end}`);
  return await res.json();
}

export async function getTopFailures(start: string, end: string) {
  const res = await fetch(`/analytics/top-failures?start_date=${start}&end_date=${end}`);
  return await res.json();
}

```

SQL-скрипт для инициализации базы данных PostgreSQL:

-- Справочник всех действий в системе

```

CREATE TABLE actions (
    action_id SMALLINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    action_name VARCHAR(50) NOT NULL
);
INSERT INTO actions (action_name) VALUES
('удаление записи'),
('включение проверки'),
('выключение проверки');

-- Информация о страховых продуктах
CREATE TABLE products (
    product_code VARCHAR(7) PRIMARY KEY,
    product_type VARCHAR(50) NOT NULL
);
INSERT INTO products (product_code, product_type) VALUES
('prod001', 'ОСАГО'),
('prod002', 'Страхование жизни');

-- Справочник типов проверок
CREATE TABLE checks (
    check_id SMALLINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    type VARCHAR(8) NOT NULL,
    check_name VARCHAR(100) NOT NULL
);
INSERT INTO checks (type, check_name) VALUES
('DQ 1', 'Проверка поступившего JSON по схеме'),
('DQ 2.1', 'Проверка типа продукта'),
('DQ 2.2.1', 'Проверка возраста субъекта'),
('DQ 2.2.2', 'Проверка пола субъекта'),
('DQ 2.3', 'Проверка типа документа');

-- Состояние проверок по страховым продуктам
CREATE TABLE check_product_status (
    id SMALLINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    product_code VARCHAR(7) REFERENCES products(product_code),
    check_id SMALLINT REFERENCES checks(check_id),
    condition BOOLEAN NOT NULL DEFAULT TRUE
);

-- Информация о запросах на проверку данных котировок
CREATE TABLE requests (
    runId VARCHAR(128) PRIMARY KEY,
    request JSON NOT NULL,
    product_code VARCHAR(7) REFERENCES products(product_code),
    status VARCHAR(15) NOT NULL,
    date TIMESTAMPTZ NOT NULL,
    deleted BOOLEAN NOT NULL DEFAULT FALSE
);

-- Журнал действий администратора над запросами (удаление записей)
CREATE TABLE request_actions (
    id BIGSERIAL PRIMARY KEY,
    FIO VARCHAR(100) NOT NULL,
    runId VARCHAR(128) REFERENCES requests(runId),
    action_id SMALLINT REFERENCES actions(action_id),
    date TIMESTAMPTZ NOT NULL
);

-- Справочник возможных результатов проверки
CREATE TABLE responses (
    response_code SMALLINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    response_name VARCHAR(15) NOT NULL,
    description_response TEXT

```

```

);
INSERT INTO responses (response_name, description_response) VALUES
('пройдена', NULL),
('не пройдена', 'Расчет скорингового балла по данному страховому продукту не предусмотрен в системе'),
('не пройдена', 'Клиент не достиг совершеннолетия'),
('не пройдена', 'Возраст клиента выше допустимого значения'),
('не пройдена', 'Выберите пол: мужской/женский'),
('не пройдена', 'Данный тип документа не подходит для расчета скорингового балла');

-- Результаты проверок данных котировок
CREATE TABLE request_response (
    id BIGSERIAL PRIMARY KEY,
    runId VARCHAR(128) REFERENCES requests(runId),
    response_code SMALLINT REFERENCES responses(response_code)
);

-- Журнал действий администратора над проверками (включение/выключение проверок)
CREATE TABLE check_actions (
    id BIGSERIAL PRIMARY KEY,
    FIO VARCHAR(100) NOT NULL,
    product_code VARCHAR(7) REFERENCES products(product_code),
    action_id SMALLINT REFERENCES actions(action_id),
    check_id SMALLINT REFERENCES checks(check_id),
    date TIMESTAMPTZ NOT NULL,
    deleted BOOLEAN DEFAULT FALSE
);

```